

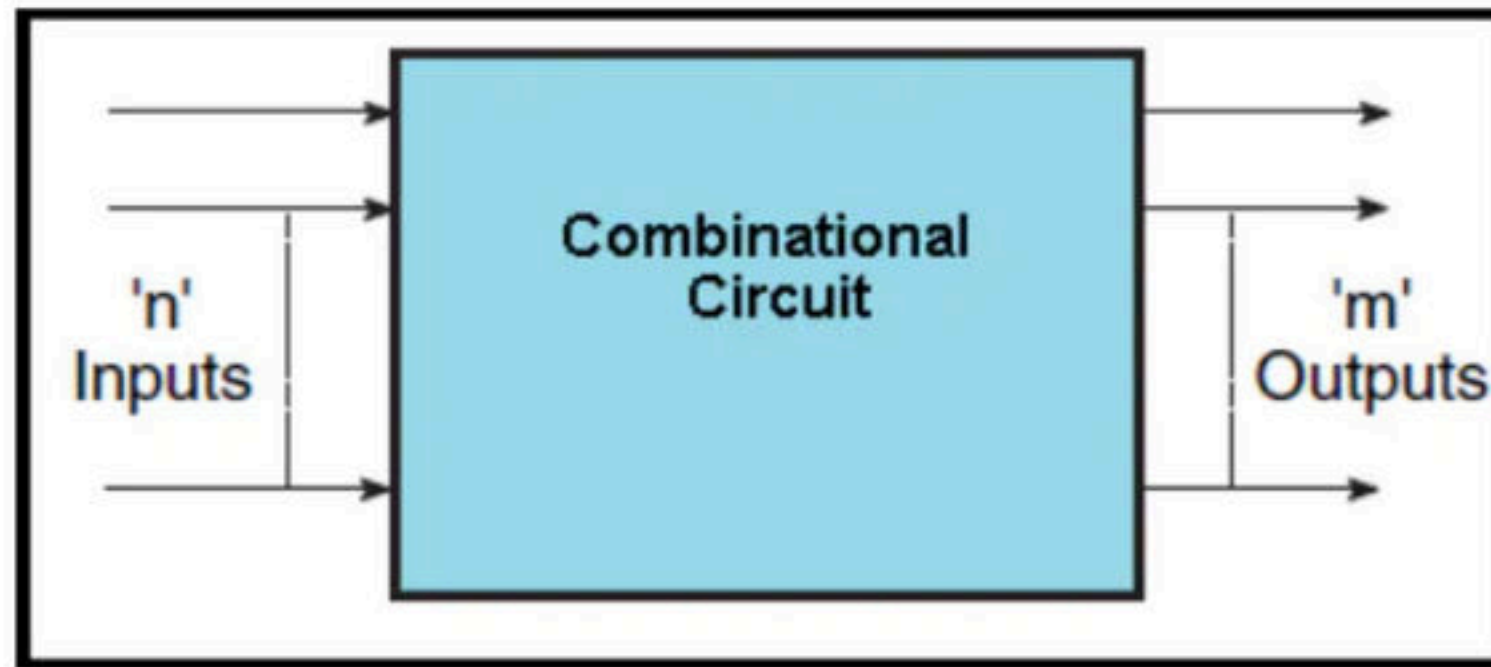


Combinational Circuits Part - III

Foundation Course on Digital Electronics

Combinational Circuits

- When logic gates are connected together to produce a specified output on certain specified combinations of input variables, with no memory involved, then the resulting circuit is called a combinational circuit.
- Output depends only on present input. $O/p = f(i/p)$, combinational circuit performs an operation that can be specified logically by a set of Boolean function.
- A combinational circuit may have n-binary inputs and m-binary outputs.



- Application, Practical computer circuits normally contain combinational and sequential logic. For e.g. the part of ALU, that does mathematical calculations is constructed using combinational logic.
- Other circuits such as half adders, full adders, half subtractors, full subtractors, multiplexers, demultiplexers, encoders and decoders are also made by using combinational logic.

Design procedure: -

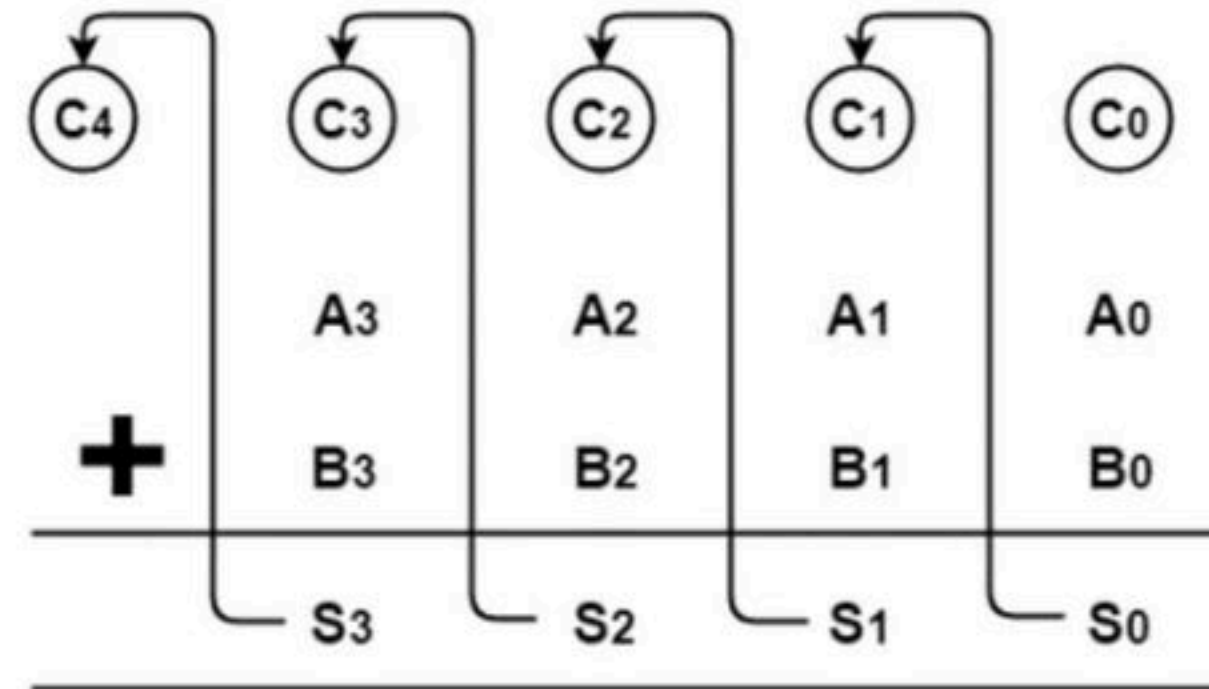
1. Analyse the given problem and identify the number of i/p and o/p variables.
2. Write the truth table based on the specification of the problem.
3. Convert the truth table in minimized Boolean expression using k-map.
4. Draw the logic circuit for the above obtained output expression.

Adder

1. An **adder** is a digital combinational circuit that performs addition of numbers. Are used in the arithmetic logic units or ALU.
2. They are also utilized in other parts of the processor, where they are used to calculate addresses, table indices, increment and decrement operators, and similar operations.
3. Although adders can be constructed for many number representations, such as binary-coded decimal or excess-3, the most common adders operate on binary numbers.
4. In cases where two's complement or ones' complement is being used to represent negative numbers, it is trivial to modify an adder into an adder–subtractor.

Why we need a half adder

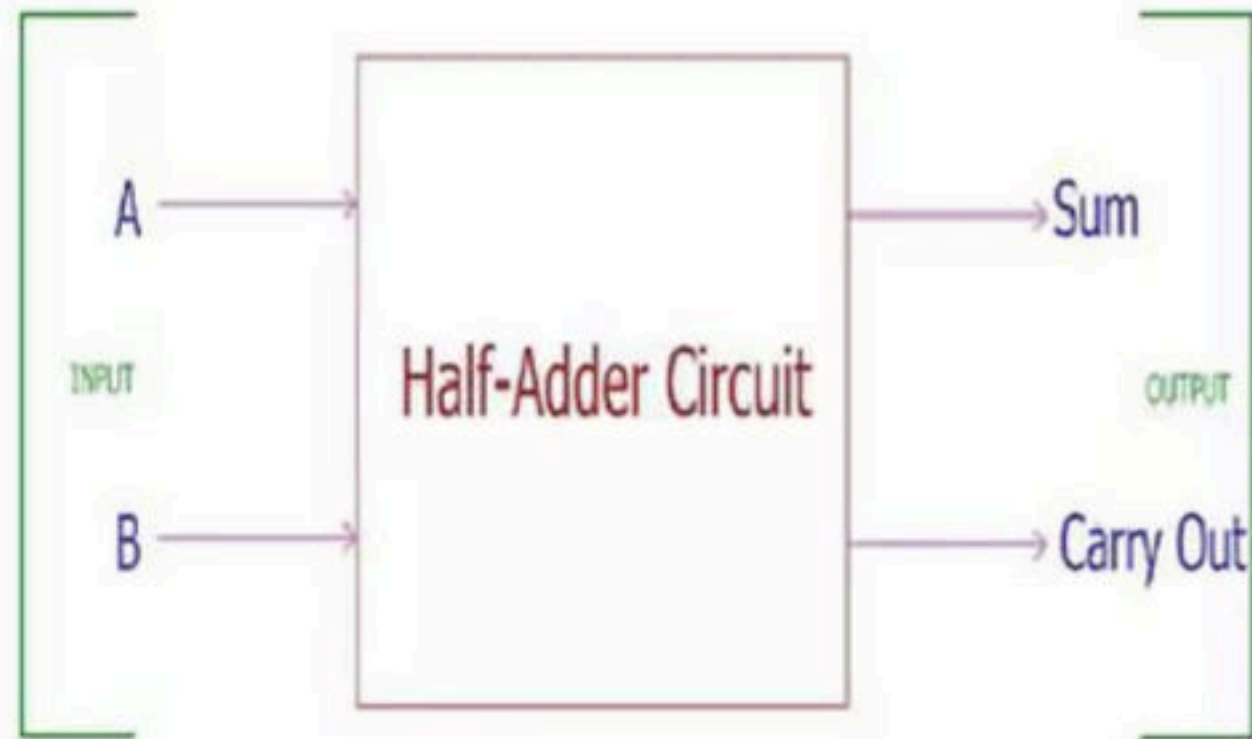
- A binary adder-subtractor is a combinational circuit that performs the arithmetic operations of addition and subtraction with binary numbers.
- We will develop this circuit by means of a hierarchical design. The half adder design is carried out first, from which we develop the full adder.
- Connecting n full adders in cascade produces a binary adder for two n -bit numbers.
- **Basics of addition:** - $(A)_x + (B)_x = ?$



Half adder

- The simplest form of addition is addition of two binary digits, consists of four possible elementary operations
- $0+0 = 0$
- $0+1 = 1$
- $1+0 = 1$
- $1+1 = 10$
- The first three operations produce a sum of two digits, but when both augend and addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a carry.

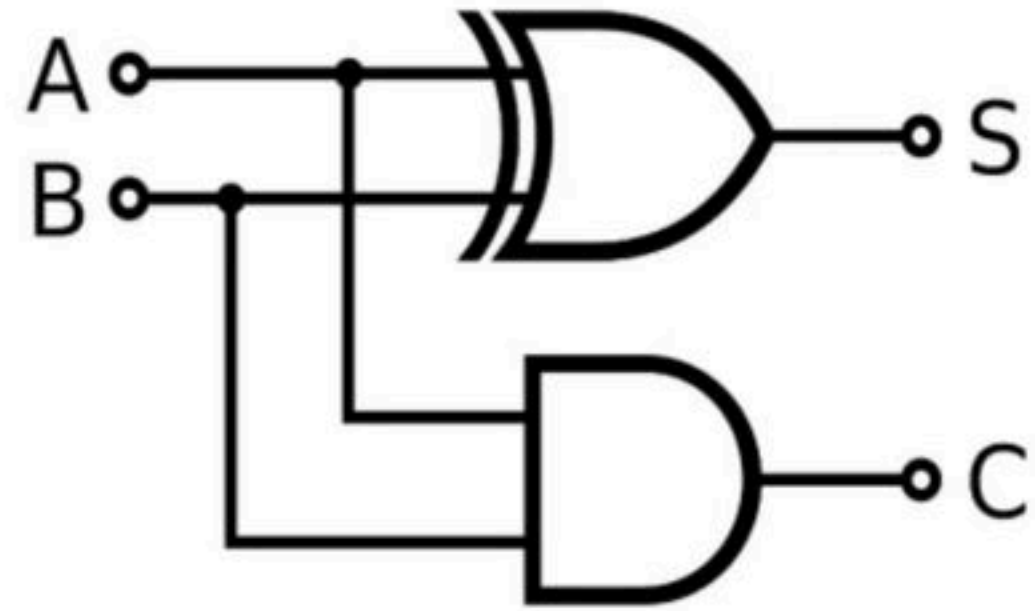
- It is a combinational circuit, which perform the arithmetic addition of two one-bit binary numbers is referred to as an half-adder.
- So, in half adder inputs are adds two single binary bits A and B , and two outputs, sum (S) and carry (C).



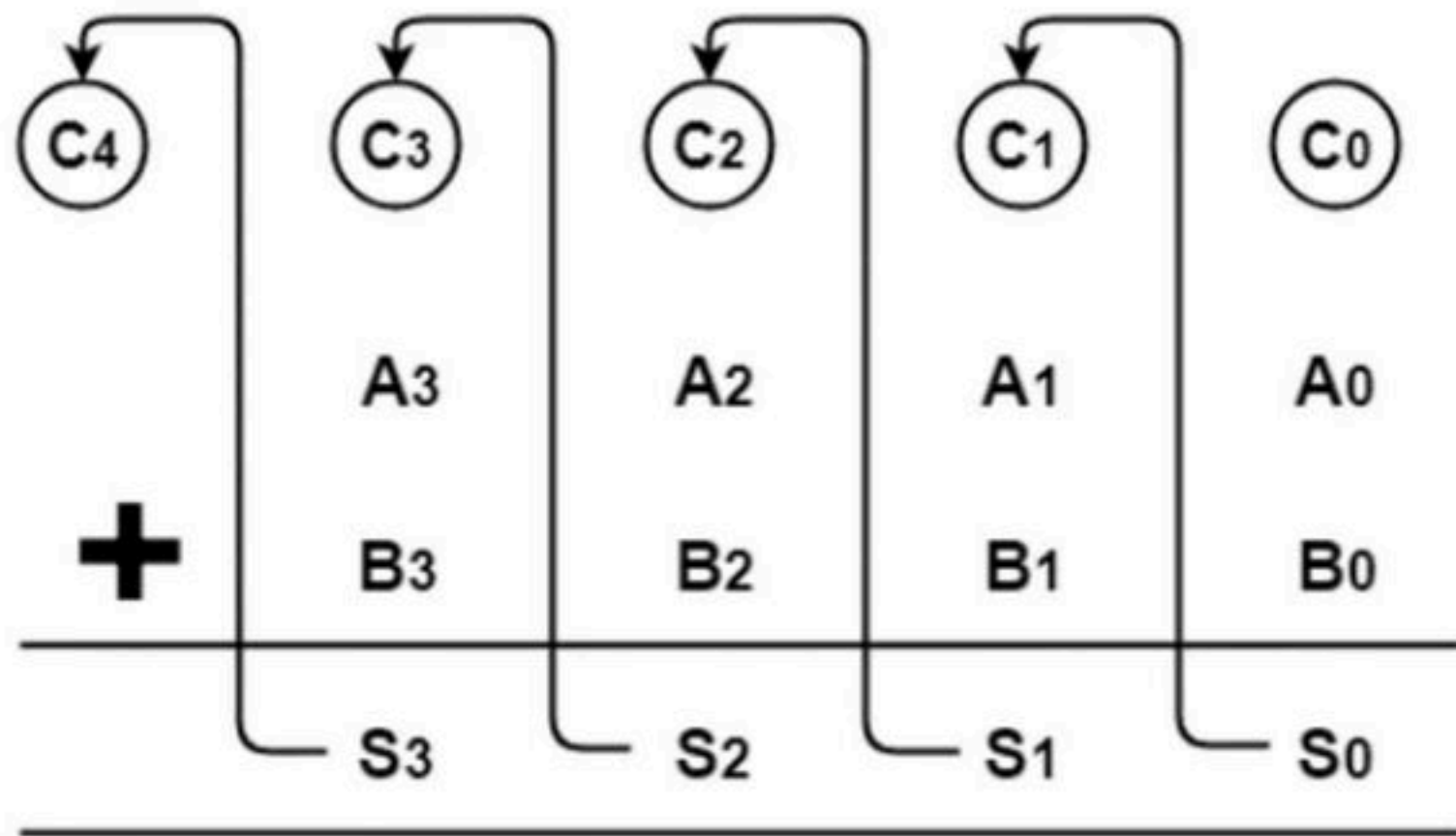
INPUTS		OUTPUTS	
A	B	Sum	Carry
0	0		
0	1		
1	0		
1	1		

Truth table for Half adder

INPUTS		OUTPUTS	
A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



1. Cost of implementation a half adder is one EX-OR gate and one AND gate.
2. A half adder has only two inputs and there is no provision to add a carry coming from the lower order bits when multi bit number addition is performed. For this reason, we have designed a full adder.



$A(A_3 A_2 A_1 A_0)$
 $B(B_3 B_2 B_1 B_0)$

Q A half-adder is also known as: **(NET-DEC-2005)**

(A) AND Circuit

(B) NAND Circuit

(C) NOR Circuit

(D) EX-OR Circuit

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

Full adder

1. A full adder is a combinational logic circuit that performs the arithmetic sum of three input bits.
2. Where A_n , B_n are the n^{th} order bits of the number A and B respectively and C_n is the carry generated from the addition of $(n-1)^{\text{th}}$ order bits.
3. It consists of three input bits, denoted by A (First operand), B (Second operand), C_{in} (Represents carry from the previous lower significant position).

- Two output bits are same as of half adder, which is Sum and Carry_{out}.
- When the augend and addend number contain more significant digits, the carry obtained from the addition of two bits is added to the next higher order pair of significant bits.



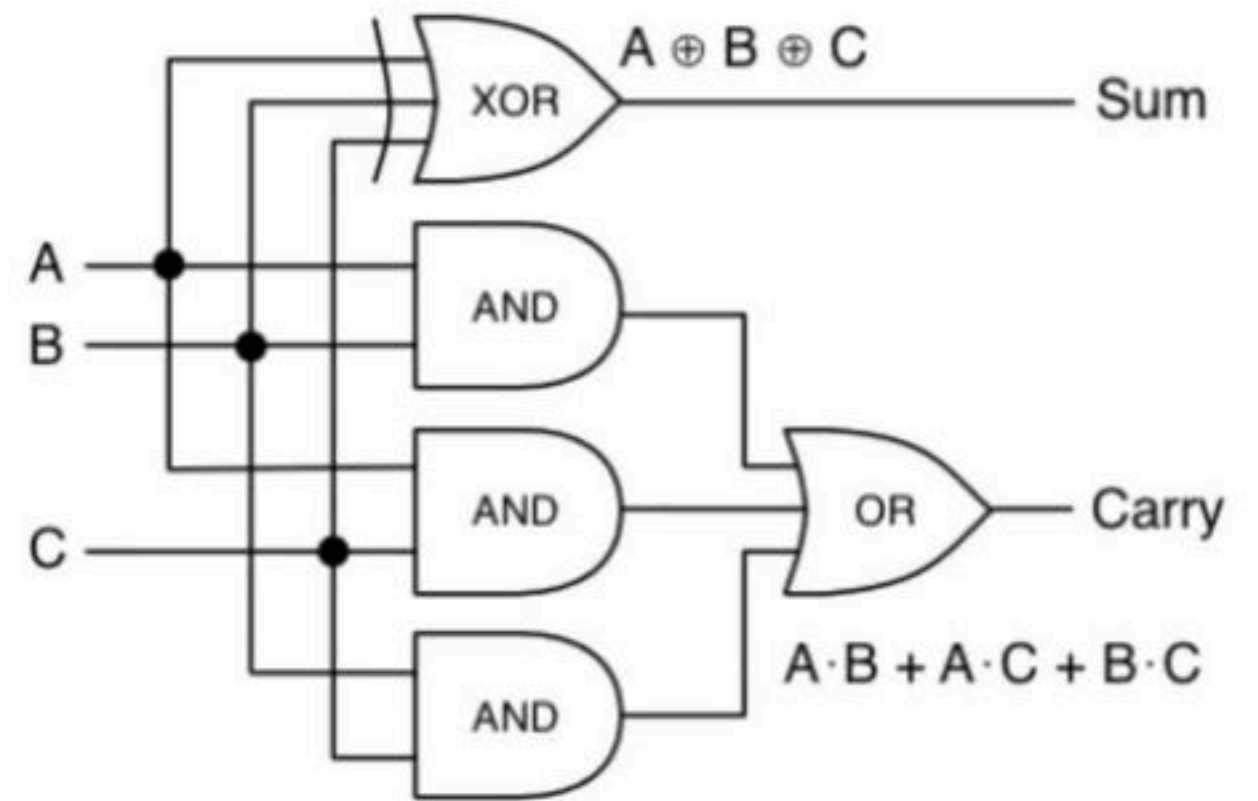
INPUTS			OUTPUTS	
A	B	C _{in}	Sum	C _{out}
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

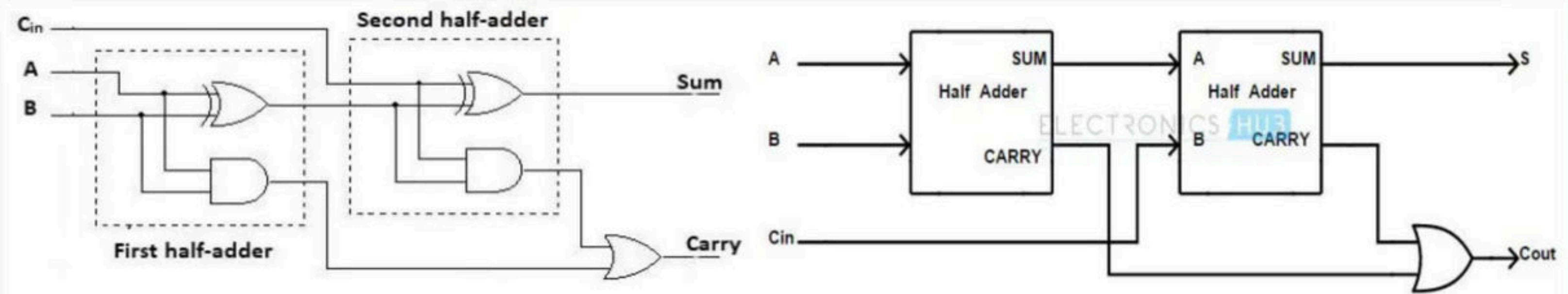
INPUTS			OUTPUTS	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

	ab	$a'b'$	$a'b$	ab	ab'
C_{in}		00	01	11	10
C_{in}'	0	0	2	6	4
C_{in}	1	1	3	7	5

	ab	$a'b'$	$a'b$	ab	ab'
C_{in}		00	01	11	10
C_{in}'	0	0	2	6	4
C_{in}	1	1	3	7	5

INPUTS			OUTPUTS	
A	B	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





Q Which of the following statements are true? **(NET-DEC-2013)**

- I.** A circuit that adds two bits, producing a sum bit and a carry bit is called half adder. ✓
- II.** A circuit that adds two bits, producing a sum bit and a carry bit is called full adder. ✗
- III.** A circuit that adds two bits and a carry bit producing a sum bit and a carry bit is called full adder. ✓
- IV.** A device that accepts the value of a Boolean variable as input and produces its complement is called an inverter. ✓

(A) I & II

(B) II & III

(C) I, II, III

(D) I, III & IV ✓

Q Consider a full - adder with the following input values:

(1) $x = 1$, $y = 0$ and $C_i(\text{carry input}) = 0$ $S = 1$ $C_o = 0$

(2) $x = 0$, $y = 1$ and $C_i = 1$ $S = 0$ $S = 1$

Compute the values of S(sum) and C_o (carry output) for the above input values. (NET-JUNE-2015)

(a) $S = 1, C_o = 0$ and $S = 0, C_o = 1$

(b) $S = 0, C_o = 0$ and $S = 1, C_o = 1$

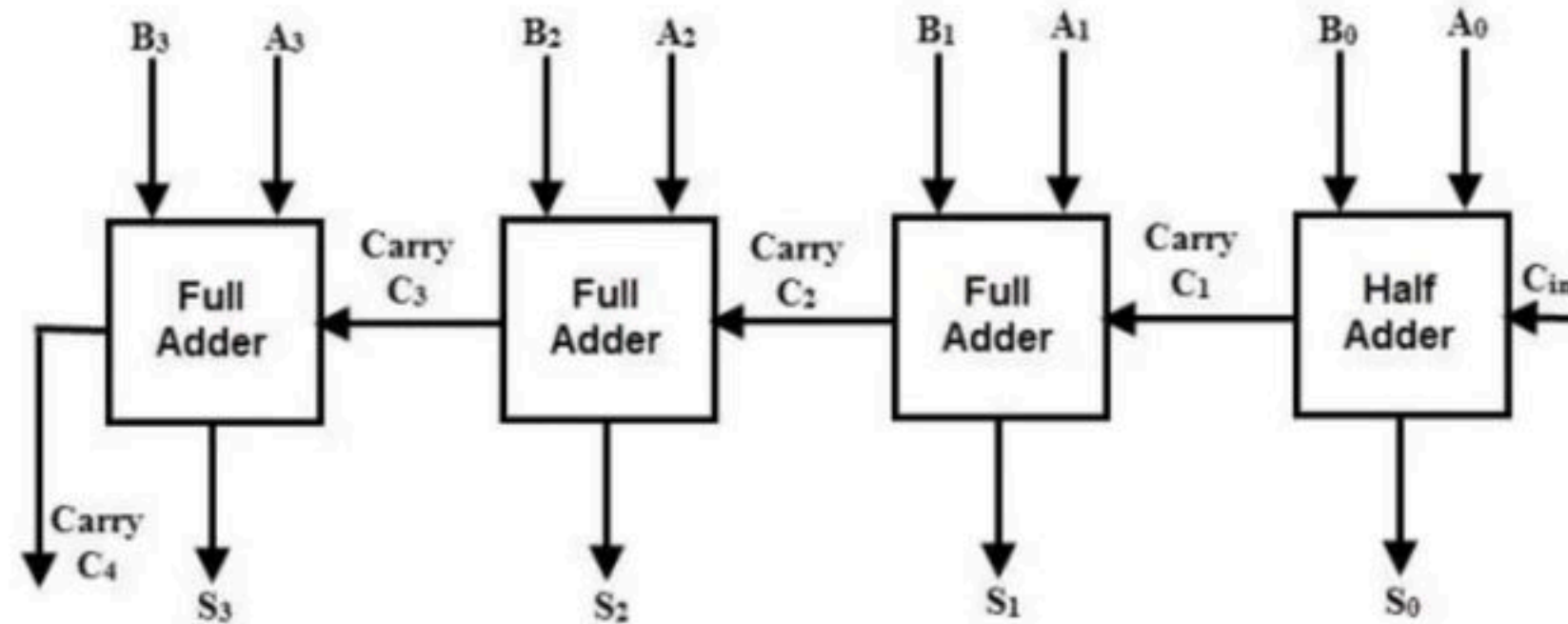
(c) $S = 1, C_o = 1$ and $S = 0, C_o = 0$

(d) $S = 0, C_o = 1$ and $S = 1, C_o = 0$

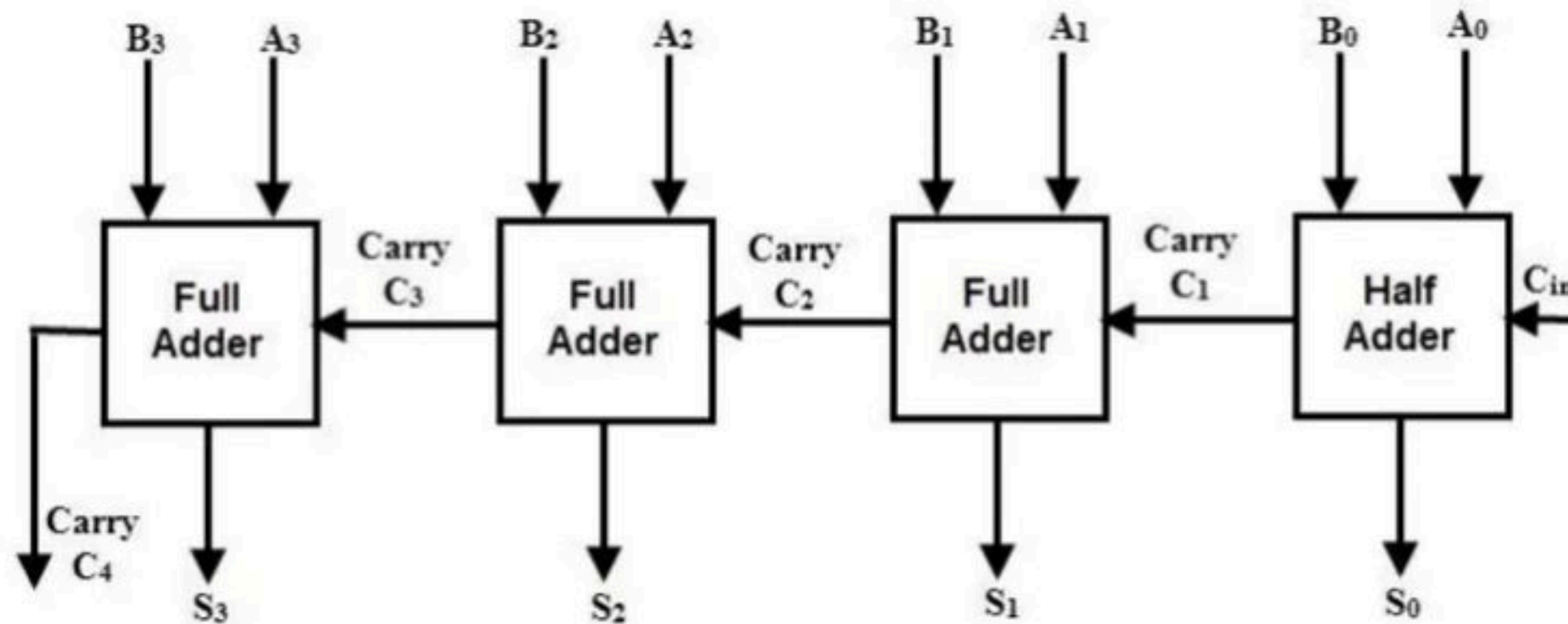


Four-bit parallel binary adder / Ripple adder

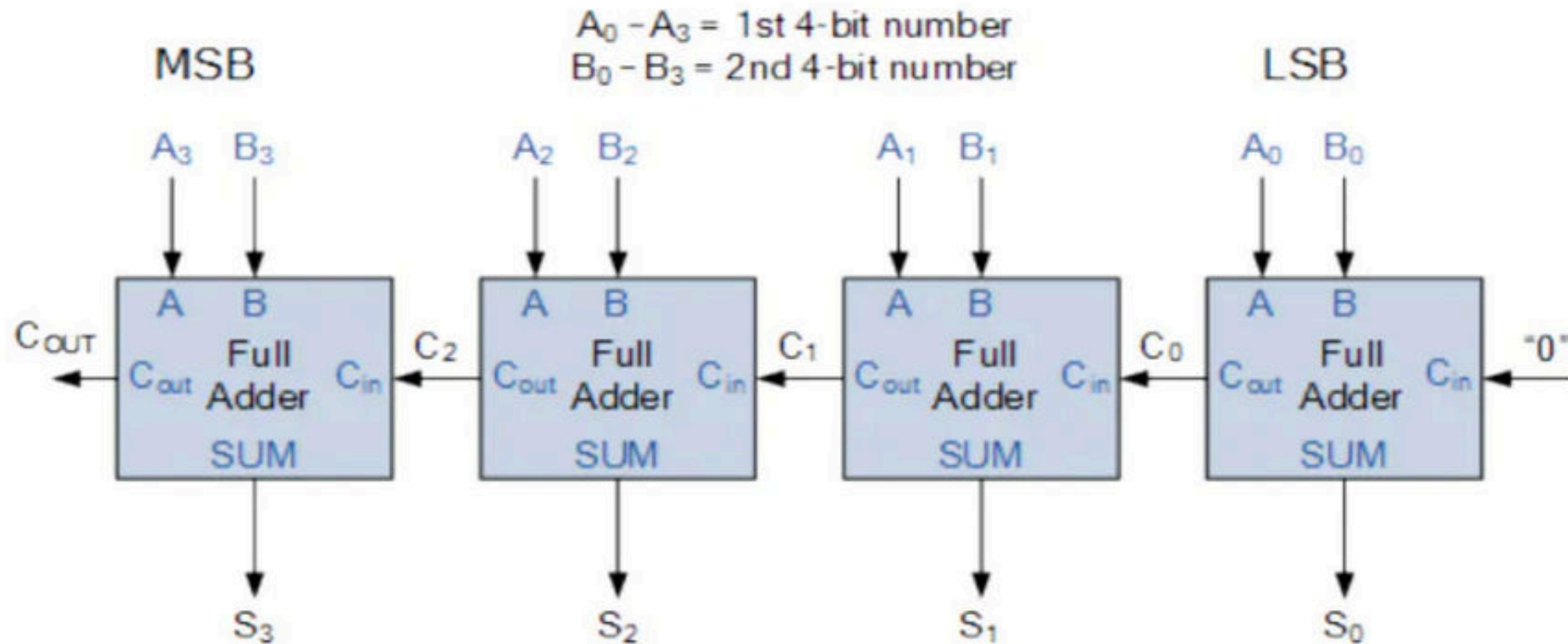
- As we know that full adder is capable of adding two 1 bit number and 1 previous carry, but in order to add binary numbers with more than one bits, additional full adders must be employed. For e.g. a four bit binary adder can be constructed using four full adders.
- These four full adders are connected in cascade, carry output of each adder is connected to the carry input of the next higher-order adder.
- So a n-bit parallel adder is constructed using 'n' number of full adders.



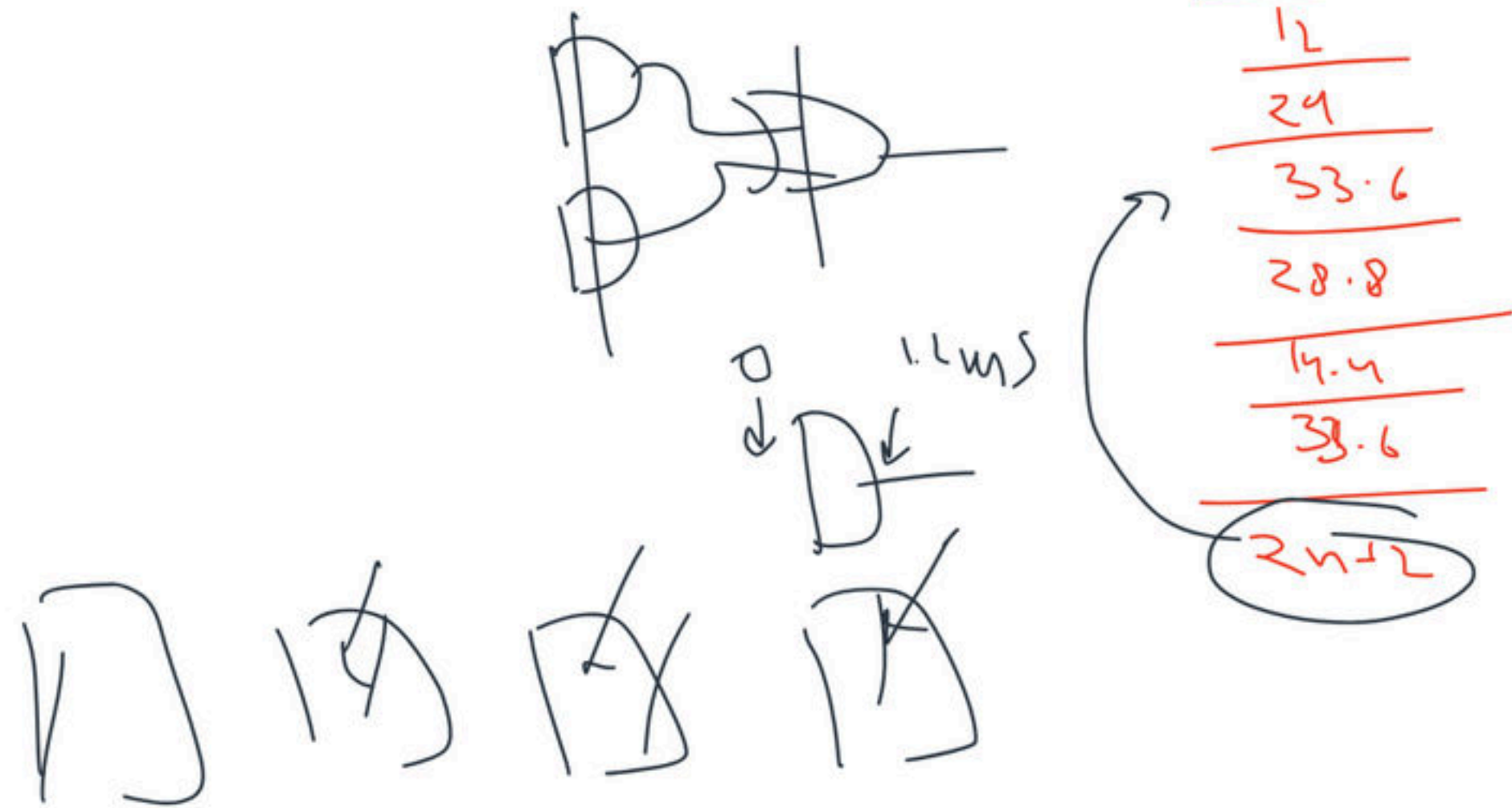
- There are some scope of improvement in adder like
 - Carry propagation delay
 - Solution is Look ahead carry generator
 - Can adder be modified to work as subtractor
 - Adder/subtractor or ripple adder

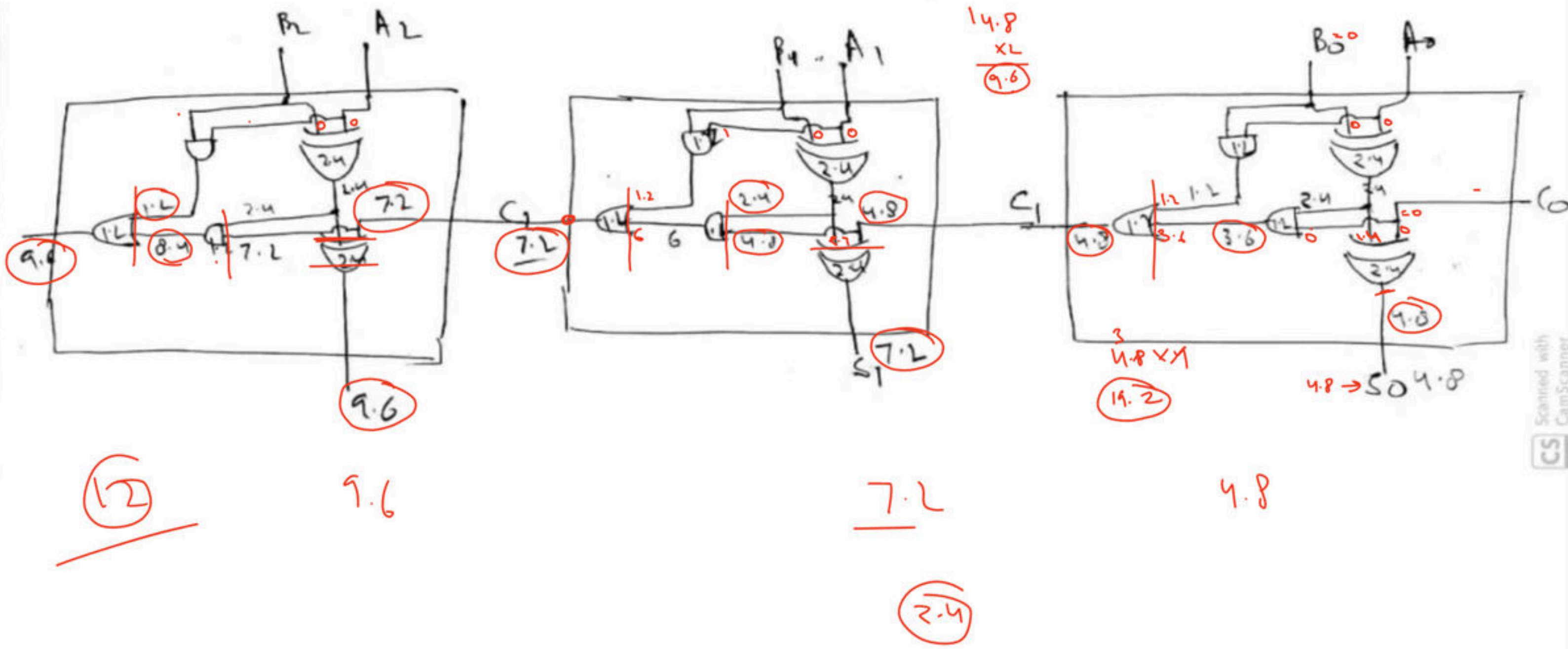


- The longest propagation delay time in an adder is the time it takes the carry to propagate through the full adders. For carry in a 4-bit adder we have 2 gate delays at each adder and for sum we have 1 gate delay.
- For an n -bit adder, there are $2n$ gate levels for the **carry to propagate from input to output**. And for Sum we have $2n-1$ Gate delays.



Q A half adder is implemented with XOR and AND gates. A full adder is implemented with two half adders and one OR gate. The propagation delay of an XOR gate is twice that of an AND/OR gate. The propagation delay of an AND/OR gate is 1.2 microseconds. A 4-bit ripple-carry binary adder is implemented by using full adders. The total propagation time of this 4-bit binary adder in microseconds is **(GATE-2015) (2 Marks)**

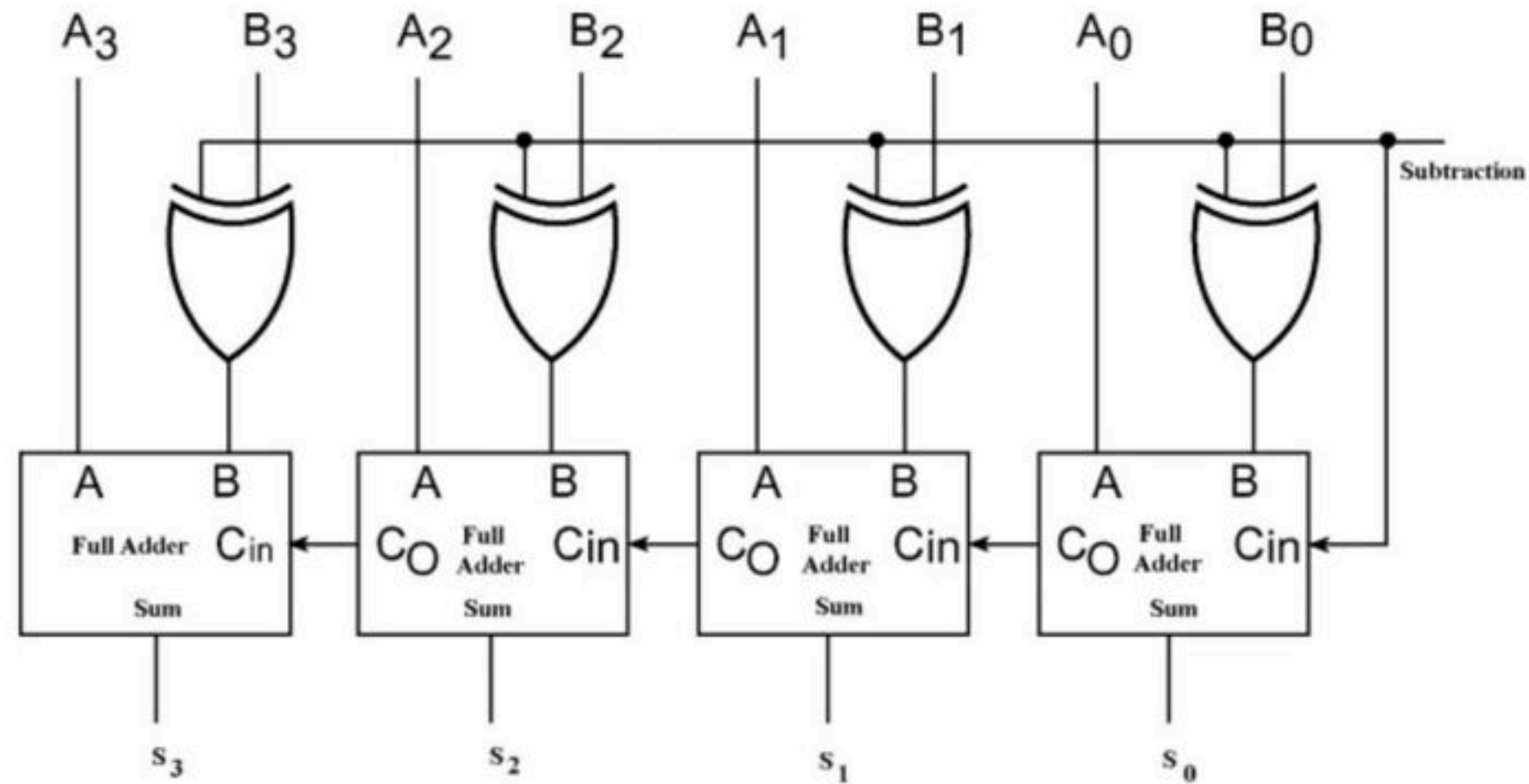




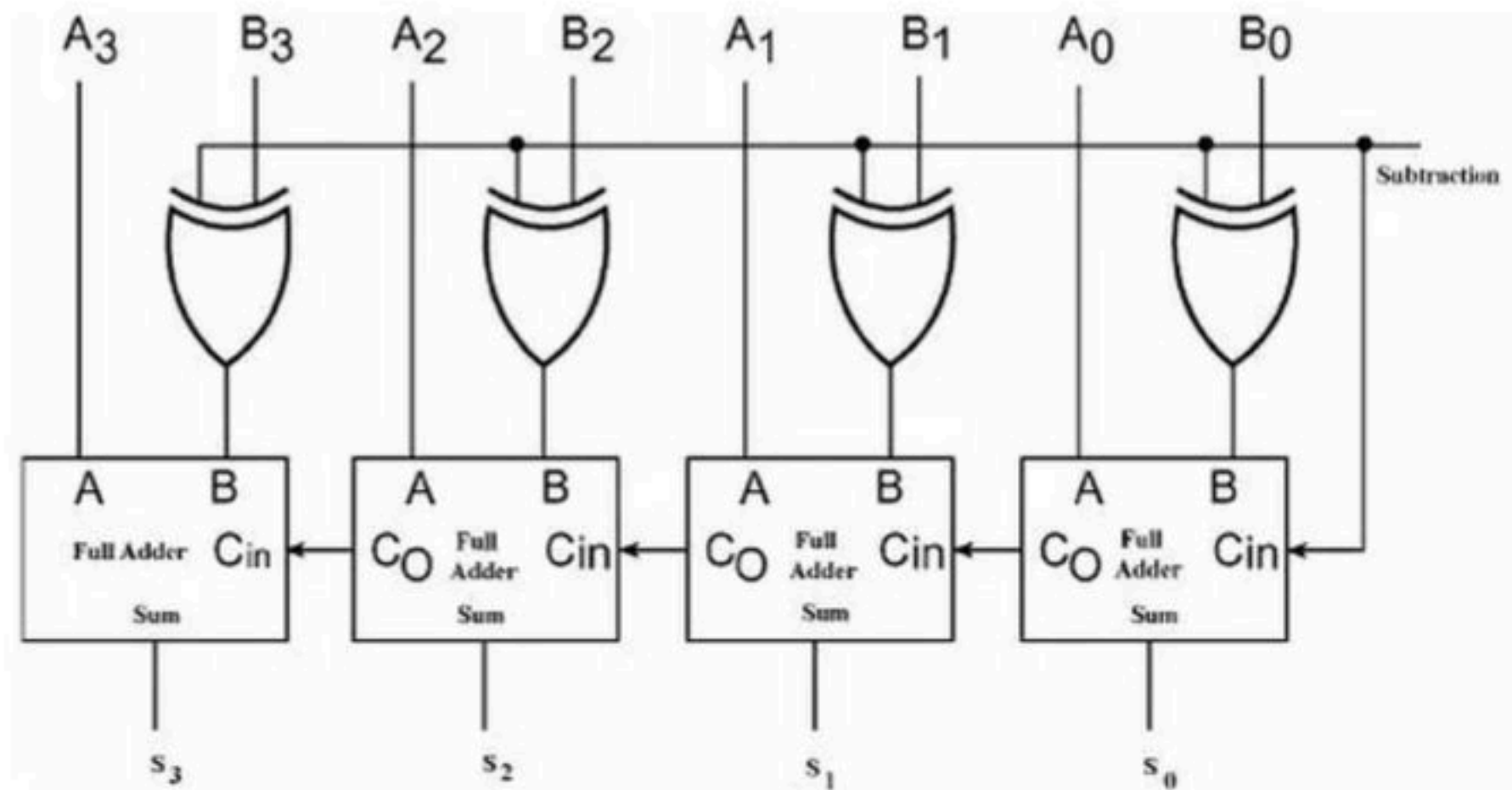
Four-bit ripple adder/subtractor

- The subtraction of unsigned binary numbers can be done most conveniently by means of complements.
- The subtraction $A - B$ can be done by taking the 2's complement of B and adding it to A .
- The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant pair of bits.
- The 1's complement can be implemented with inverters, and a 1 can be added to the sum through the input carry.

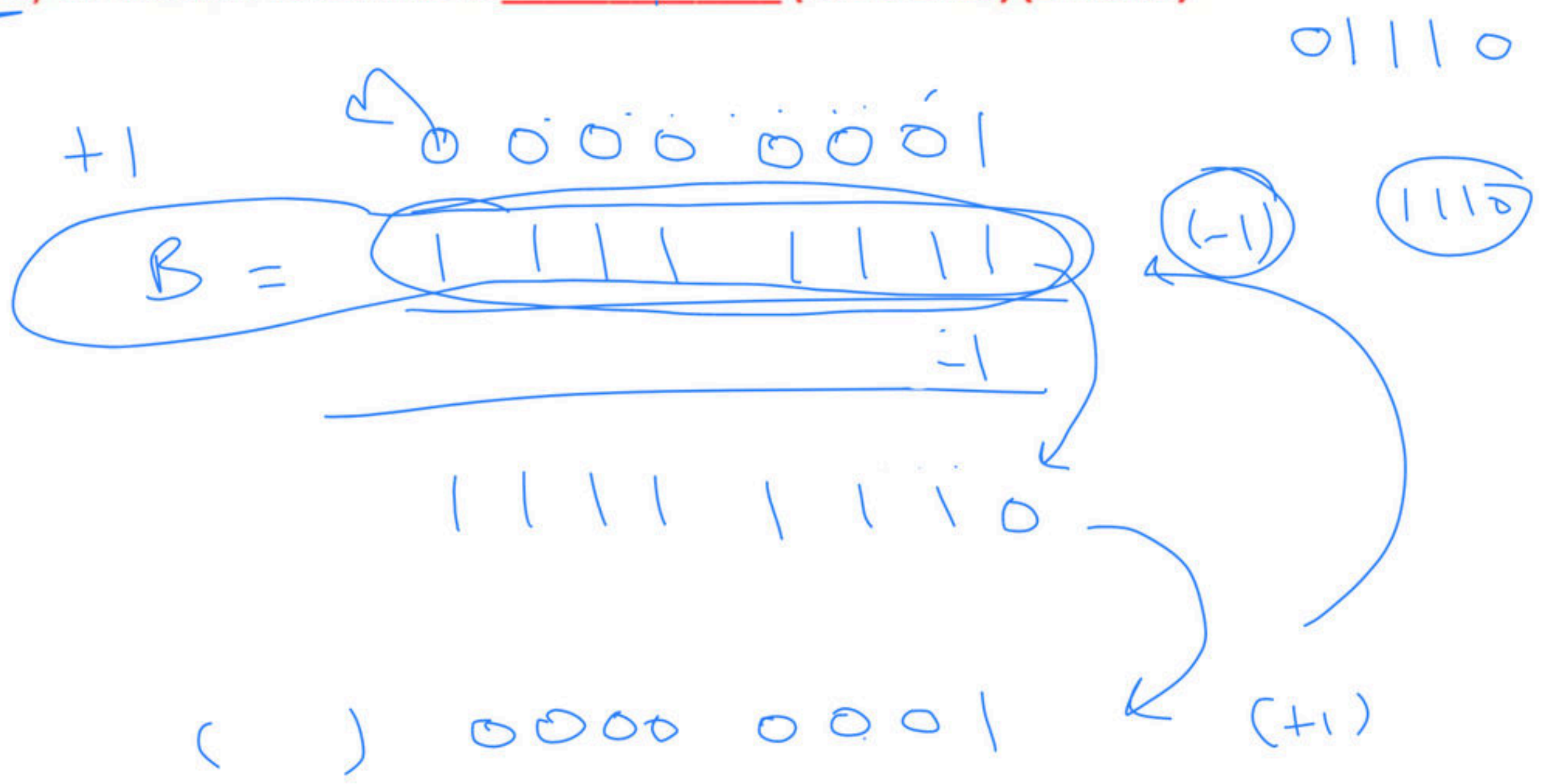
- The circuit for subtracting $A - B$ consists of an adder with inverters placed between each data input B and the corresponding input of the full adder.



- The mode input M controls the operation. When $M = 0$, the circuit is an adder, and when $M = 1$, the circuit becomes a subtractor.
- When $M = 0$, we have $B \oplus 0 = B$. The full adders receive the value of B, the input carry is 0, and the circuit performs A plus B.
- When $M = 1$, we have $B \oplus 1 = B'$ and $C_0 = 1$. The B inputs are all complemented and a 1 is added through the input carry. The circuit performs the operation A plus the 2's complement of B.



Q Consider an eight-bit ripple-carry adder for computing the sum of A and B, where A and B are integers represented in 2's complement form. If the decimal value of A is one, the decimal value of B that leads to the longest latency for the sum to stabilize is -1 (**GATE-2016**) (2 Marks)



+3

→ 0 0 1 1

-3

+3

0 0 1 1

1 1 0 0

+1

(-3) →

1 1 0 1

~~0~~ 1 0 1

(+5)

~~-2~~ 1 1 1 0

-1

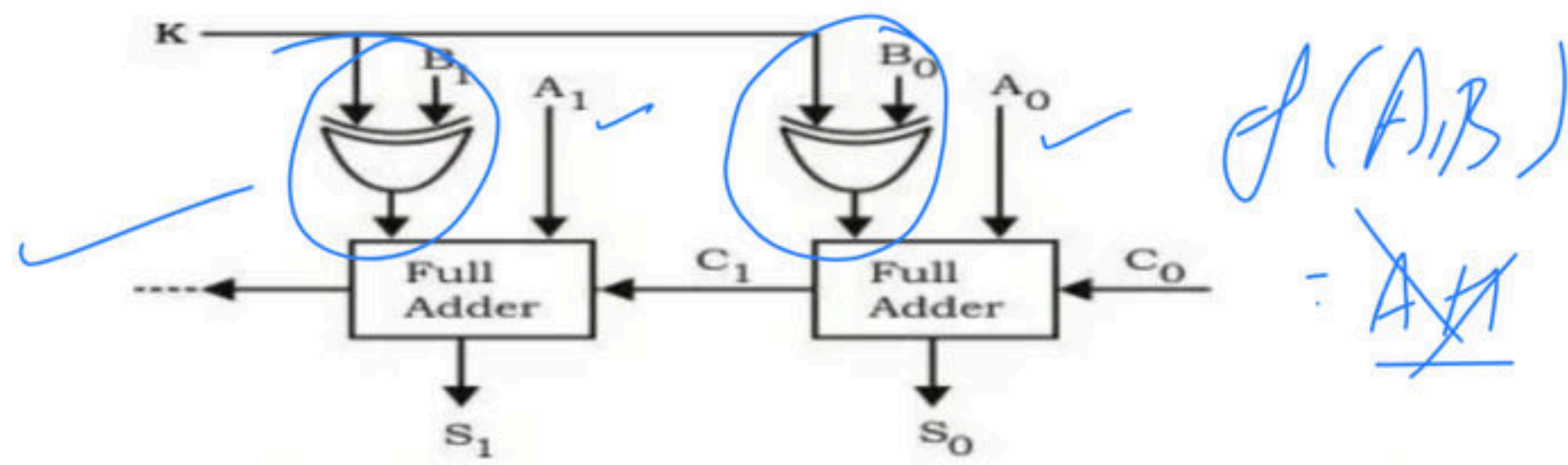
1 1 0 1

+2

0 0 1 0

Q Consider the ALU shown below.

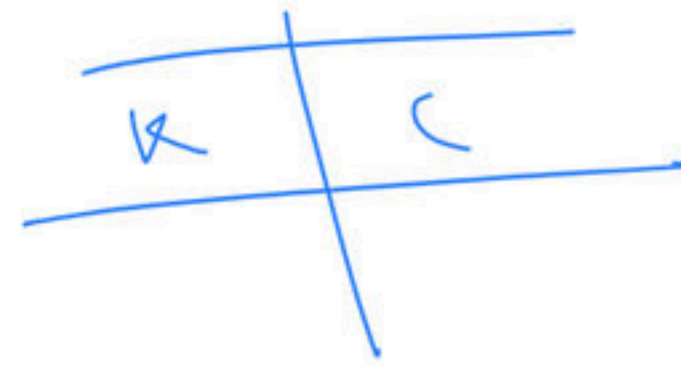
$K=0 \quad C_0=0$
 $K=1 \quad C_0=1$



If the operands are in 2's complement representation, which of the following operations can be performed by suitably setting the control lines K and C_0 only (+ and - denote addition and subtraction respectively)? **(GATE-2007)**

(2 Marks)

(A) $A + B$, and $A - B$, but not $A + 1$ ✓
(C) $A + B$, but not $A - B$, or $A + 1$ -5

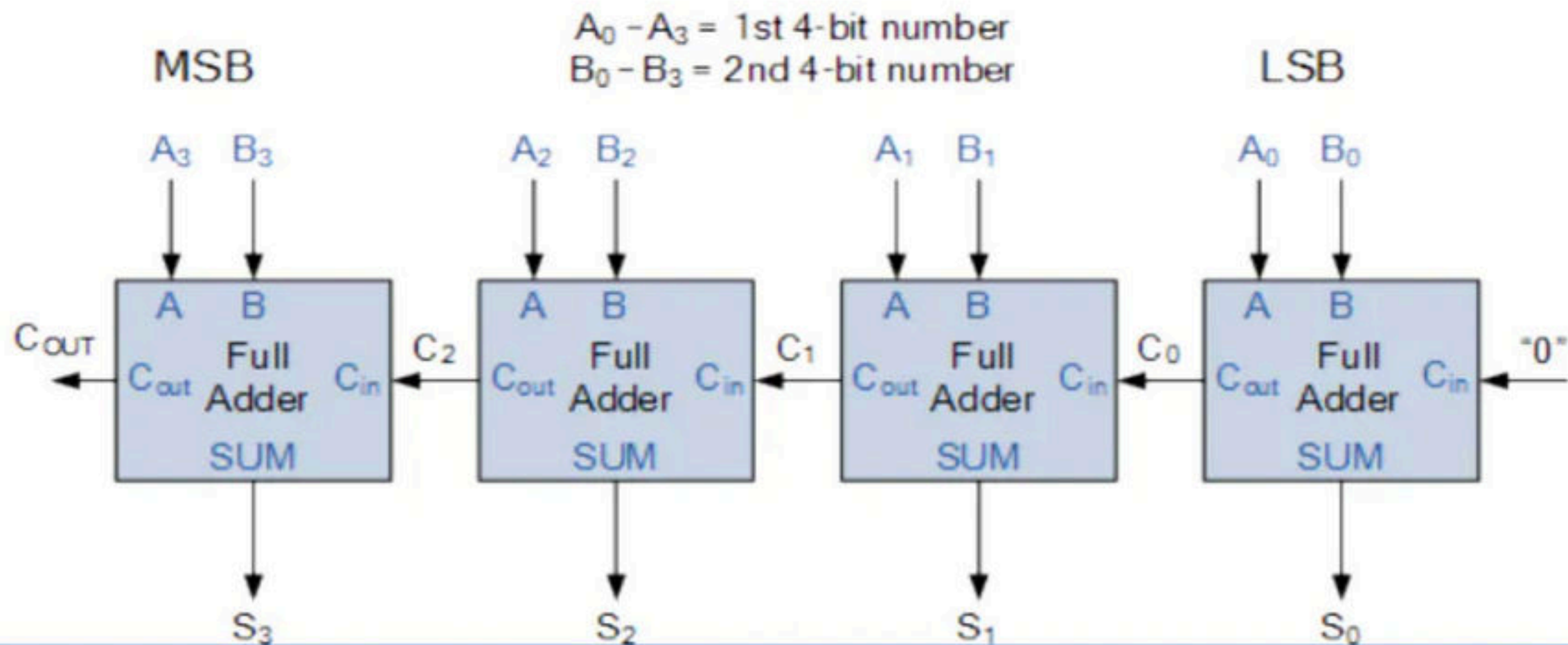


(B) $A + B$, and $A + 1$, but not $A - B$ -15
(D) $A + B$, and $A - B$, and $A + 1$ -12

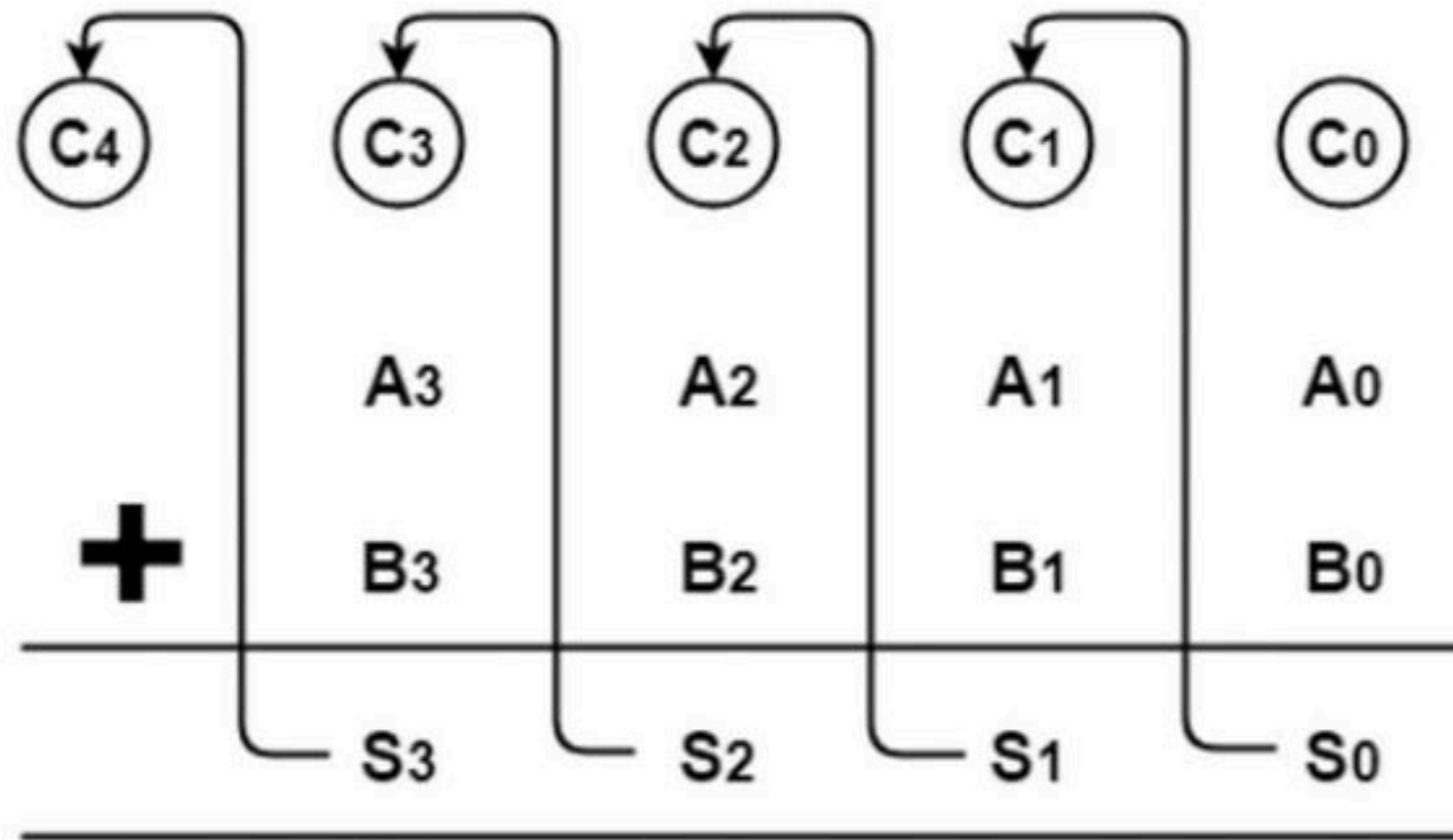
$A + B$

Look ahead carry adder

- In parallel adder all bits of augend and addend are available for computation initially, but sum and carry outputs of any stage cannot be produced until the input carry occurs. This leads to delay in the addition process known as carry propagation delay.
- In any combinational circuit, the signal must propagate through the gates before the correct output sum is available in the output terminals. **The total propagation time is equal to the propagation delay of a typical gate * times the number of gate levels in the circuit.**



- The carry propagation time is an important attribute of the adder because it limits the speed with which two numbers are added. The solution to delay is to increase the complexity of the equipment in such a way that the carry delay time is reduced.
- To solve this problem most widely used technique employs the principle of 'look ahead carry'. This method utilizes logic gates to look at the lower order bits of the augend and addend to see if a higher order carry is to be generated. It uses two functions carry generate G_i and carry propagate P_i



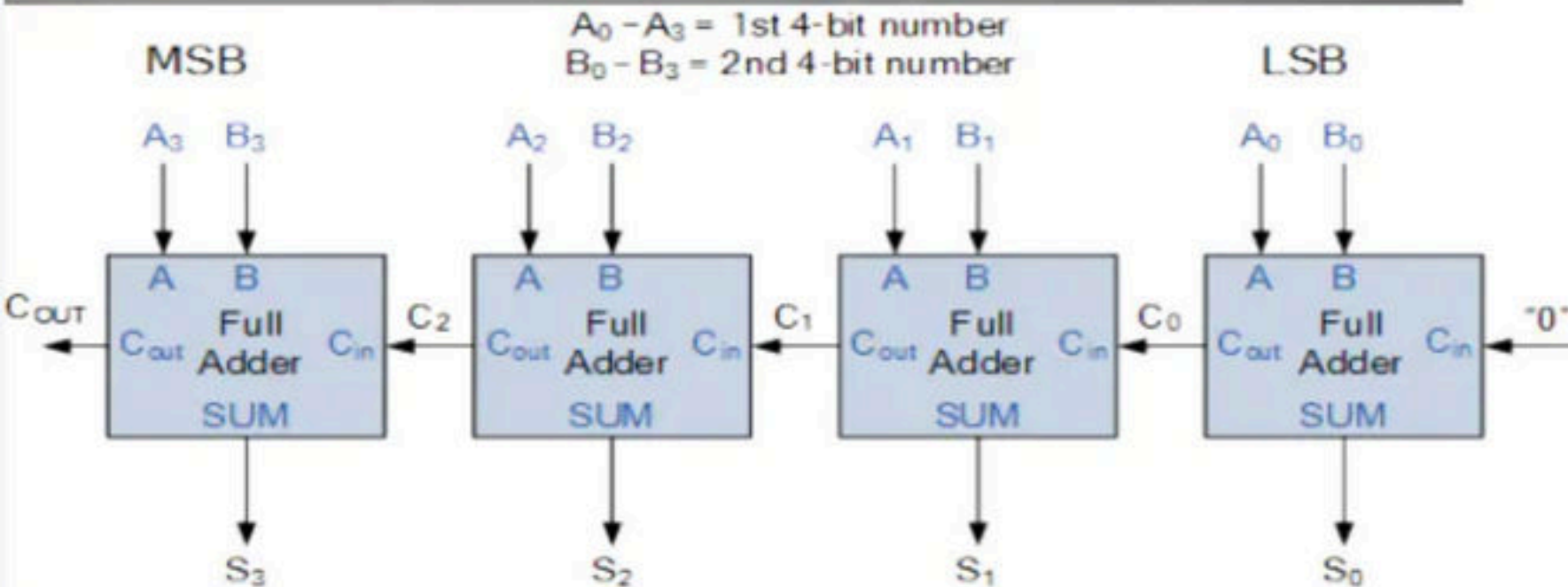
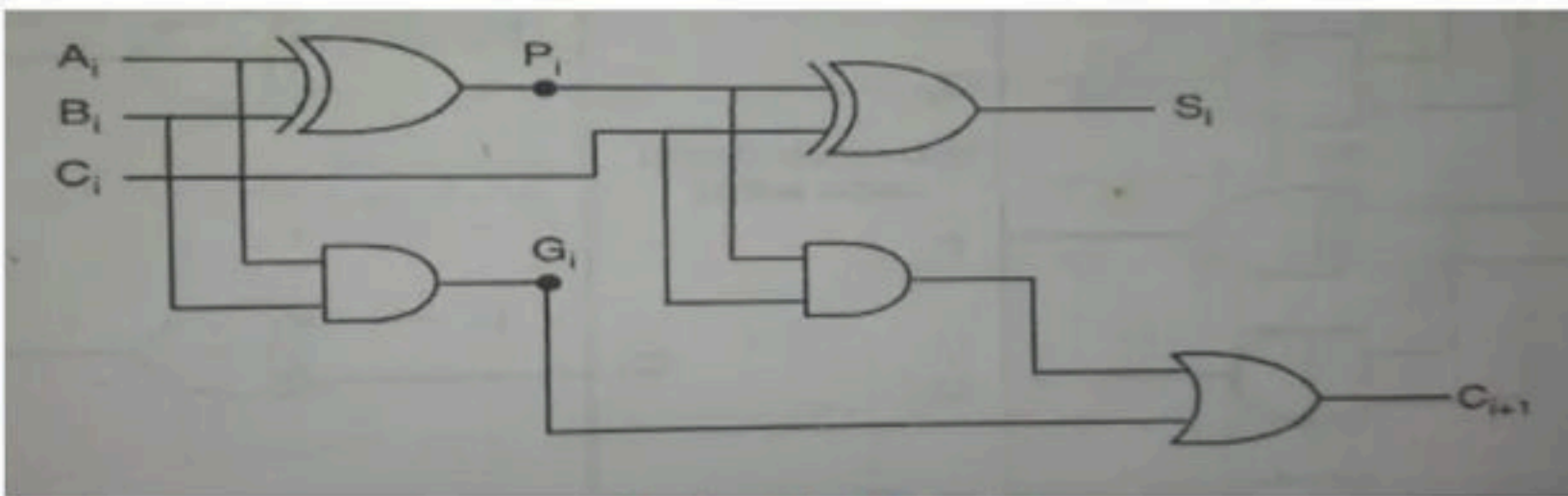
$$\begin{matrix} A(A_3 & A_2 & A_1 & A_0) \\ B(B_3 & B_2 & B_1 & B_0) \end{matrix}$$

$$P_i = A_i \oplus B_i$$

$$G_i = A_i \cdot B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$



- G_i is called a **carry generate**, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i .
- P_i is called a **carry propagate**, because it determines whether a carry into stage i will propagate into stage $i + 1$.
- We now write the Boolean functions for the carry outputs of each stage and substitute the value of each C_i from the previous equations:

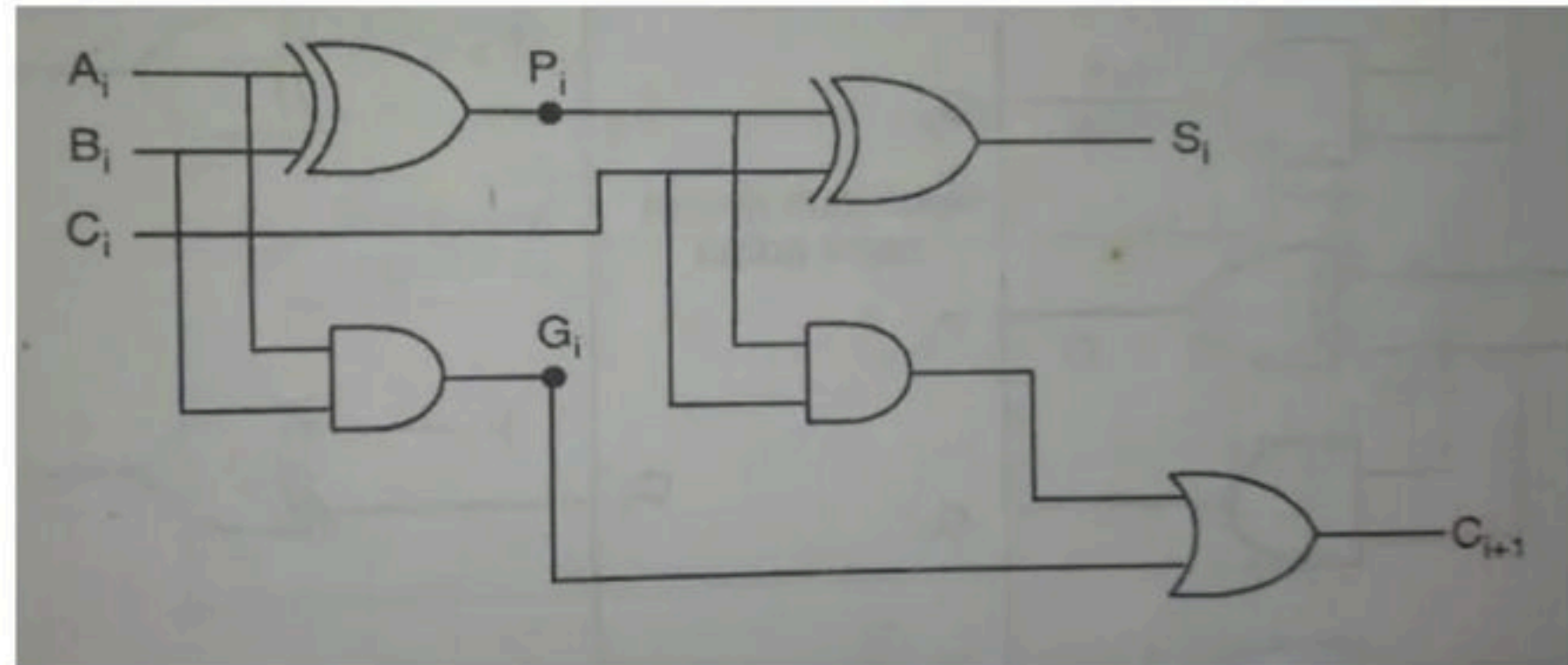
- $$P_i = A_i \oplus B_i$$

$$G_i = A_i \cdot B_i$$

$$S_i = P_i \oplus C_i$$

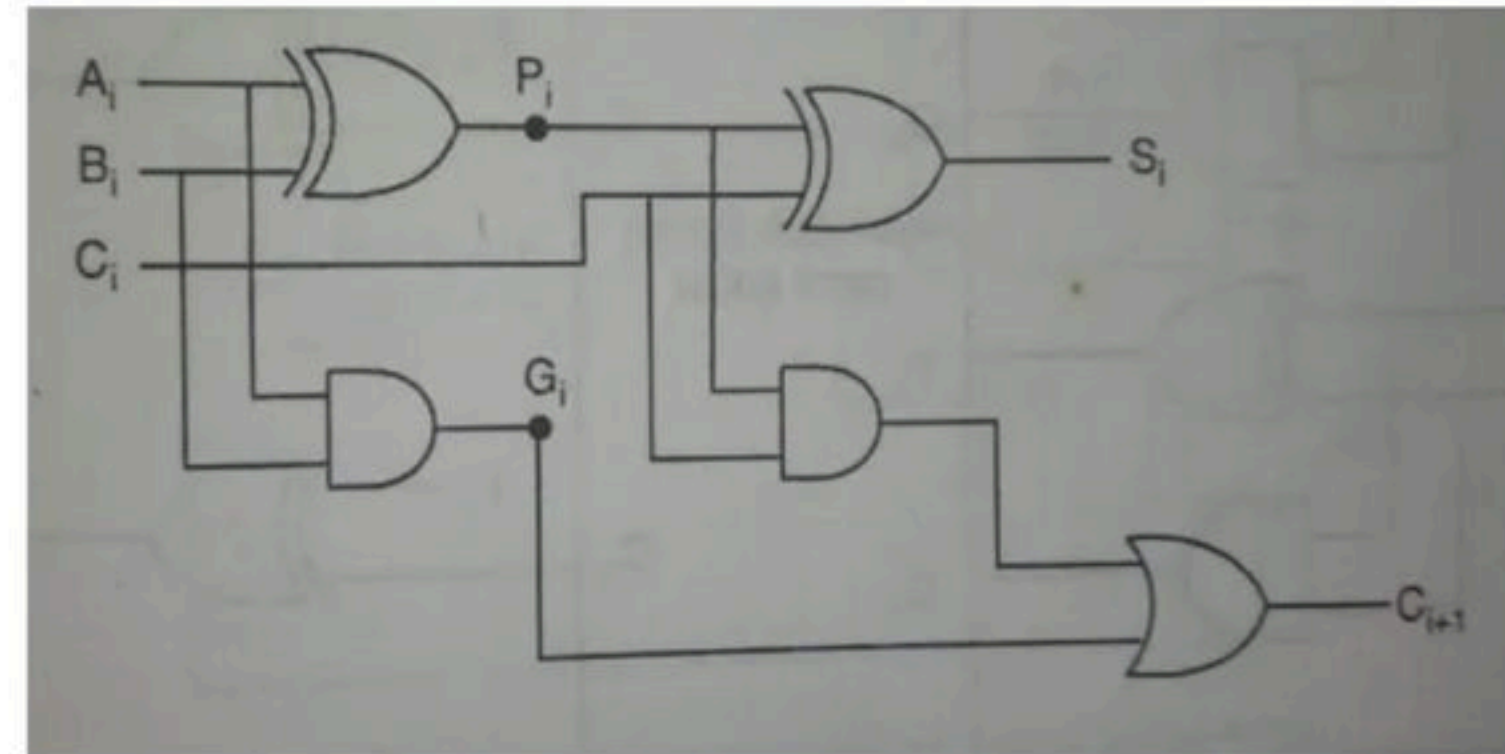
$$C_{i+1} = G_i + P_i C_i$$

- $C_0 =$
 $C_1 =$
 $C_2 =$
 $C_3 =$

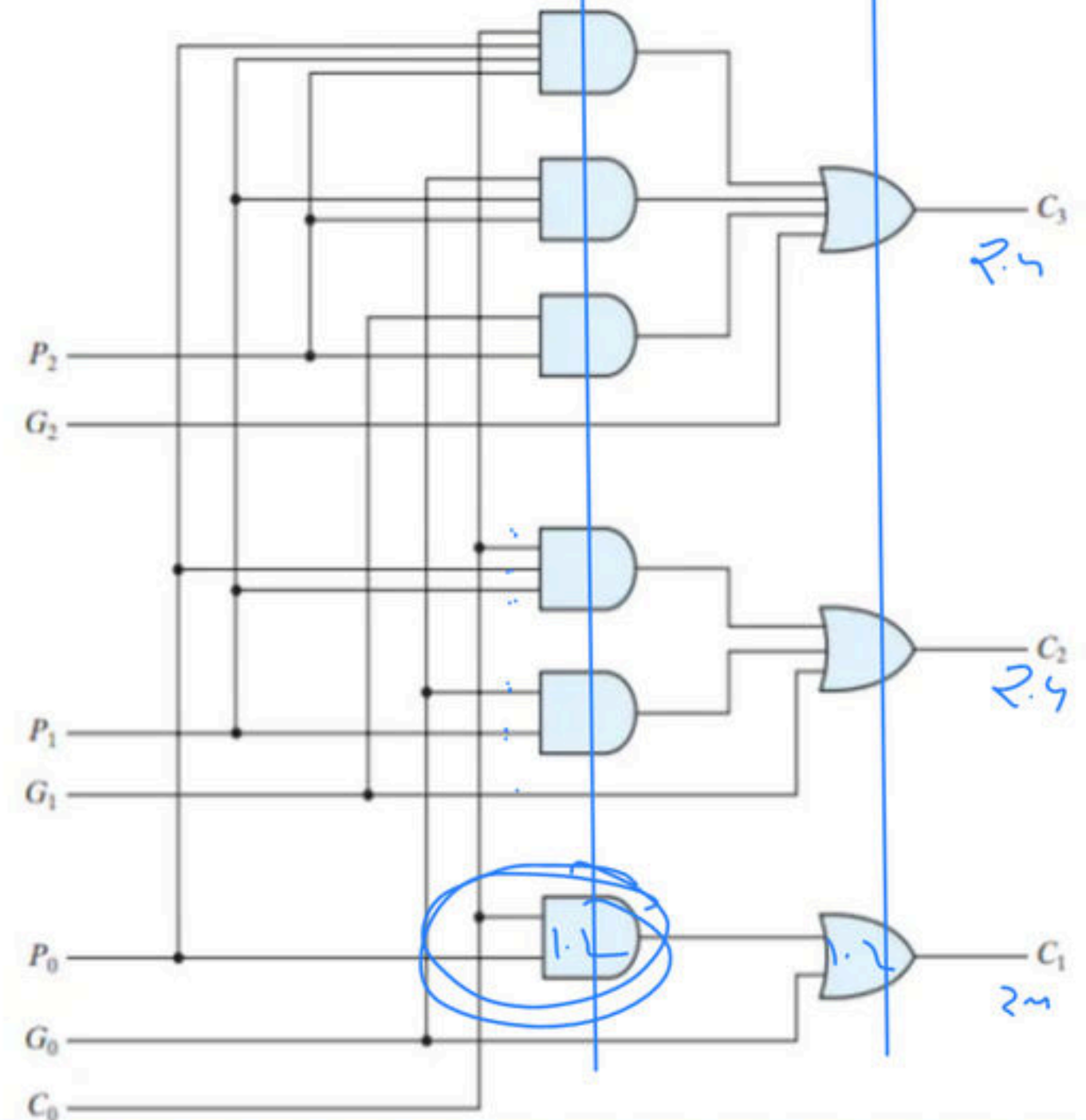
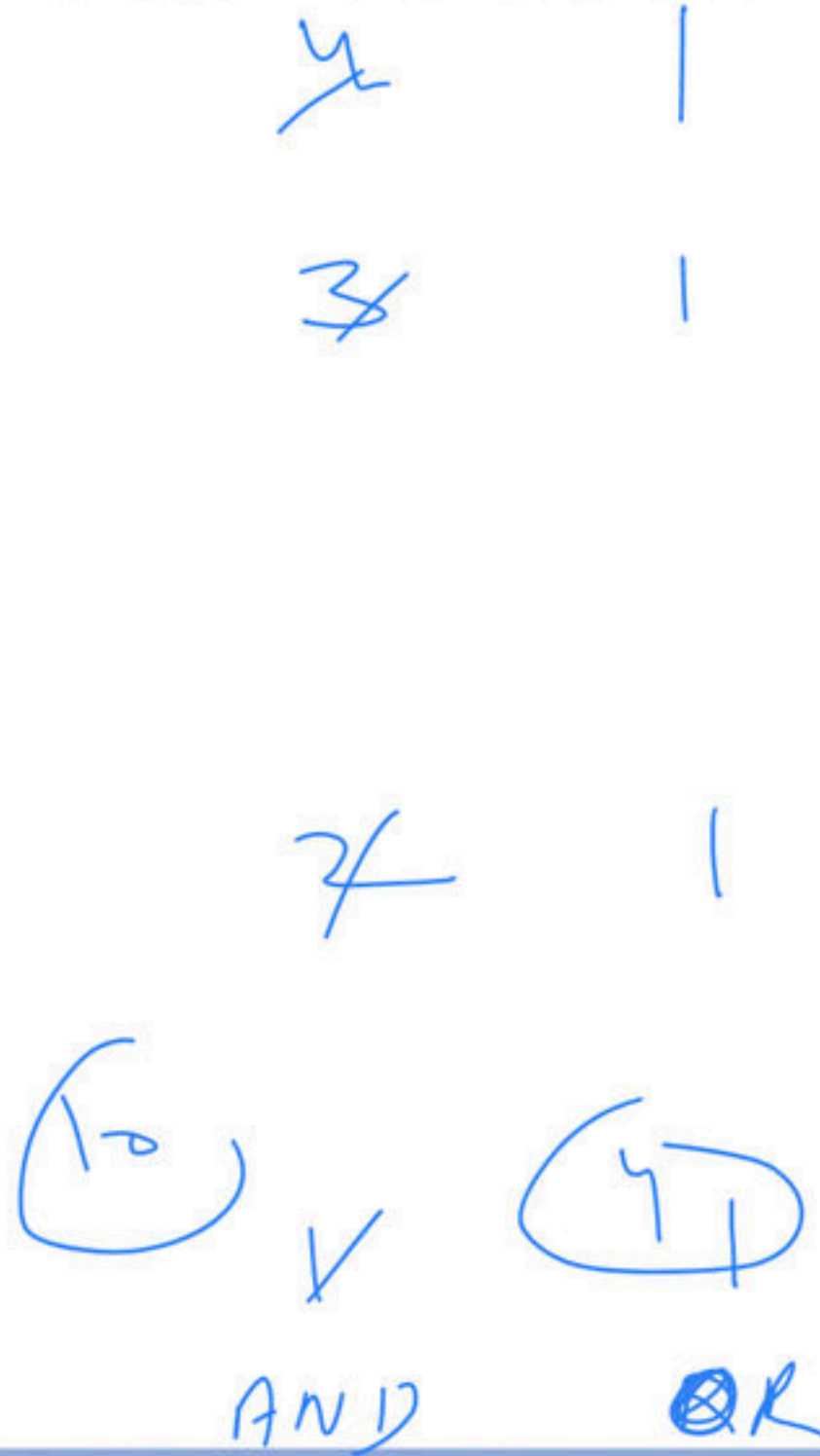


- G_i is called a **carry generate**, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i .
- P_i is called a **carry propagate**, because it determines whether a carry into stage i will propagate into stage $i + 1$.
- We now write the Boolean functions for the carry outputs of each stage and substitute the value of each C_i from the previous equations:

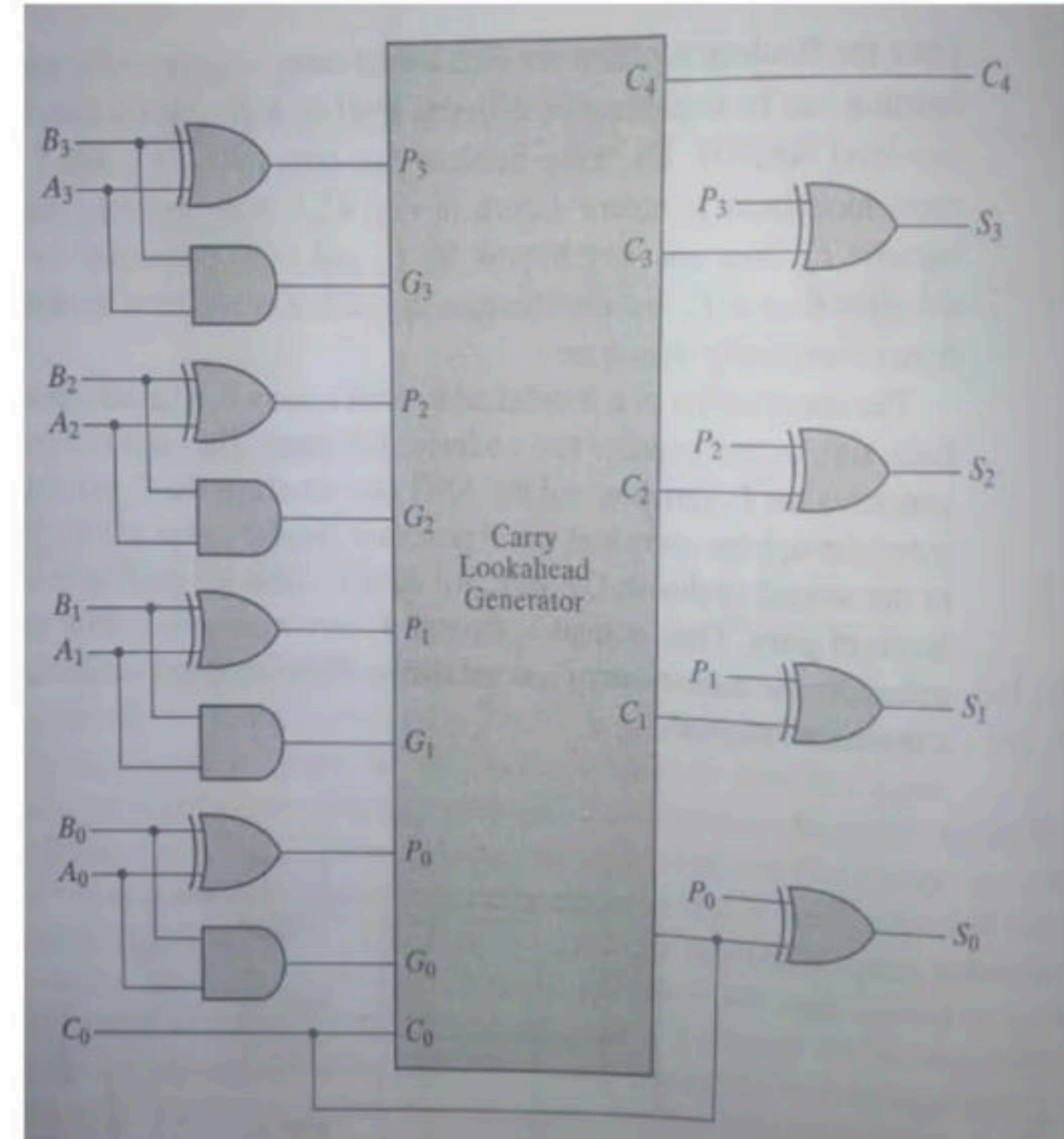
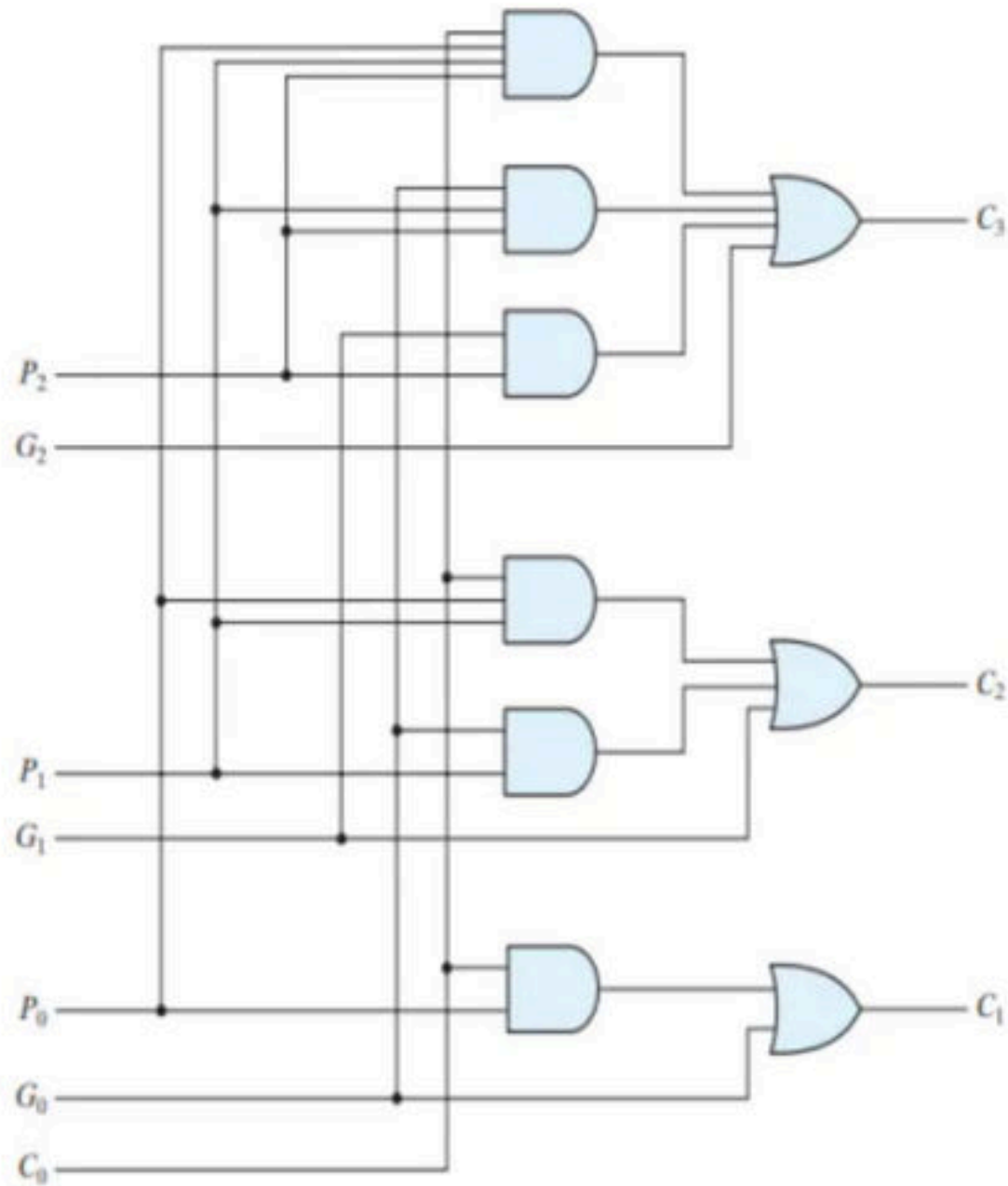
- $P_i = A_i \oplus B_i$
 $G_i = A_i \cdot B_i$
 $S_i = P_i \oplus C_i$
 $C_{i+1} = G_i + P_i C_i$
- $C_0 = 0$
 $C_1 = G_0 + P_0 C_0$
 $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$
 $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
 $C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$



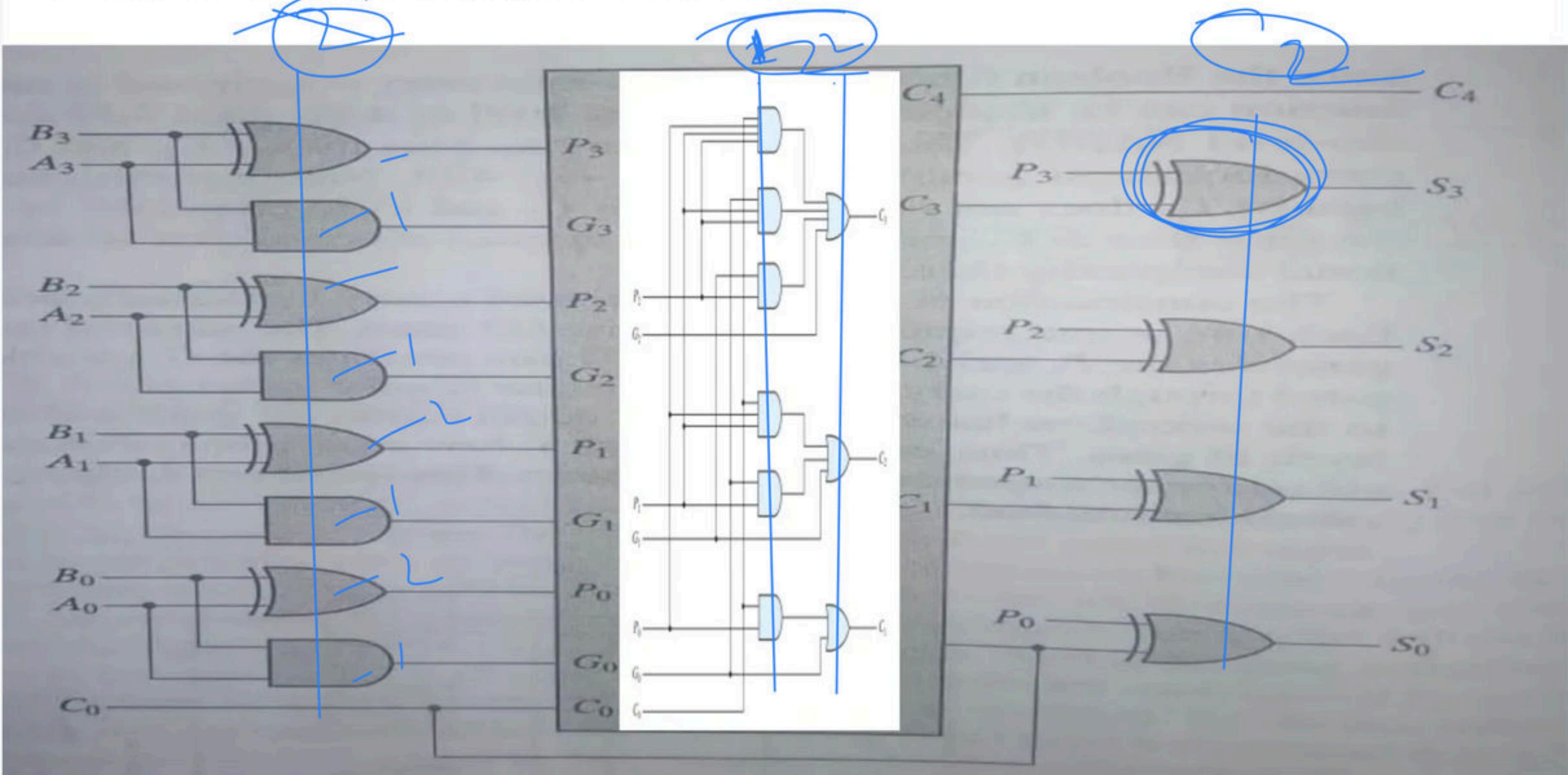
- Since the Boolean function for each output carry is expressed in sum-of-products form. Each function can be implemented with one level of AND gates followed by an OR gate (or by a two-level NAND).



- All output carries are generated after a delay through two levels of gates. Thus, outputs S_1 through S_3 have equal propagation delay times.



- All output carries are generated after a delay through two levels of gates. Thus, outputs S_1 through S_3 have equal propagation delay times.



Q Consider a carry lookahead adder for adding two n -bit integers, built using gates of fan-in at most two. The time to perform addition using this adder is **(GATE-2016) (2 Marks)**

(A) $\Theta(1)$

34

(B) $\Theta(\log(n))$

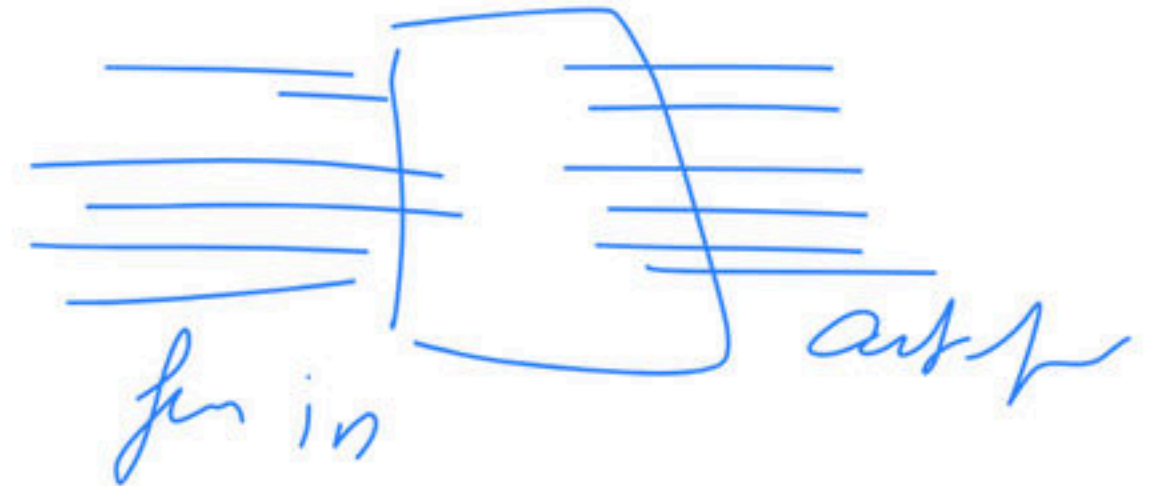
47

(C) $\Theta(\sqrt{n})$

2^n
3

(D) $\Theta(n)$

11



Q In a look-ahead carry generator, the carry generates function G_i and the carry propagate function P_i for inputs A_i and B_i are given by **(GATE-2007) (2 Marks)**

$P_i = A_i \oplus B_i$ and $G_i = A_i B_i$

The expressions for the sum bit S_i and the carry bit C_{i+1} of the look-ahead carry adder are given by:

$S_i = P_i \oplus C_i$ and $C_{i+1} = G_i + P_i C_i$, where C_0 is the input carry.

Consider a two-level logic implementation of the look-ahead carry generator. Assume that all P_i and G_i are available for the carry generator circuit and that the AND and OR gates can have any number of inputs. The number of AND gates and OR gates needed to implement the look-ahead carry generator for a 4-bit adder with S_3, S_2, S_1, S_0 and C_4 as its outputs are respectively:

(A) 6, 3

10

(B) 10, 4

76

(C) 6, 4

12

(D) 10, 5

3

Q Given two three-bit numbers $a_2a_1a_0$ and $b_2b_1b_0$ and c , the carry in, the function that represents the carry generate function when these two numbers are added is **(GATE-2006) (2 Marks)**

(A) $a_2b_2 + a_2a_1b_1 + a_2a_1a_0b_0 + a_2a_0b_1b_0 + a_1b_2b_1 + a_1a_0b_2b_0 + a_0b_2b_1b_0$ ← 12

(B) $a_2b_2 + a_2b_1b_0 + a_2a_1b_1b_0 + a_2a_0b_2b_1 + a_1a_0b_2 + a_1a_0b_2b_0 + a_2a_0b_1b_0$

(C) $a_2 + b_2 + (a_2 \oplus b_2)(a_1 + b_1 + (a_1 \oplus b_1)(a_0 + b_0))$

(D) $a_2b_2 + \overline{a_2}a_1b_1 + \overline{a_2}\overline{a_1}a_0b_0 + \overline{a_2}a_0\overline{b_1}b_0 + a_1\overline{b_2}b_1 + \overline{a_1}a_0\overline{b_2}b_0 + a_0\overline{b_2}\overline{b_1}b_0$



Q A 4-bit carry lookahead adder, which adds two 4-bit numbers, is designed using AND, OR, NOT, NAND, NOR gates only. Assuming that all the inputs are available in both complemented and uncomplemented forms and the delay of each gate is one-time unit, what is the overall propagation delay of the adder? Assume that the carry network has been implemented using two-level AND-OR logic. **(GATE-2004) (2 Marks)**

(A) 4 time units

46

(B) 6 time units

✓ 23

(C) 10 time units

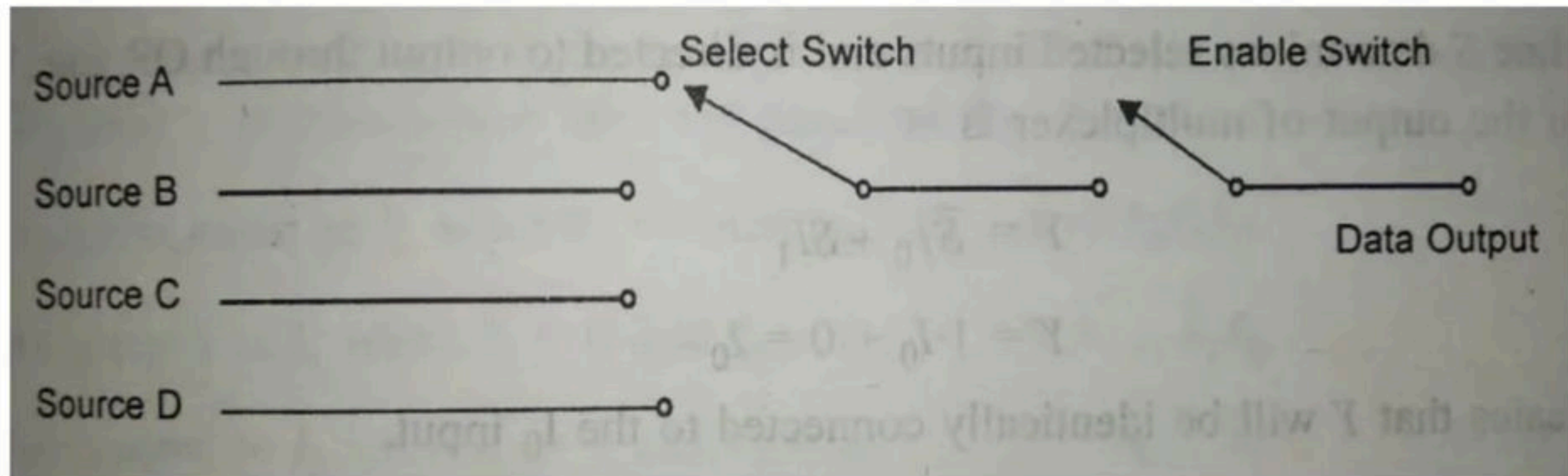
25

(D) 12 time units

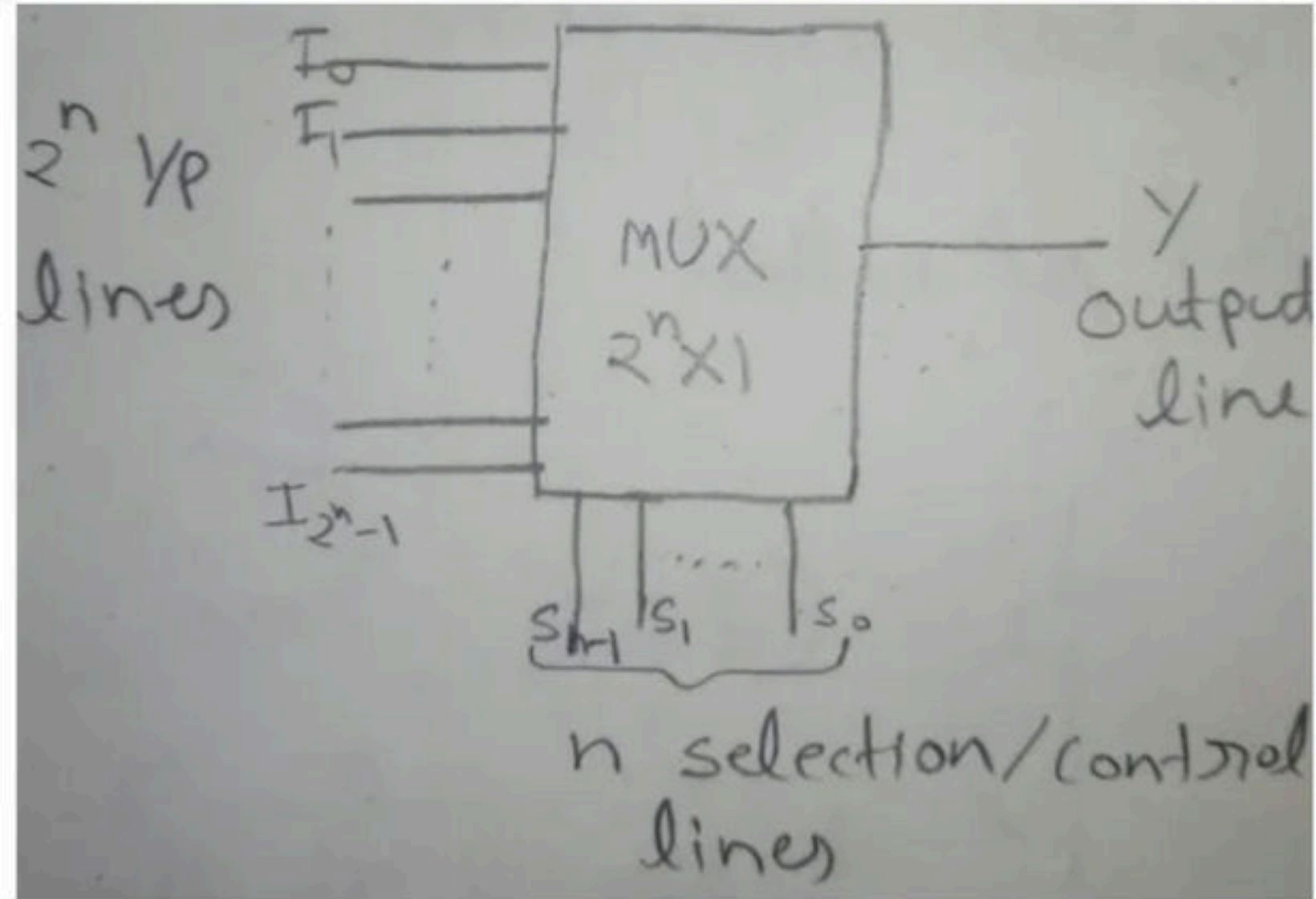
✓

Multiplexer (Selector)

- Multiplexer are special and one of the most widely used combinational circuits.
- Main requirement is out of many inputs we have to select one for e.g. telephone or train leaving the station. So multiplexer do not perform any logical operation or comparison, it just acts as a switch or relay.



1. A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.
2. A multiplexer is also called a data selector, since it selects one of many inputs and steers the binary information to the output line.
3. The selection of a particular input line is controlled by a set of selection lines.
4. There are 2^n input lines and n selection lines whose bit combinations determine which input is to be selected.
5. No of Input line $= 2^n$



Note:

- Can never have two i/p connected to out at any time

Application

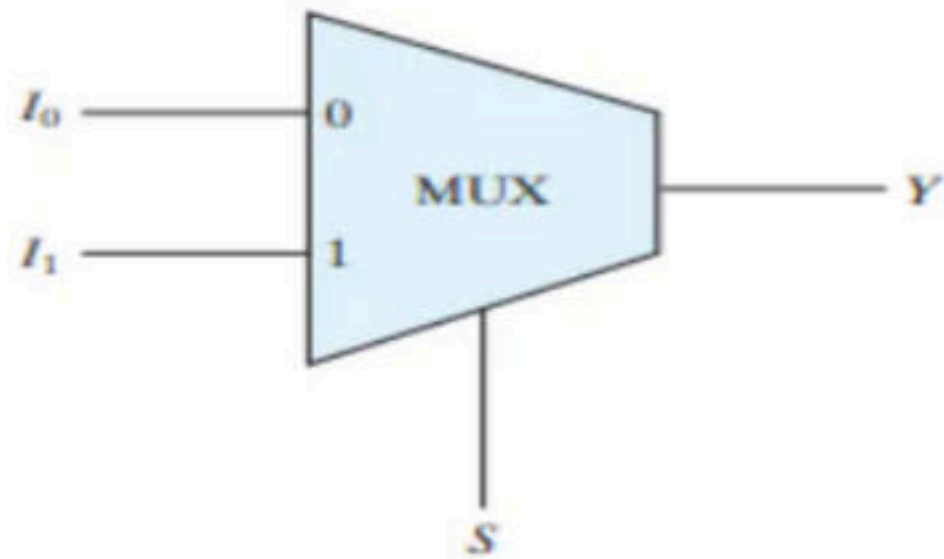
- Parallel data to serial data conversion
- Used as data selector, as the output of a multiplexer is directed from one of various inputs
- Used in implementation of Boolean functions
- Used in communication systems, **Computer Memory, Telephone Network, Transmission from the Computer System of a Satellite**

A two-to-one-line multiplexer

- This multiplexer acts like an electronic switch that selects one of two sources.
- The circuit has two data input lines, one output line, and one selection line S.

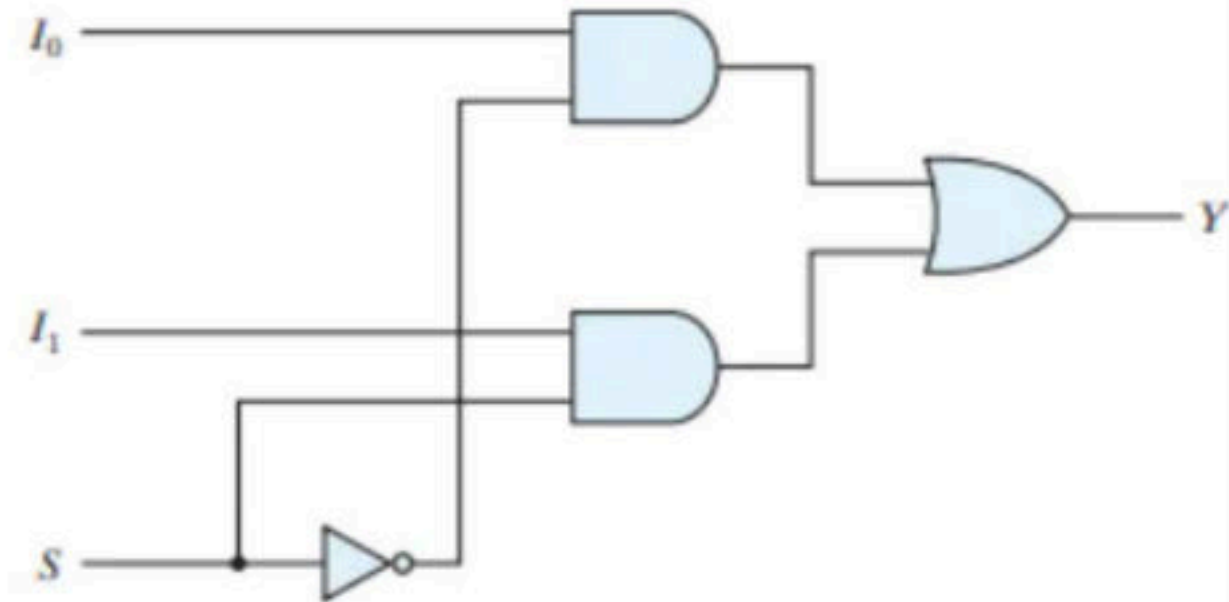
A two-to-one-line multiplexer

- When $S = 0$, the upper AND gate is enabled and I_0 has a path to the output.
- When $S = 1$, the lower AND gate is enabled and I_1 has a path to the output.



(b) Block diagram

Input	Output
S	Y
0	I_0
1	I_1



(a) Logic diagram

Characteristic equation: $Y = S_0' I_0 + S_0 I_1$

Q A multiplexer is a logic circuit that **(NET-JUNE-2011)**

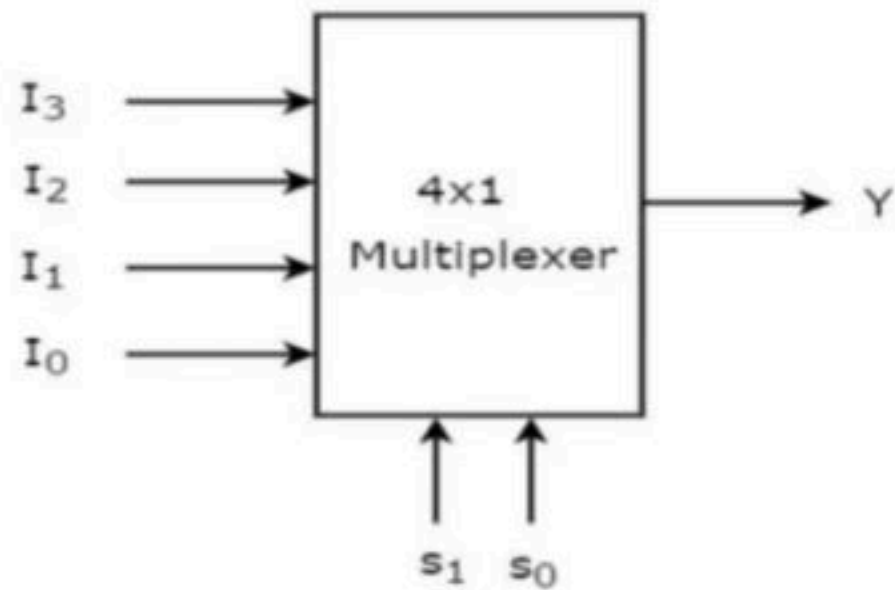
- (A)** accepts one input and gives several outputs
- (B)** accepts many inputs and gives many output
- (C)** accepts many inputs and gives one output
- (D)** accepts one input and gives one output

Case study of 4 to 1

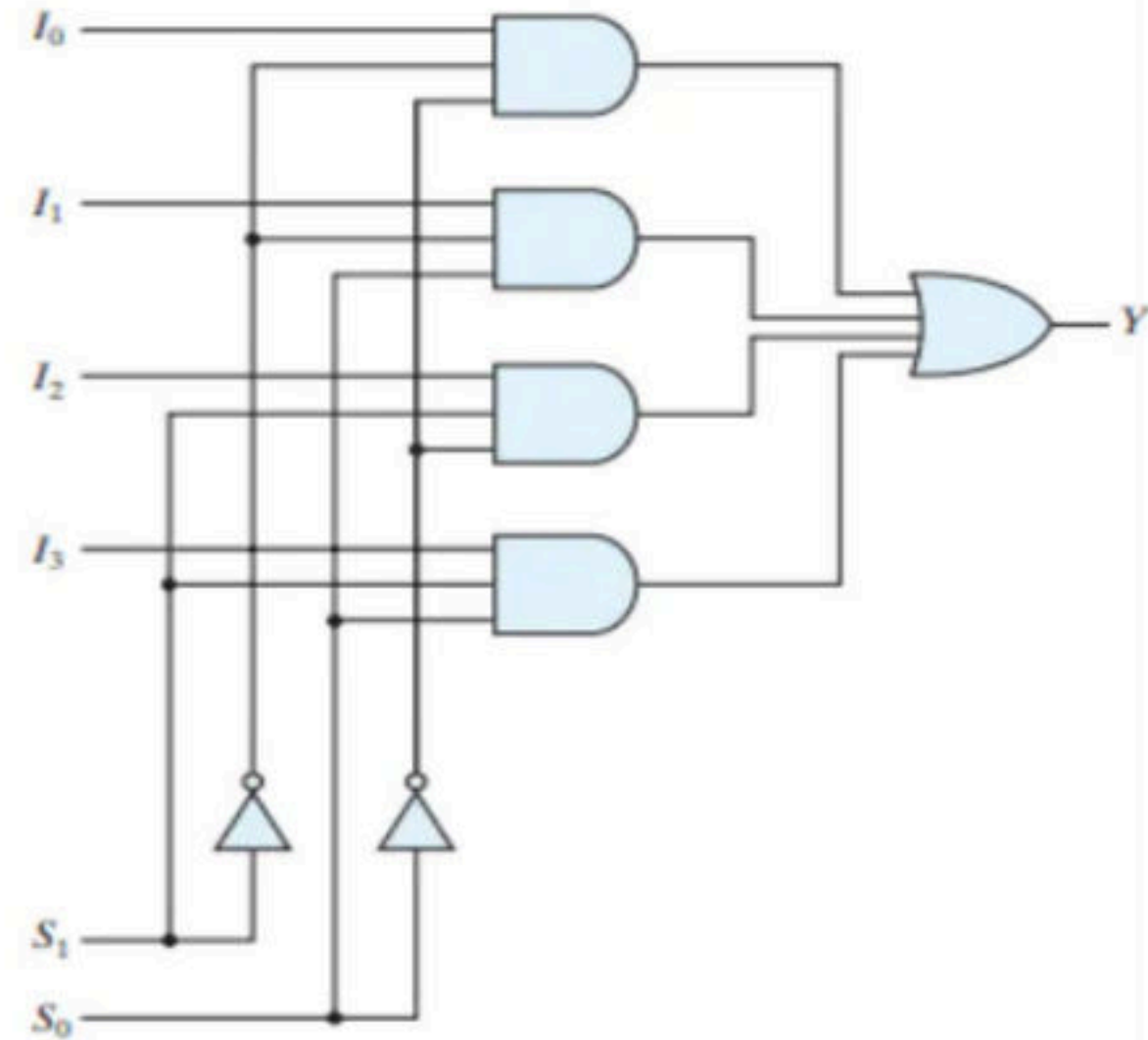
- The circuit has four data input lines I_0, I_1, I_2, I_3 one output line Y , and two selection line S_0 and S_1 .

Case study of 4 to 1

- Each of the four inputs, I_0 through I_3 , is applied to one input of an AND gate.
- Selection lines S_1 and S_0 are decoded to select a particular AND gate.
- The outputs of the AND gates are applied to a single OR gate that provides the one-line output.



Input		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



Characteristic equation

$$Y = E [(S'_1 S'_0 I_0) + (S'_1 S_0 I_1) + (S_1 S'_0 I_2) + (S_1 S_0 I_3)]$$

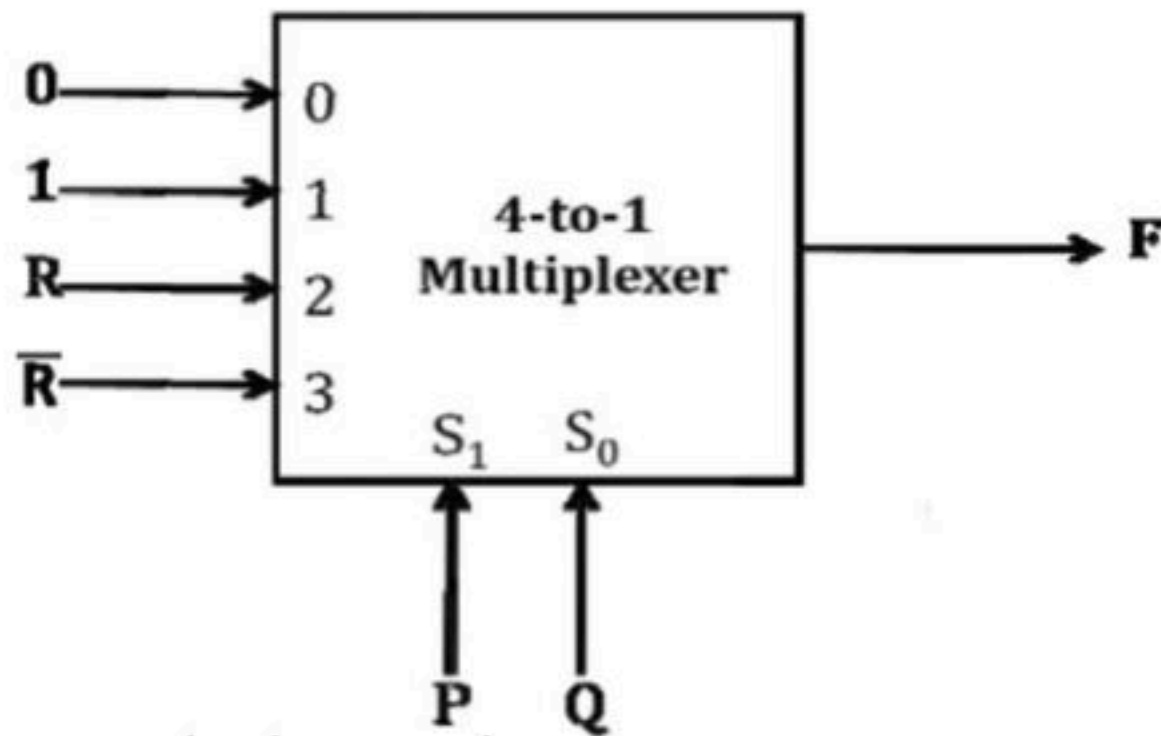
Q Consider a 4-to-1 multiplexer with two select lines S_1 and S_0 , given below
The minimal sum-of-products form of the Boolean expression for the output F of the multiplexer is **(Gate-CS-2014-Set-1) (2 Marks)**

(A) $P'Q + QR' + PQ'R$

(C) $P'QR + P'QR' + QR' + PQ'R$

(B) $P'Q + P'QR' + PQR' + PQ'R$

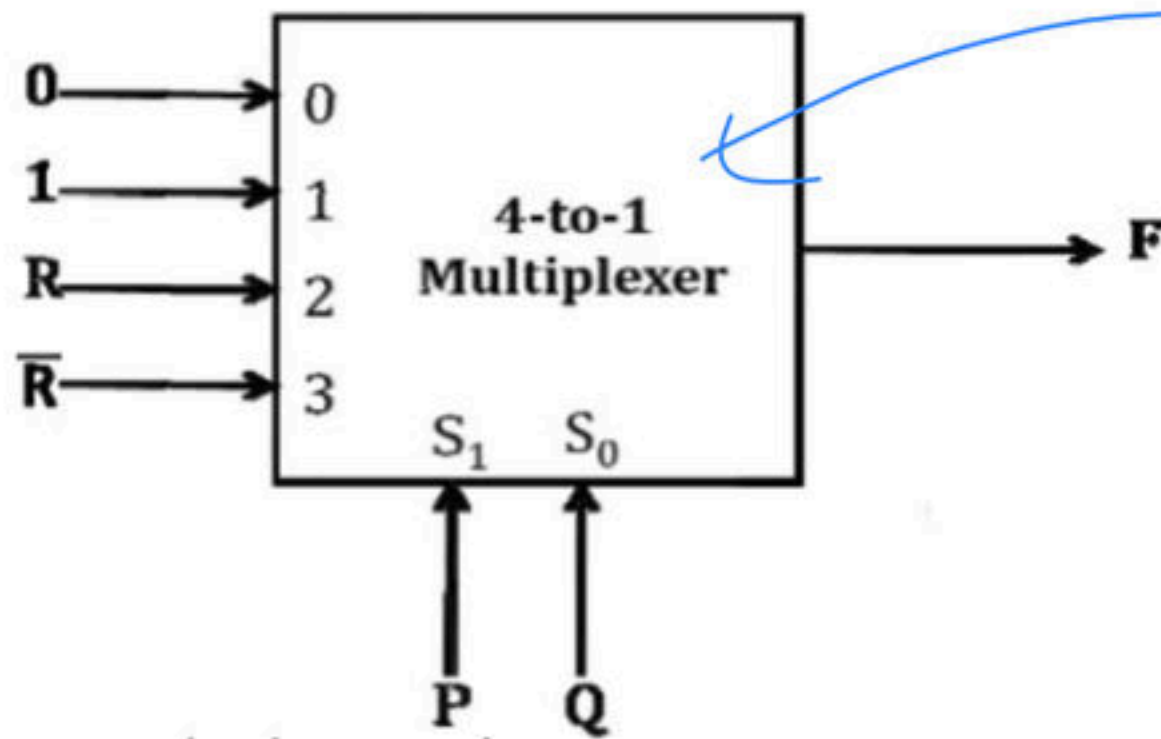
(D) PQR'



Q Consider a 4-to-1 multiplexer with two select lines S_1 and S_0 , given below
 The minimal sum-of-products form of the Boolean expression for the output F of the multiplexer is **(Gate-CS-2014-Set-1) (2 Marks)**

- (A) $P'Q + QR' + PQ'R$ ✓
 (C) $P'QR + P'QR' + QR' + PQ'R$

- (B) $P'Q + P'QR' + PQR' + PQ'R$
 (D) PQR'



$$Y = [(S'_1 S'_0 I_0) + (S'_1 S_0 I_1) + (S_1 S'_0 I_2) + (S_1 S_0 I_3)]$$

$$\cancel{\bar{P}\bar{Q} \cdot 0} + \bar{P}\bar{Q} \cdot 1 + P\bar{Q} \cdot R + P\bar{Q} \cdot \bar{R}$$

$$\cancel{\bar{P}\bar{Q}} + \cancel{P\bar{Q}}R + P\bar{Q}\bar{R}$$

	$\bar{P}$	$\bar{Q}$	$P\bar{Q}$	$P\bar{Q}$	$R\bar{Q}$
$\bar{R}$			1	1	
R			1		1

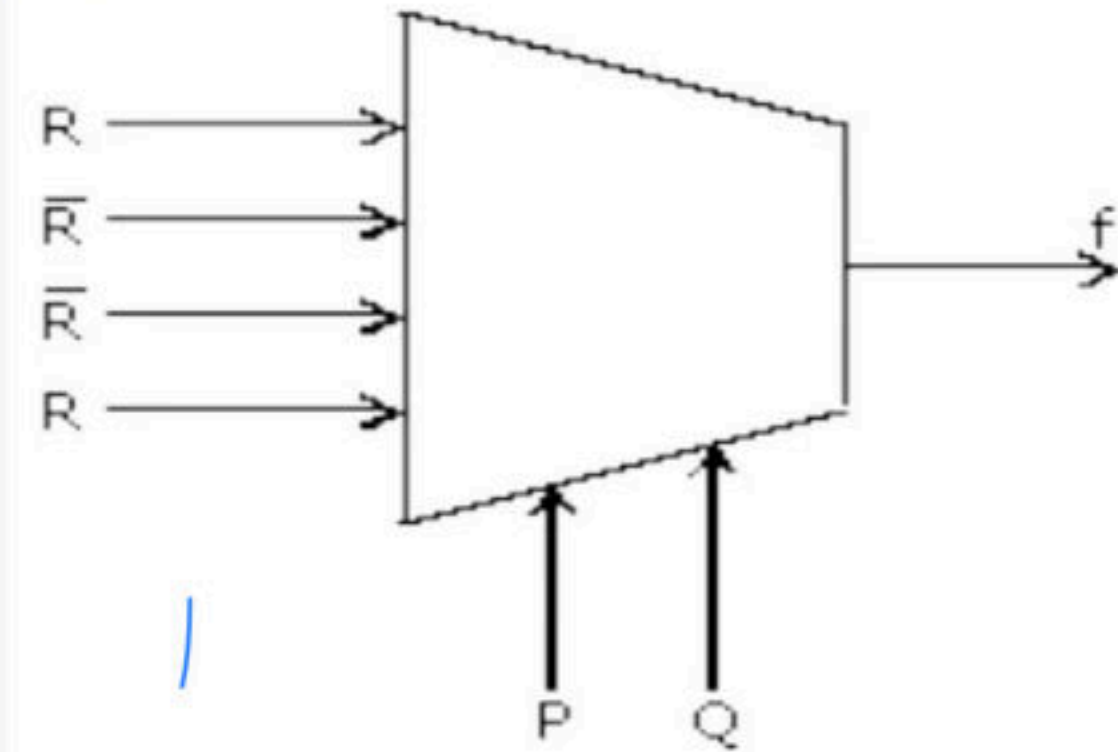
Q The Boolean expression for the output 'f' of the multiplexer shown below is **(GATE-2010) (2 Marks)**

(A) $(P \oplus Q \oplus R)'$

(B) $P \oplus Q \oplus R$

(C) $(P + Q + R)'$

(D) $P + Q + R$



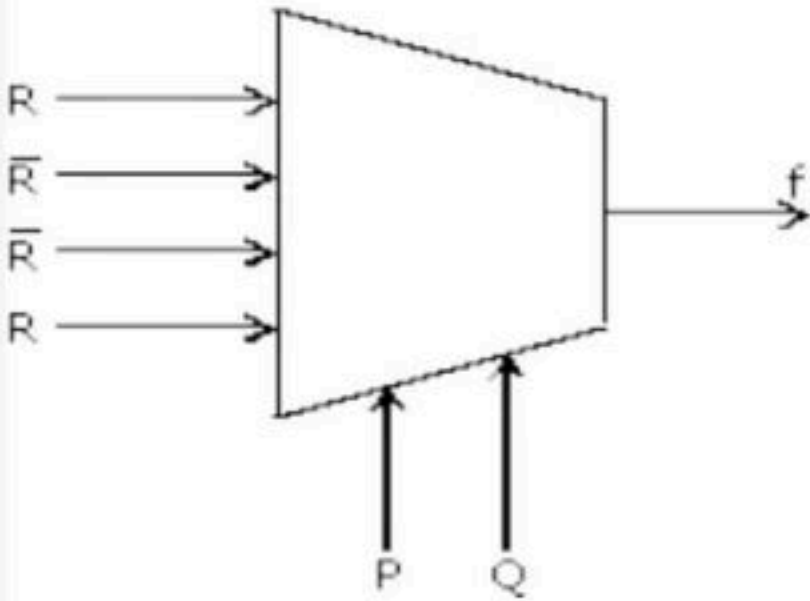
Q The Boolean expression for the output 'f' of the multiplexer shown below is **(GATE-2010) (2 Marks)**

(A) $(P \oplus Q \oplus R)'$

(B) $P \oplus Q \oplus R$

(C) $(P + Q + R)'$

(D) $P + Q + R$



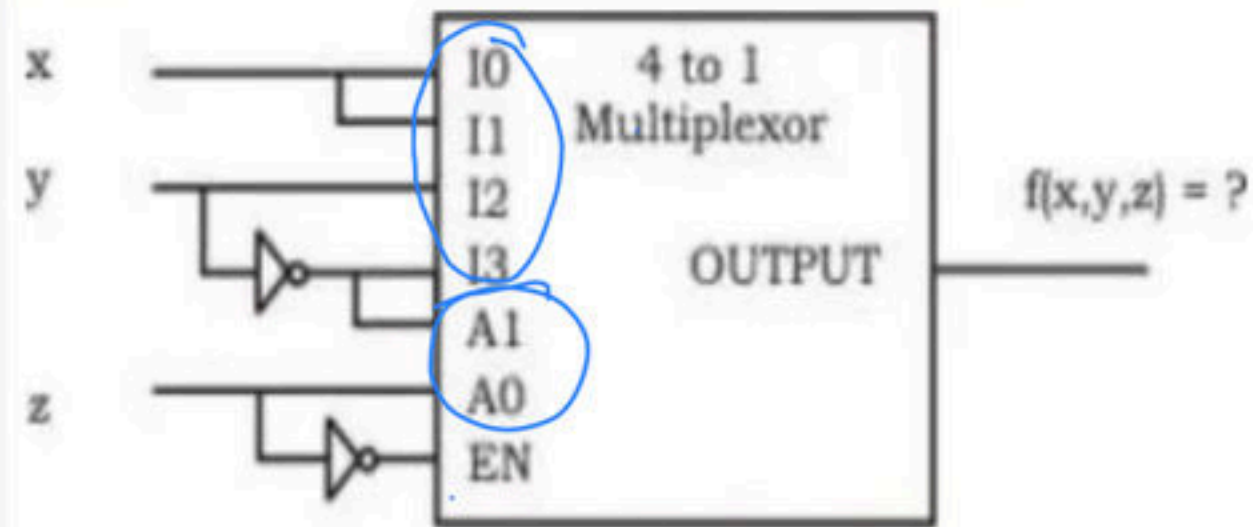
$$Y = [(S'_1 S'_0 I_0) + (S'_1 S_0 I_1) + (S_1 S'_0 I_2) + (S_1 S_0 I_3)]$$

$$\overline{P} \overline{Q} R + \overline{P} Q \overline{R} + P \overline{Q} R + P Q \overline{R}$$

Q Consider the following multiplexor where I_0, I_1, I_2, I_3 are four data input lines selected by two address line combinations $A_1A_0 = 00, 01, 10, 11$ respectively and f is “the output of the multiplexor. EN is the enable input. **(Gate-CS-2002) (2 Marks)**

The function $f(x, y, z)$ implemented by the above circuit is :

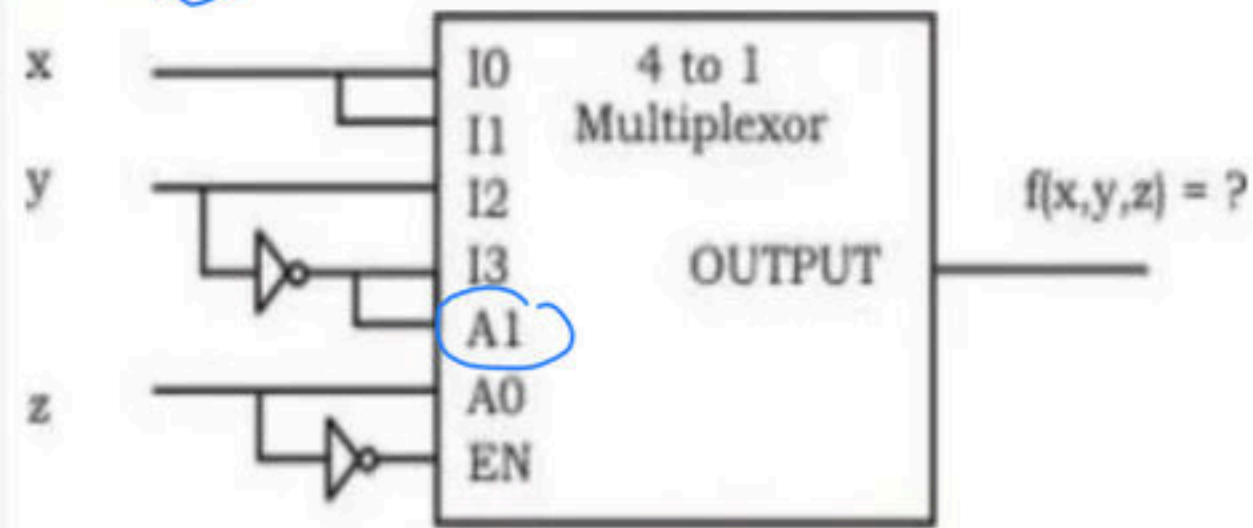
- (A)** xyz' **(B)** $xy + z$ **(C)** $x + z$ **(D)** None of these



Q Consider the following multiplexor where I_0, I_1, I_2, I_3 are four data input lines selected by two address line combinations $A_1A_0 = 00, 01, 10, 11$ respectively and f is “the output of the multiplexor. EN is the enable input. **(Gate-CS-2002) (2 Marks)**

The function $f(x, y, z)$ implemented by the above circuit is :

- (A) xyz' (B) $xy + z$ (C) $x + z$ (D) None of these

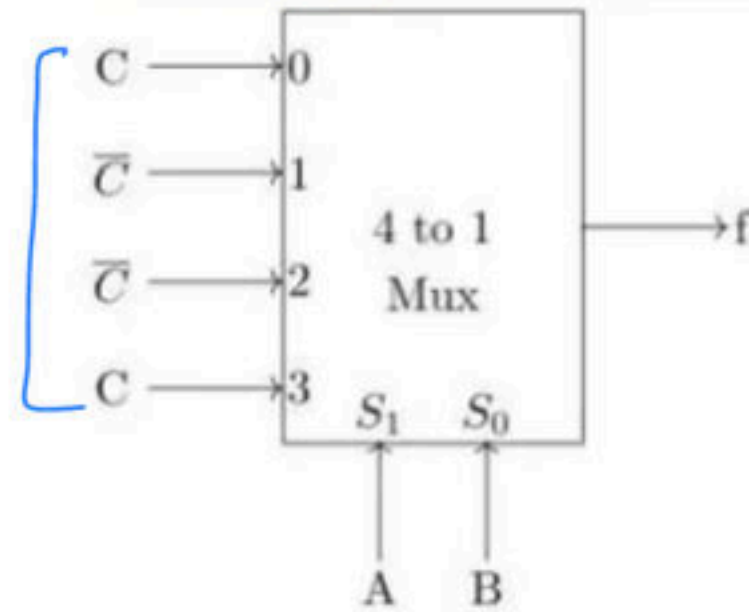


$$Y = E [(S'_1 S'_0 I_0) + (S'_1 S_0 I_1) + (S_1 S'_0 I_2) + (S_1 S_0 I_3)]$$

$$\overline{z} \left[\overline{z}x + \overline{z}y + \overline{z}\overline{y} + \overline{z}\overline{z} \right]$$

$xy\overline{z}$

Q Consider the circuit in below figure. f implements **(GATE-1996) (2 Marks)**



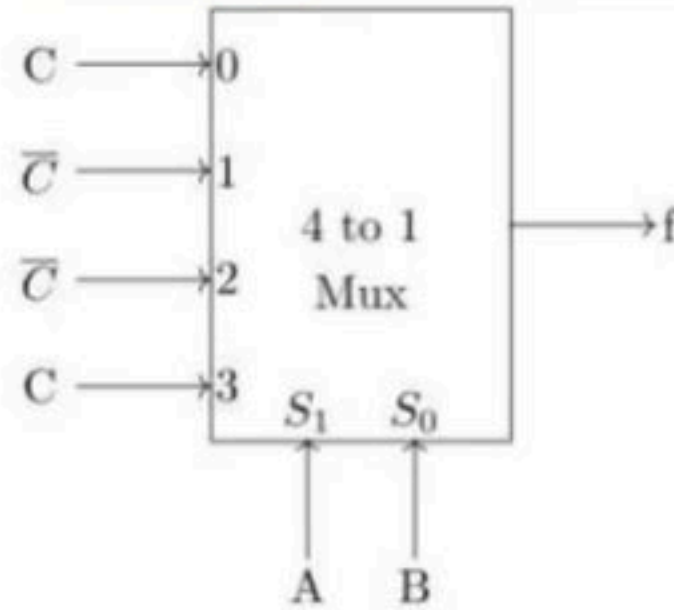
(A) $(ABC)' + A'BC' + ABC$

(B) $A + B + C$

(C) $A \oplus B \oplus C$ 

(D) $AB + BC + CA$

Q Consider the circuit in below figure. f implements **(GATE-1996) (2 Marks)**



(A) $(ABC)' + A'BC' + ABC$

(B) $A + B + C$

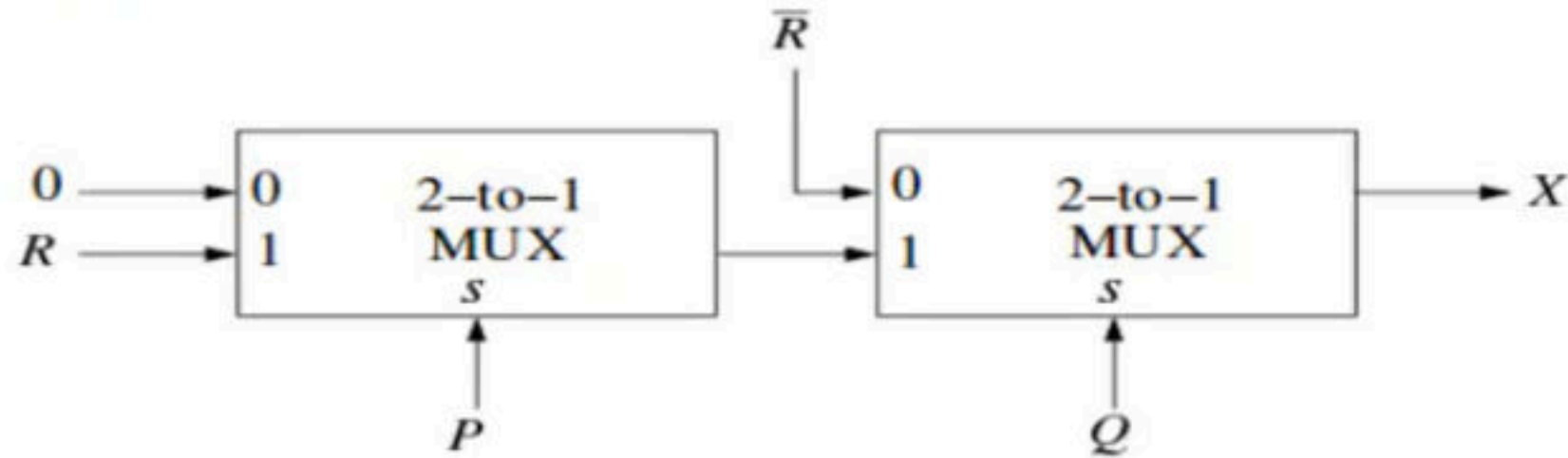
(C) $A \oplus B \oplus C$

(D) $AB + BC + CA$

$$Y = [(S'_1 S'_0 I_0) + (S'_1 S_0 I_1) + (S_1 S'_0 I_2) + (S_1 S_0 I_3)]$$

Many multiplexer

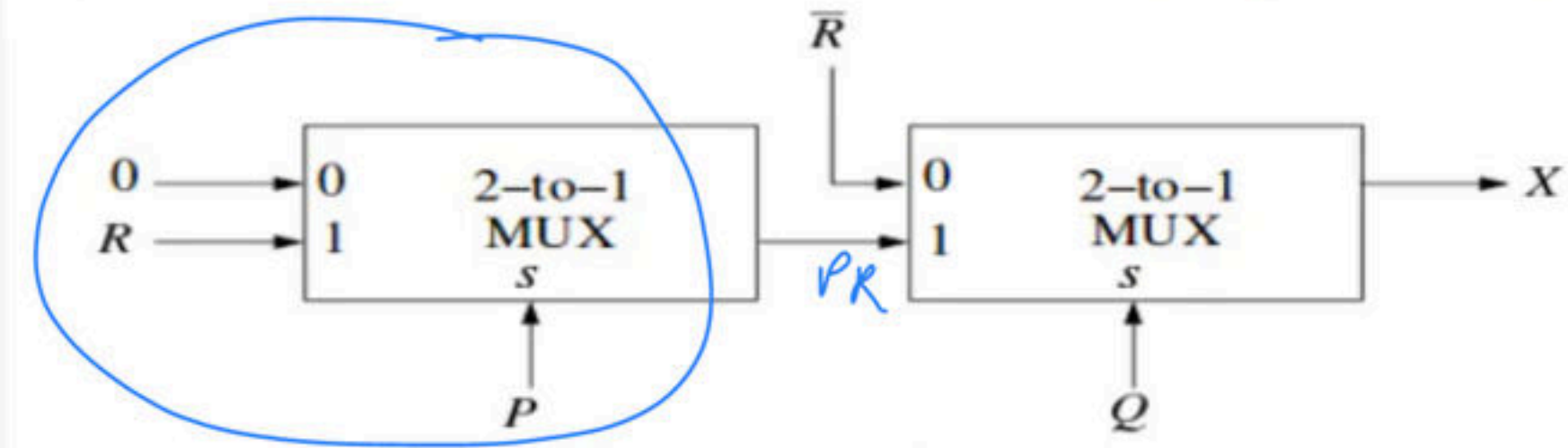
Q Consider the two cascaded 2-to-1 multiplexers as shown in the figure. (GATE-2016) (2 Marks)



The minimal sum of products form of the output X is

- (A) $\bar{P}\bar{Q} + PQR$ -17
- (B) $\bar{P}Q + QR$ -10
- (C) $PQ + \bar{P}\bar{Q}R$ -7
- (D) $\bar{Q}\bar{R} + PQR$ -69

Q Consider the two cascaded 2-to-1 multiplexers as shown in the figure. (GATE-2016) (2 Marks)



The minimal sum of products form of the output X is

(A) $\bar{P}\bar{Q} + PQR$

(B) $\bar{P}Q + QR$

(C) $PQ + \bar{P}\bar{Q}R$

(D) $\bar{Q}\bar{R} + PQR$

$$Y = S_0' I_0 + S_0 I_1$$

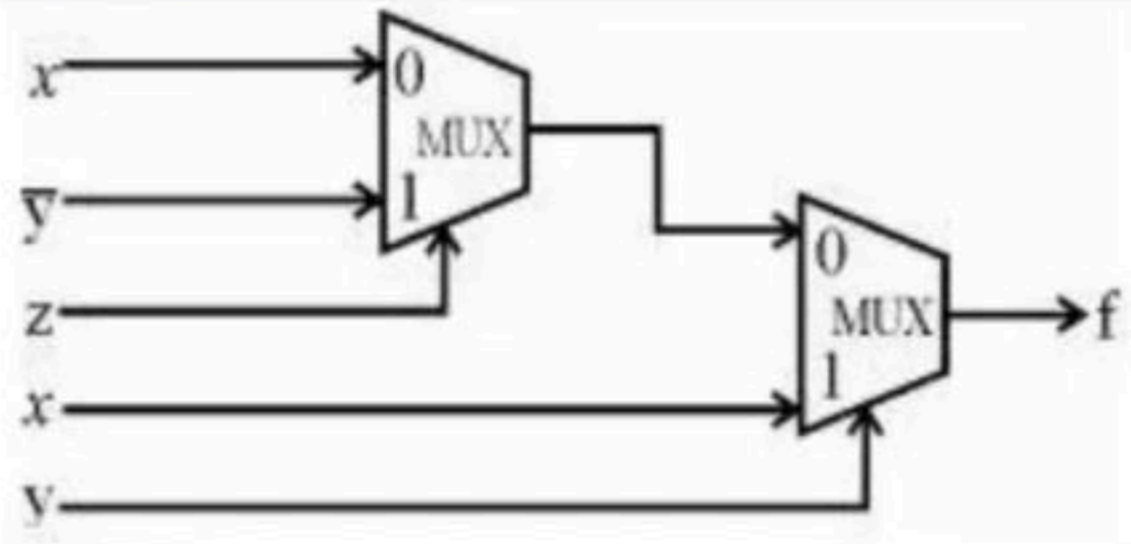
~~$\bar{P} \cdot \bar{Q}$~~ + $P \cdot R = PR$

$$Y = S_0' I_0 + S_0 I_1$$

$\bar{Q}\bar{R} + PQR$

$\bar{Q}\bar{R} + PQR$

Q Consider the below circuit and find the output function $f(x, y, z)$. (NET-JUNE-2012)



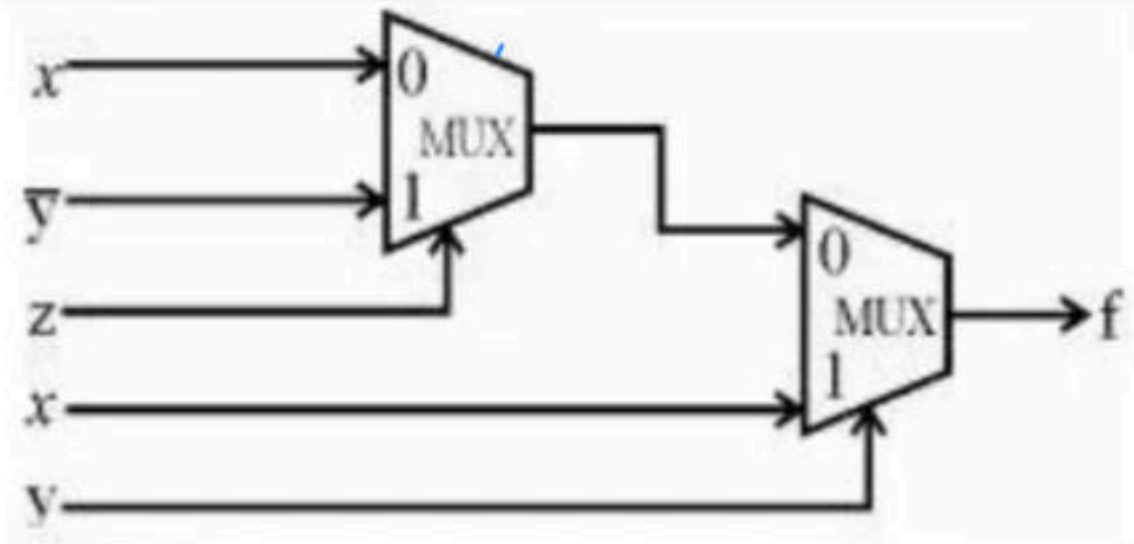
(A) $xz' + xy + y'z$

(B) $xz' + xy + y'z'$

(C) $xz + xy + y'z'$

(D) $xz + xy' + y'z$

Q Consider the below circuit and find the output function $f(x, y, z)$. (NET-JUNE-2012)



$$Y = S_0' I_0 + S_0 I_1$$

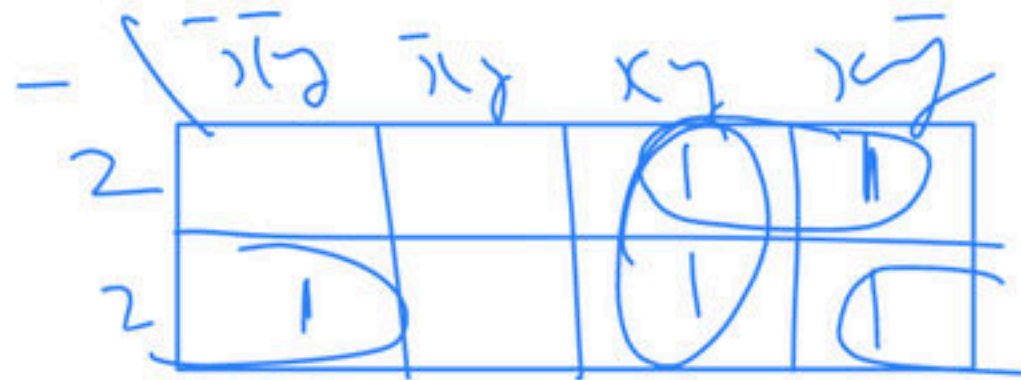
$$\bar{z} \cdot x + z \cdot \bar{y}$$

(A) ~~$xz' + xy + y'z$~~ ✓

(B) $xz' + xy + y'z'$

(C) $xz + xy + y'z'$

(D) $xz + xy' + y'z$



$$Y = S_0' I_0 + S_0 I_1$$

$$\bar{y} (\bar{z} x + z y) + y \cdot x$$

$$\cancel{\bar{y} \bar{z}} + z \bar{y} + x y$$

Q The following circuit implements a two-input AND gate using two 2-1 multiplexers. **(GATE-2007) (1 Marks)**

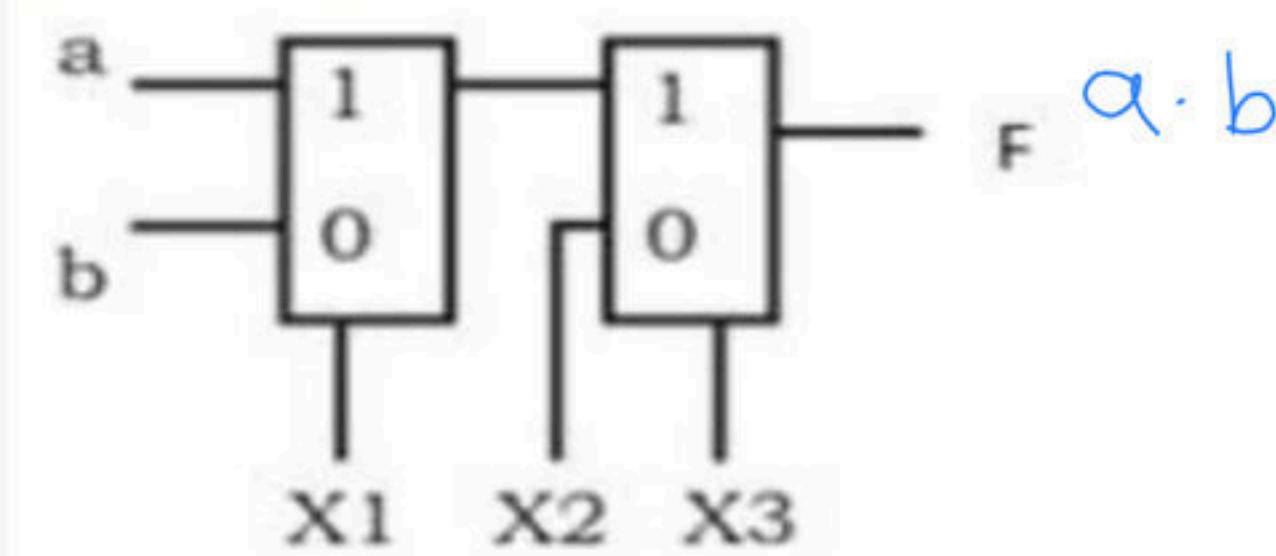
What are the values of X_1 , X_2 , X_3 ?

(A) $X_1=b$, $X_2=0$, $X_3=a$

(C) $X_1=a$, $X_2=b$, $X_3=1$

(B) $X_1=b$, $X_2=1$, $X_3=b$

(D) $X_1=a$, $X_2=0$, $X_3=b$



Q The following circuit implements a two-input AND gate using two 2-1 multiplexers. **(GATE-2007) (1 Marks)**

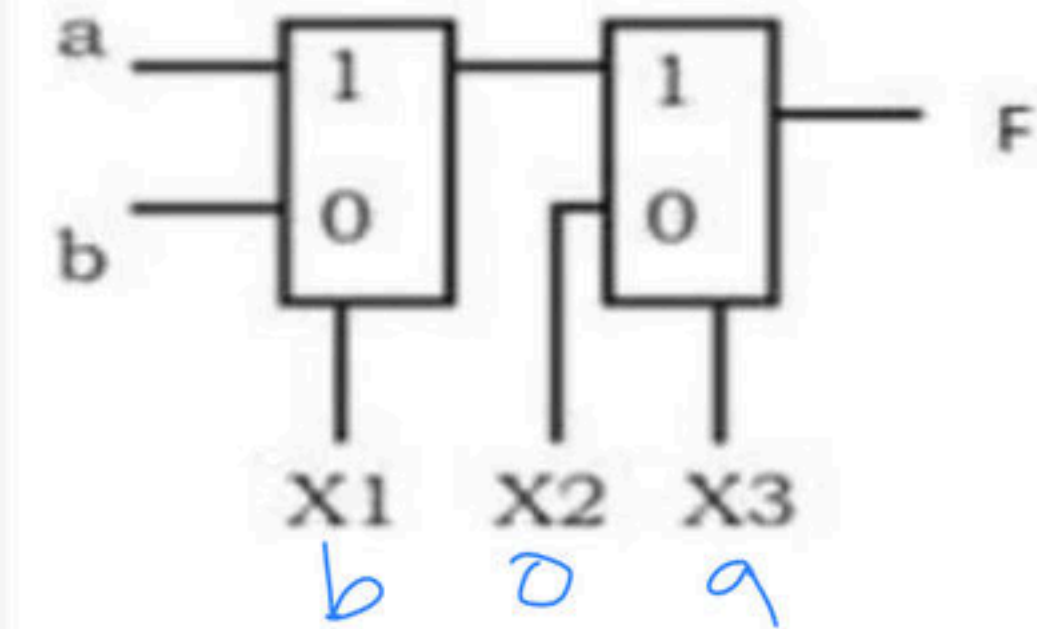
What are the values of X_1, X_2, X_3 ?

(A) $X_1=b, X_2=0, X_3=a$ ✓

(C) $X_1=a, X_2=b, X_3=1$

(B) $X_1=b, X_2=1, X_3=b$

(D) $X_1=a, X_2=0, X_3=b$



$$Y = S_0' I_0 + S_0 I_1$$

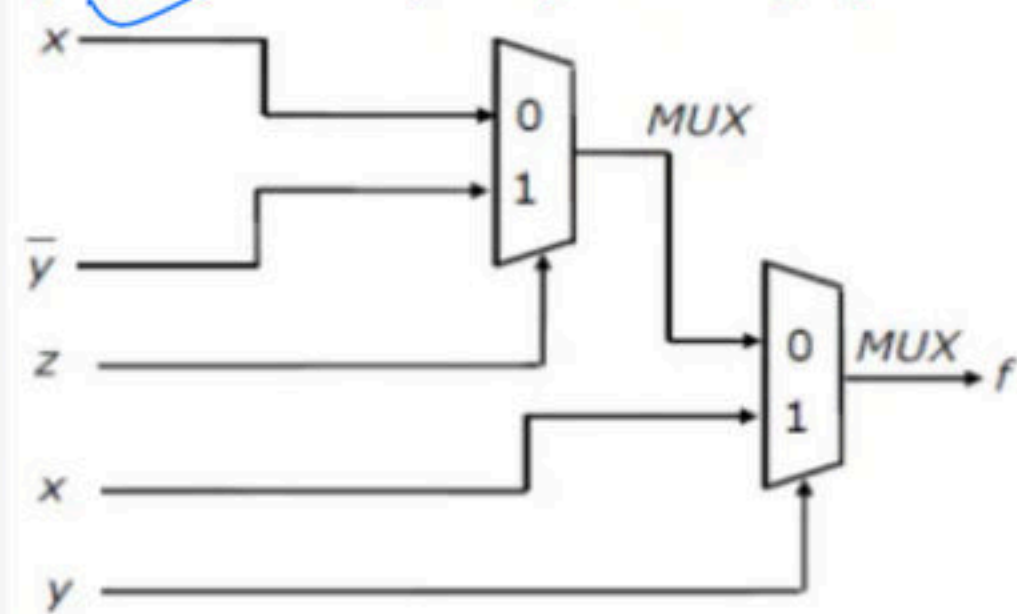
$$\cancel{b} \cancel{b} + b a = \boxed{ab}$$

$$Y = S_0' I_0 + S_0 I_1$$

$$\cancel{a} \cdot \cancel{0} + a \cdot ab = \boxed{ab}$$

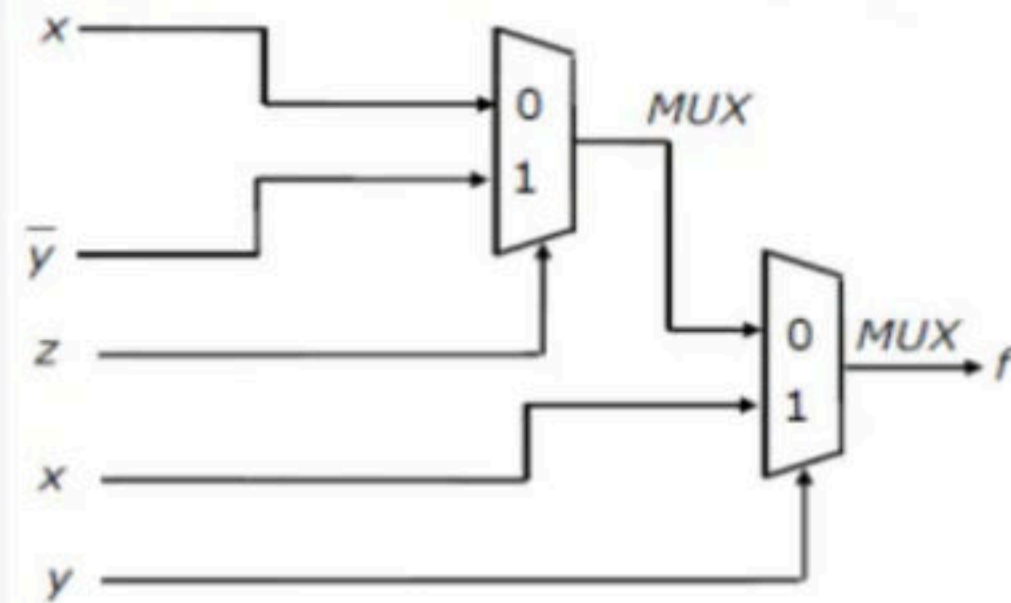
Q Consider the circuit above. Which one of the following options correctly represents $f(x, y, z)$? (**Gate-CS-2006**) (2 Marks)

- (A)** $xz' + xy + y'z$ **(B)** $xz' + xy + (yz)'$ **(C)** $xz + xy + (yz)'$ **(D)** $xz + xy' + y'z$



Q Consider the circuit above. Which one of the following options correctly represents $f(x, y, z)$? (**Gate-CS-2006**) (2 Marks)

(A) $xz' + xy + y'z$ **(B)** $xz' + xy + (yz)'$ **(C)** $xz + xy + (yz)'$ **(D)** $xz + xy' + y'z$

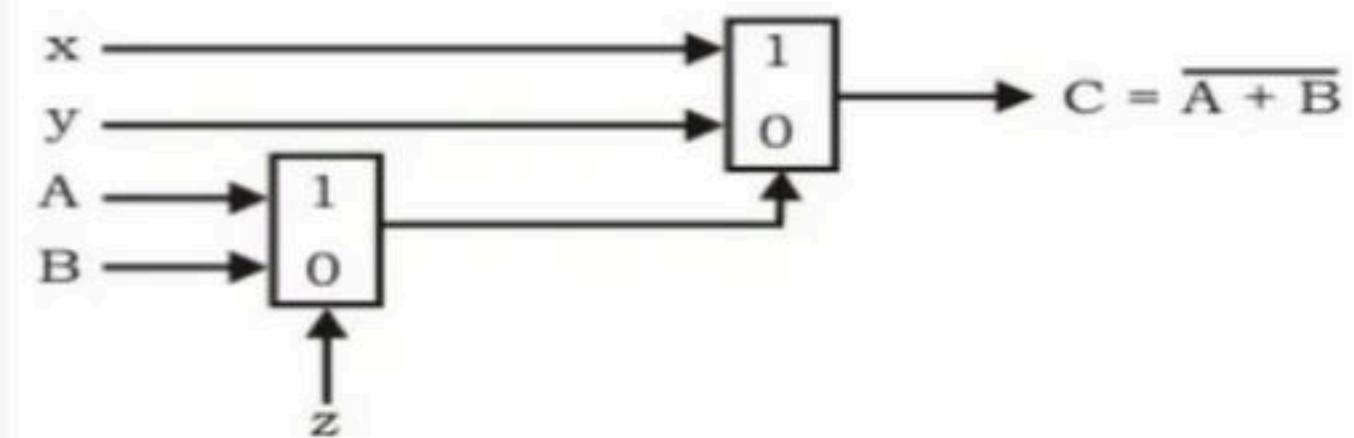


$$Y = S_0' I_0 + S_0 I_1$$

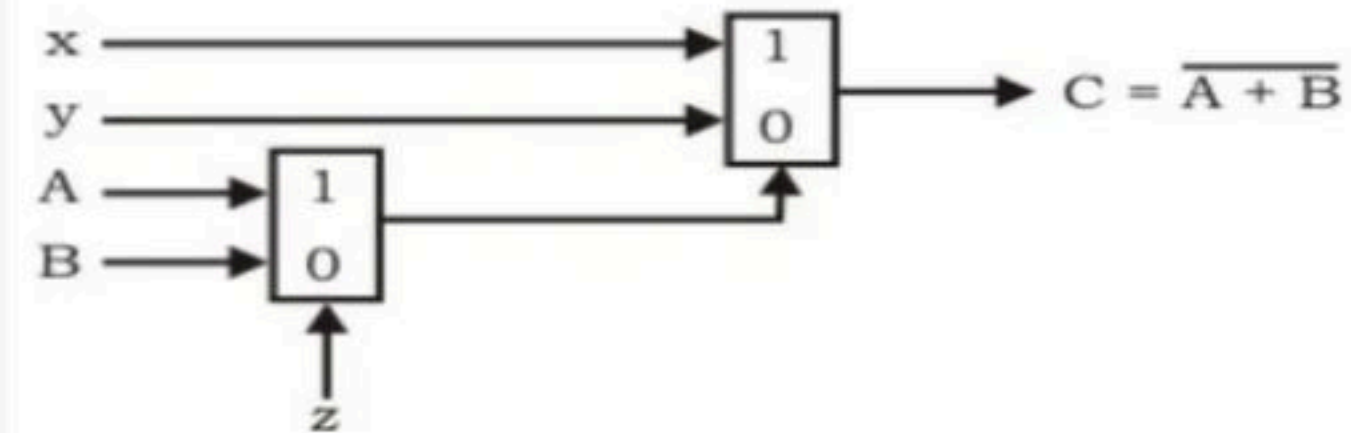
$$Y = S_0' I_0 + S_0 I_1$$

Q The circuit shown below implements a 2-input NOR gate using two 2-4 MUX (control signal 1 selects the upper input). What are the values of signals x, y and z? **(GATE-2005) (2 Marks)**

(A) 1, 0, B **(B)** 1, 0, A **(C)** 0, 1, B **(D)** 0, 1, A



(D) 0, 1, A

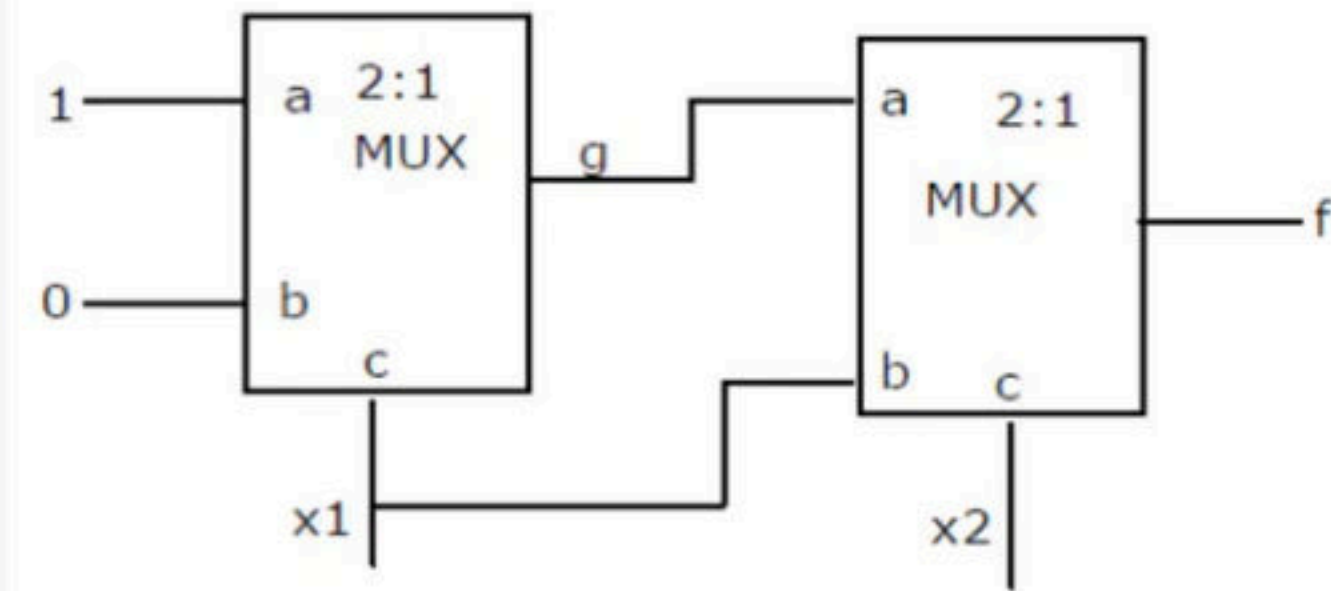


$$Y = S_0' I_0 + S_0 I_1$$

$$Y = S_0' I_0 + S_0 I_1$$

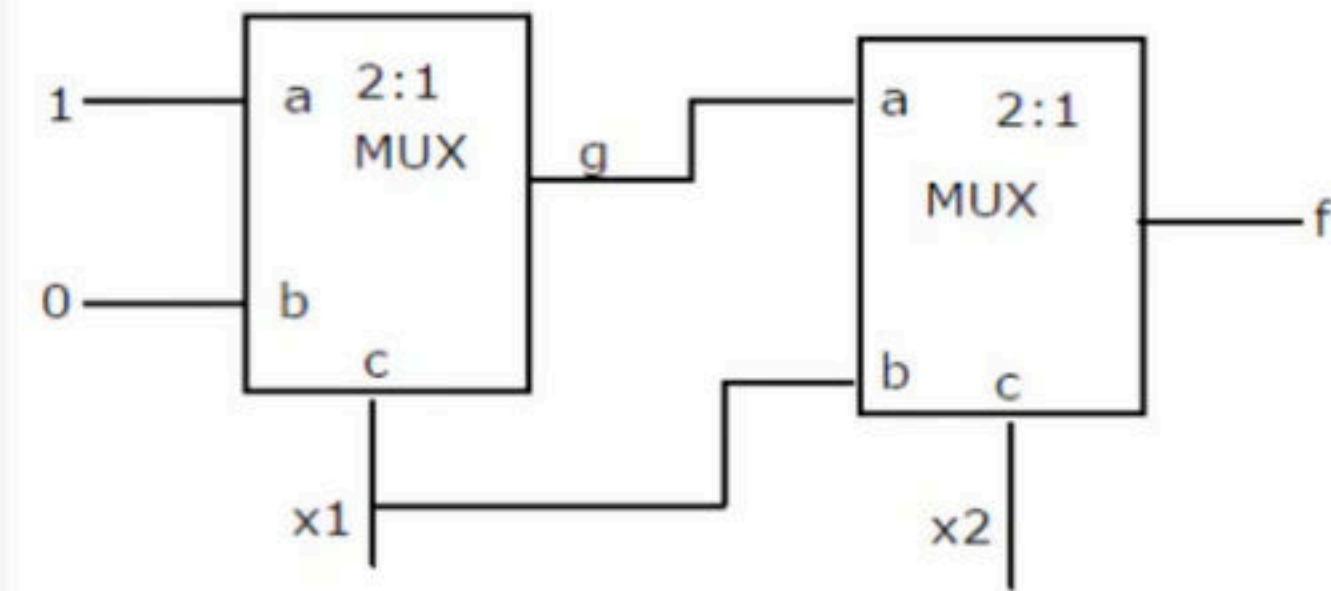
Q Consider the circuit shown below. The output of a 2:1 Mux is given by the function $(ac' + bc)$, Which of the following is true? **(GATE-2001) (2 Marks)**

- (A)** $f = x_1' + x_2$ **(B)** $f = x_1'x_2 + x_1x_2'$ **(C)** $f = x_1x_2 + x_1'x_2'$ **(D)** $f = x_1 + x_2'$



Q Consider the circuit shown below. The output of a 2:1 Mux is given by the function $(ac' + bc)$, Which of the following is true? **(GATE-2001) (2 Marks)**

- (A)** $f = x_1' + x_2$ **(B)** $f = x_1'x_2 + x_1x_2'$ **(C)** $f = x_1x_2 + x_1'x_2'$ **(D)** $f = x_1 + x_2'$



$$Y = S_0' I_0 + S_0 I_1$$

$$Y = S_0' I_0 + S_0 I_1$$

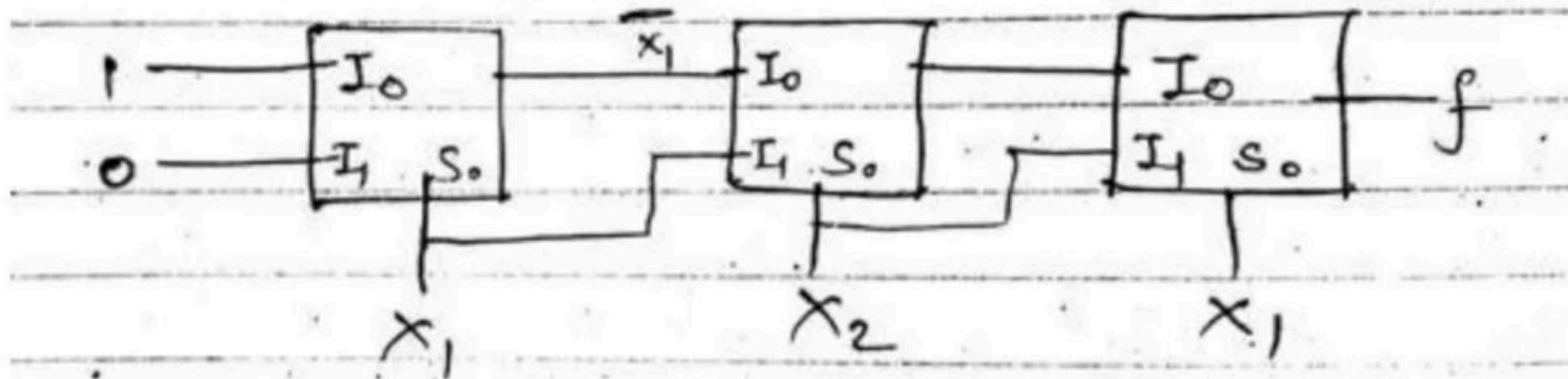
Q Consider the circuit shown below. Which of the following is true?

(A) $f = x_1' + x_2$

(B) $f = x_1'x_2 + x_1x_2'$

(C) $f = x_1x_2 + x_1'x_2'$

(D) none



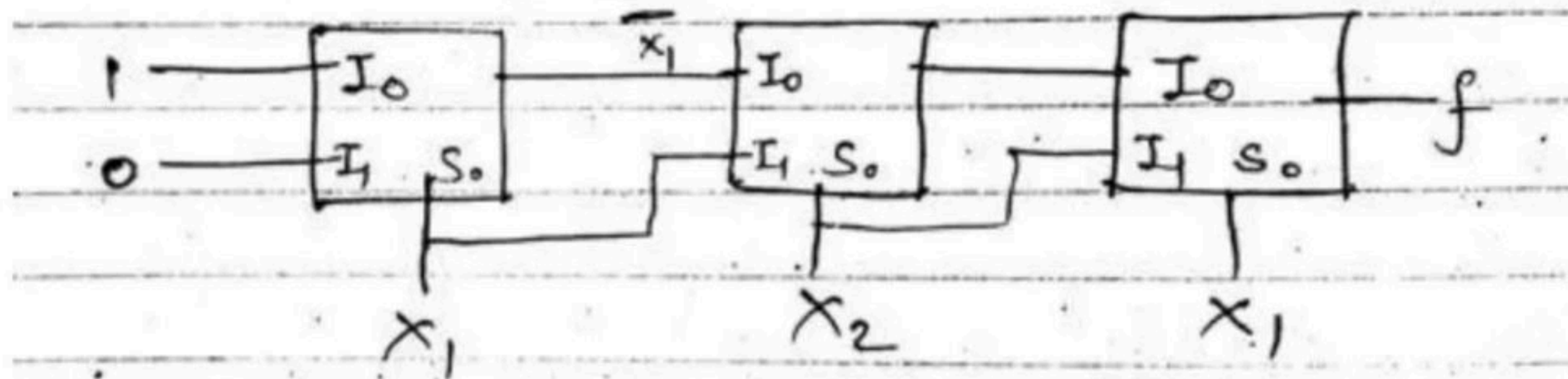
Q Consider the circuit shown below. Which of the following is true?

(A) $f = x_1' + x_2$

(B) $f = x_1'x_2 + x_1x_2'$

(C) $f = x_1x_2 + x_1'x_2'$

(D) none



$$Y = S_0' I_0 + S_0 I_1$$

$$Y = S_0' I_0 + S_0 I_1$$

$$Y = S_0' I_0 + S_0 I_1$$

Q Suppose only one multiplexer and one inverter are allowed to be used to implement any Boolean function of n variables. What is the minimum size of the multiplexer needed? **(Gate-CS-2007) (2 Marks)**

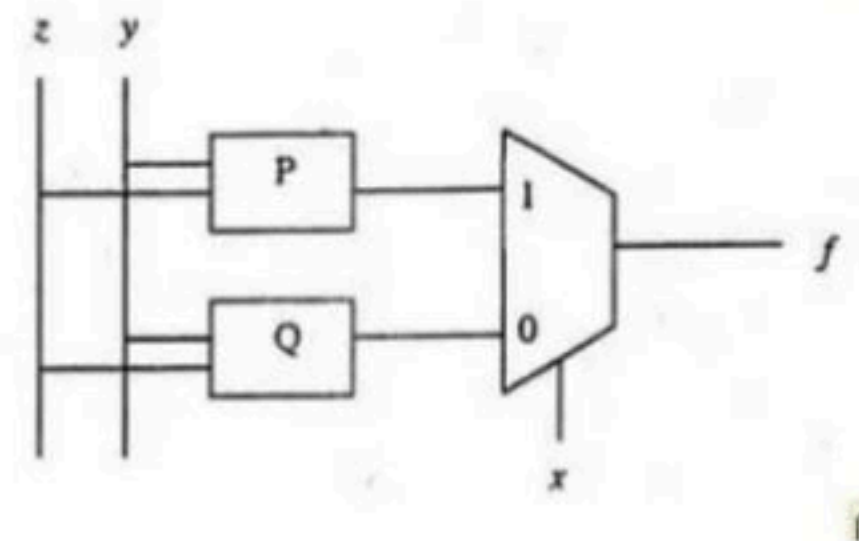
- (A)** 2^n line to 1-line
(C) 2^{n-1} line to 1-line

- (B)** 2^{n+1} line to 1 line
(D) 2^{n-2} line to 1 line

In order to implement a n variable switching function, a MUX must have :

- (A) 2^n inputs (B) $2^n + 1$ inputs (C) 2^{n-1} inputs (D) $2^n - 1$ inputs

Q The majority function is a Boolean function $f(x, y, z)$ that takes the value 1 whenever a majority of the variables x, y, z is 1. In the circuit diagram for the majority function shown below, the logic gates for the boxes labelled P and Q are, respectively? **(GATE-2006) (2 Marks)**



(A) XOR, AND

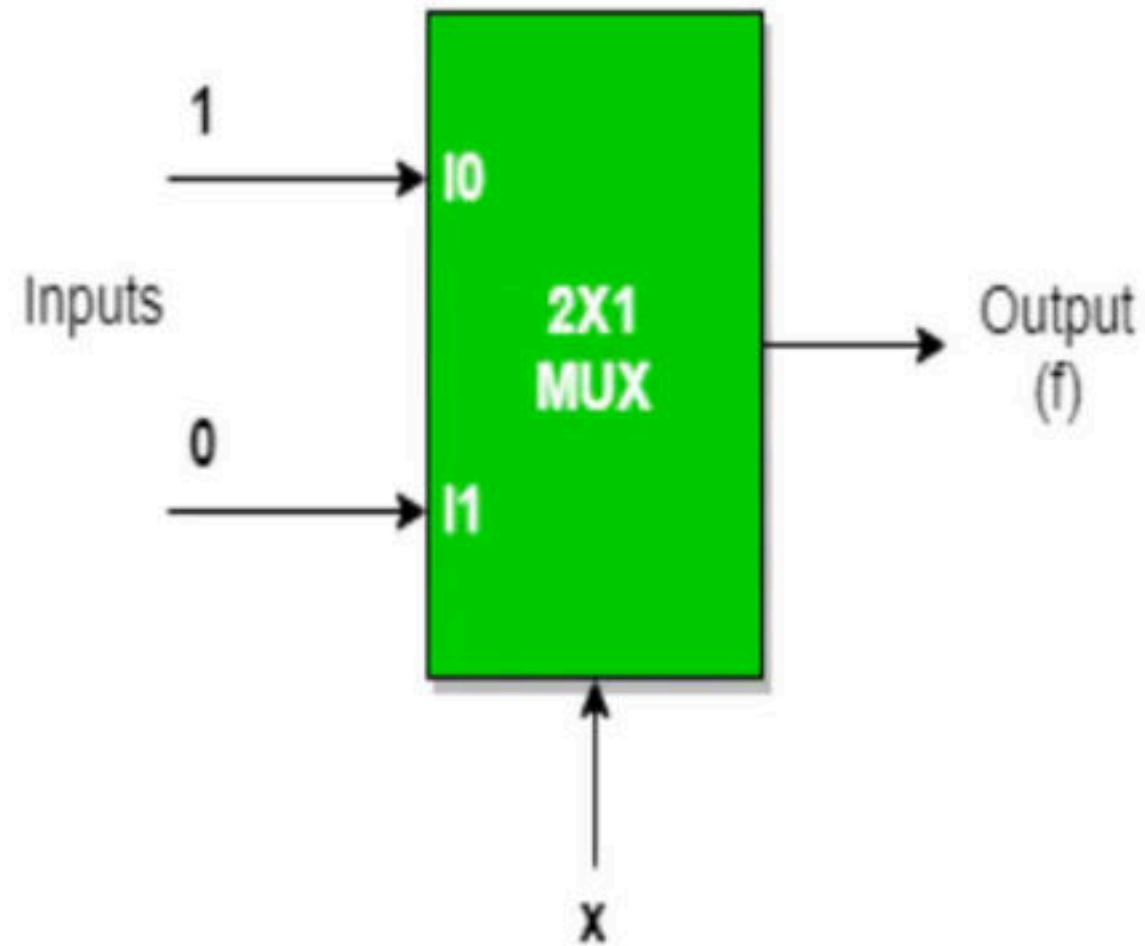
(B) XOR, XOR

(C) OR, OR

(D) OR, AND

NOT GATE

NOT GATE



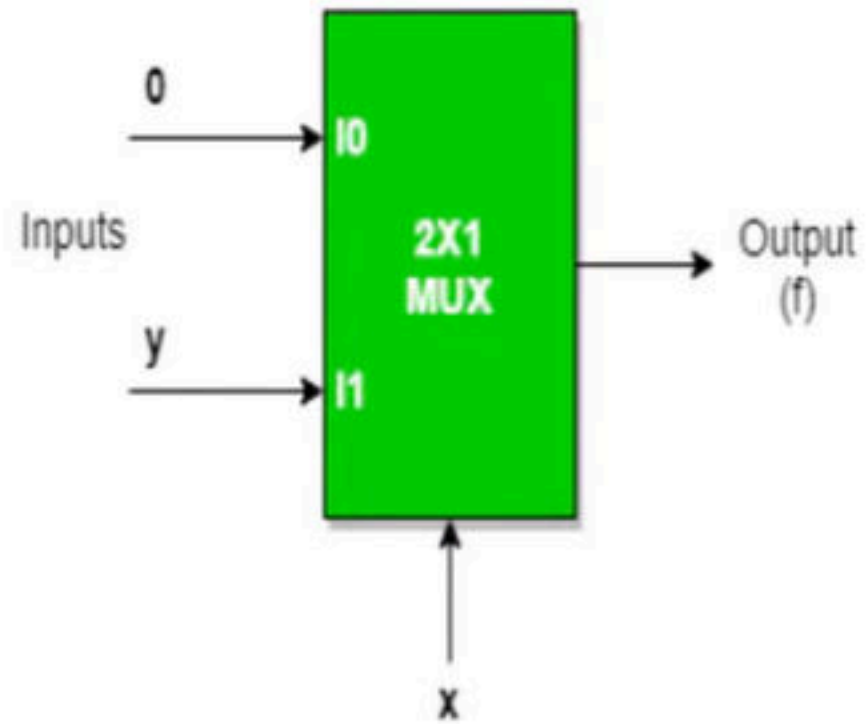
Truth Table

x	f
0	1
1	0

AND GATE

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

AND GATE



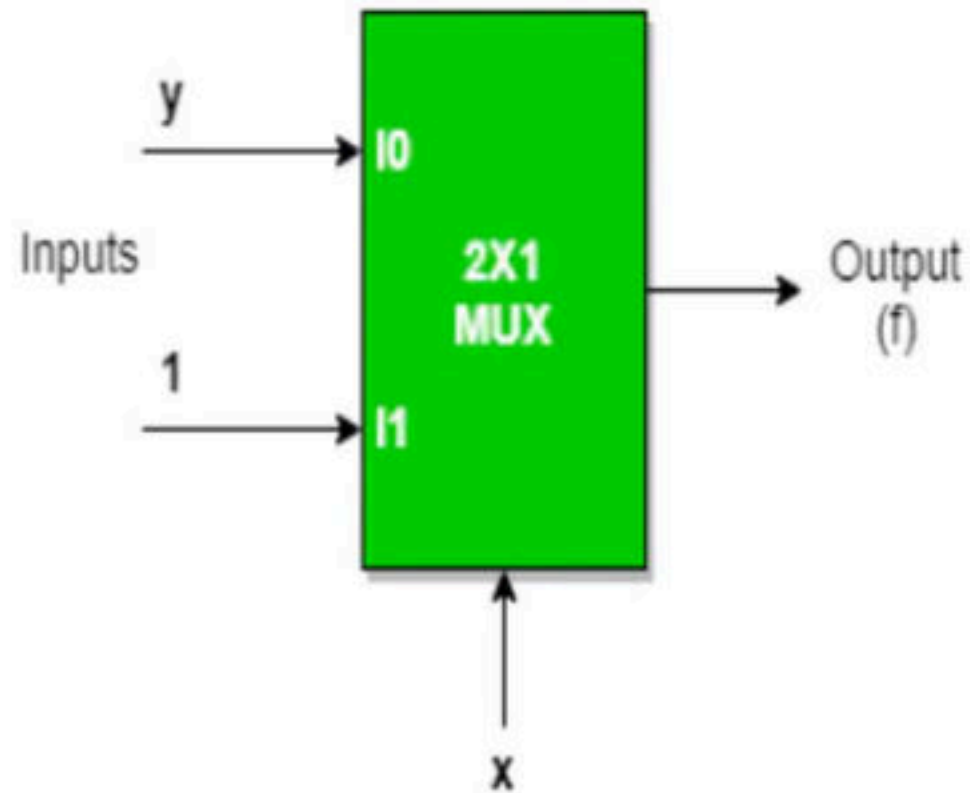
Truth Table

x	y	f	$f \rightarrow y$
0	0	0	f = 0
0	1	0	
1	0	0	f = y
1	1	1	

OR GATE

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

OR GATE



Truth Table

x	y	f	$f \rightarrow y$
0	0	0	f = y
0	1	1	
1	0	1	f = 1
1	1	1	

Multiplexer Expansion

(Implementation of Higher order MUX using lower order MUX)

Q 2 : 1 MUX are required to implement 4 : 1 MUX ?

Input		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Q 16 : 1 MUX using 4 : 1 MUX ?

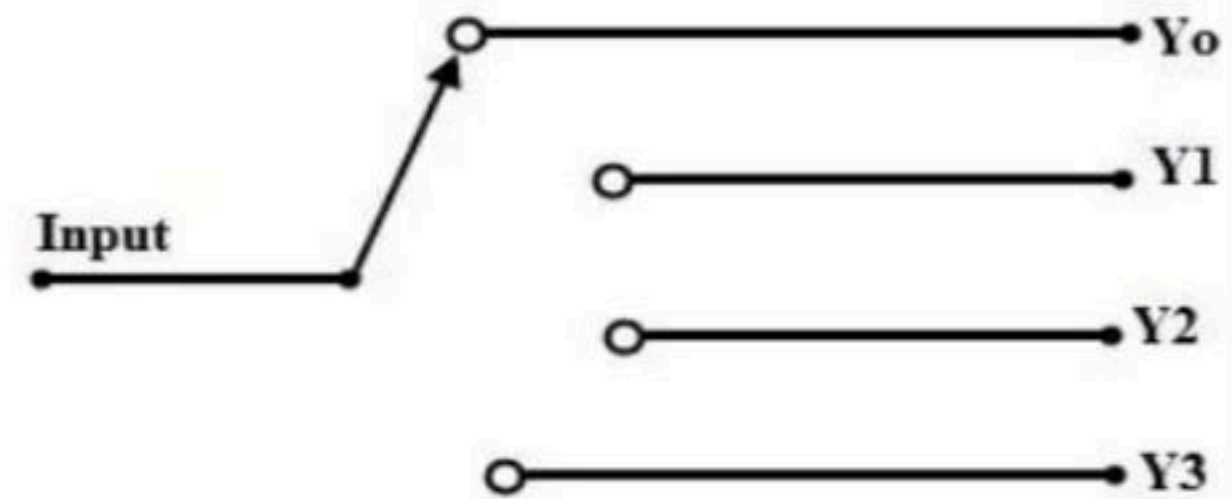
Q how many 4X1 Mux are required in order to construct 128X1?

Q how many 8X1 Mux are required in order to construct 4096X1?

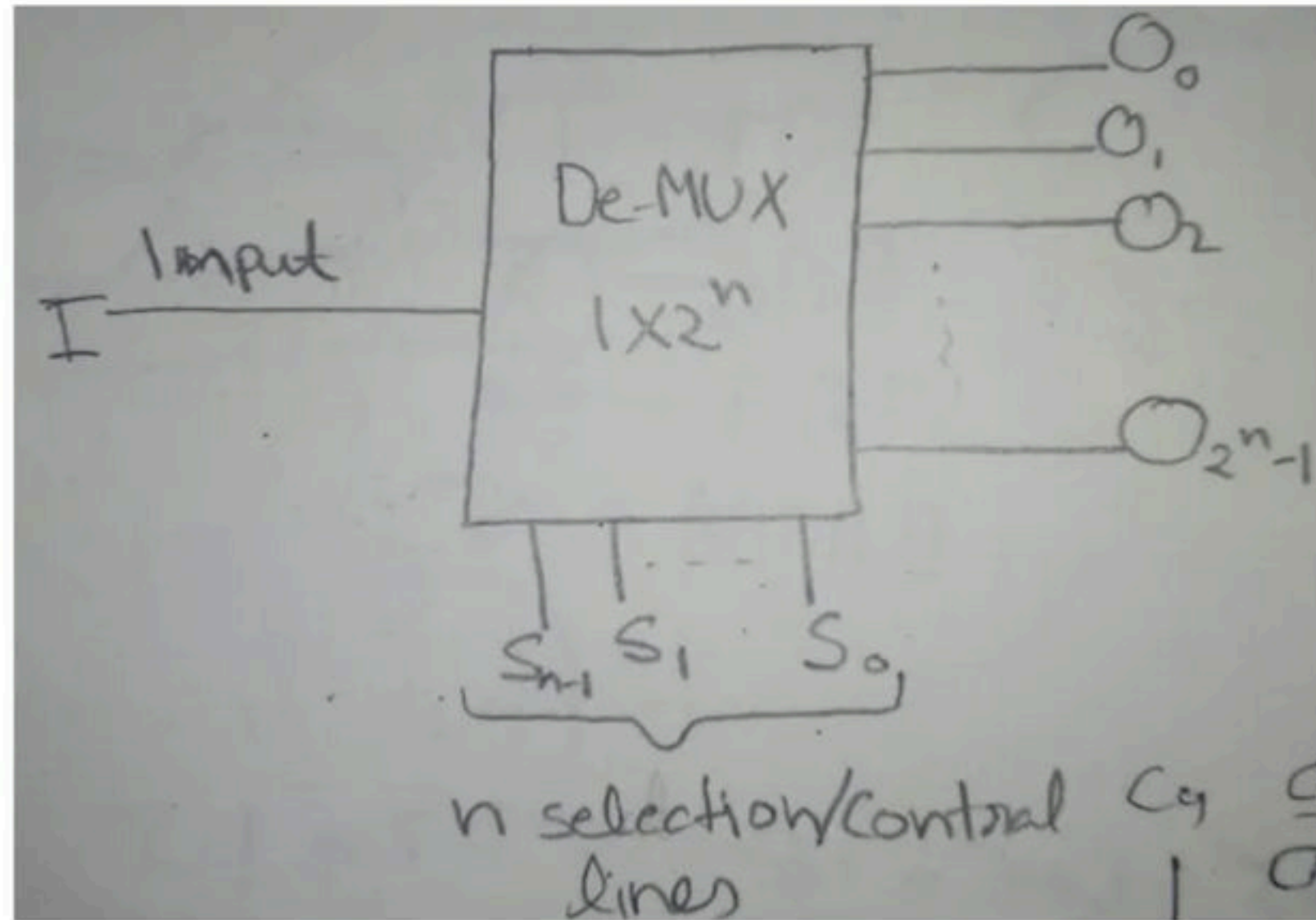
Given:	m X 1
Target:	n X 1
No of levels(K)	$\log_m n$
No of Mux at i th level (x_i)	(n/m_i)
Total Mux required	$\sum_{i=1}^{i=k} x_i$
Maximum capacity	$m^k \times 1$

Demultiplexer

1. A demultiplexer (or DeMux) is a device that takes a single input line and routes it to one of several digital output lines.
2. A demultiplexer is also called a data distributor.
3. It is conceptually same as Mux just with reverse logic.



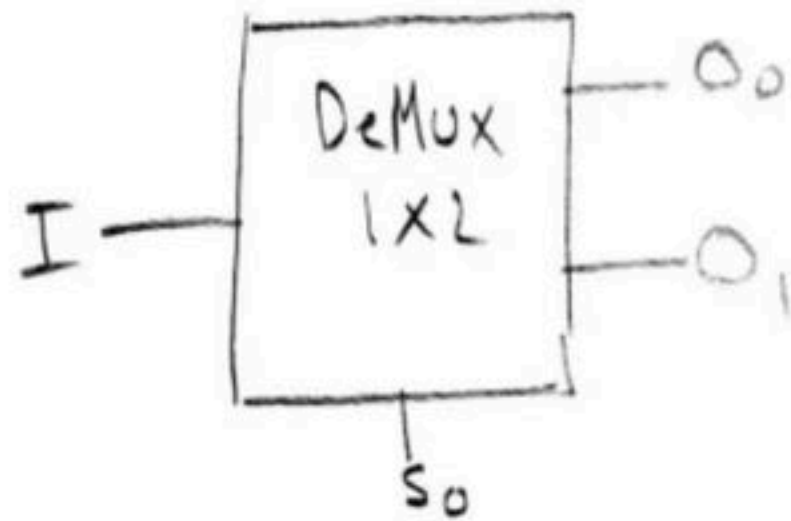
- A demultiplexer of 2^n outputs has n select lines, which are used to select which output line to send the input.



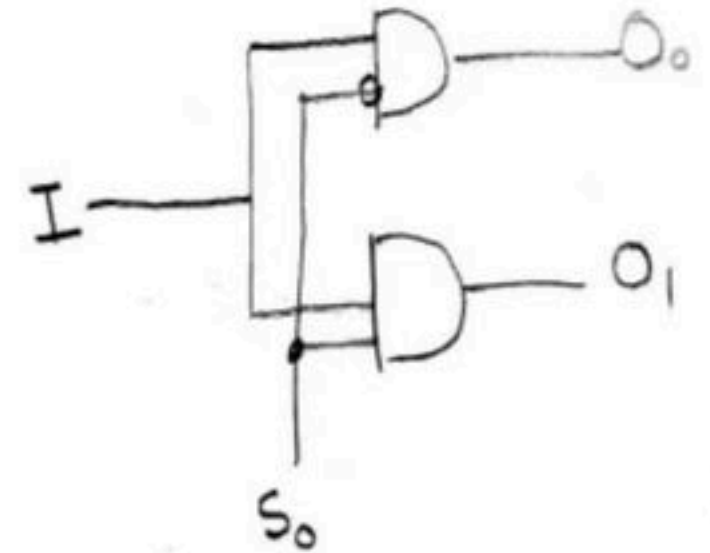
1 to 2 Demultiplexer

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

1 to 2 Demultiplexer



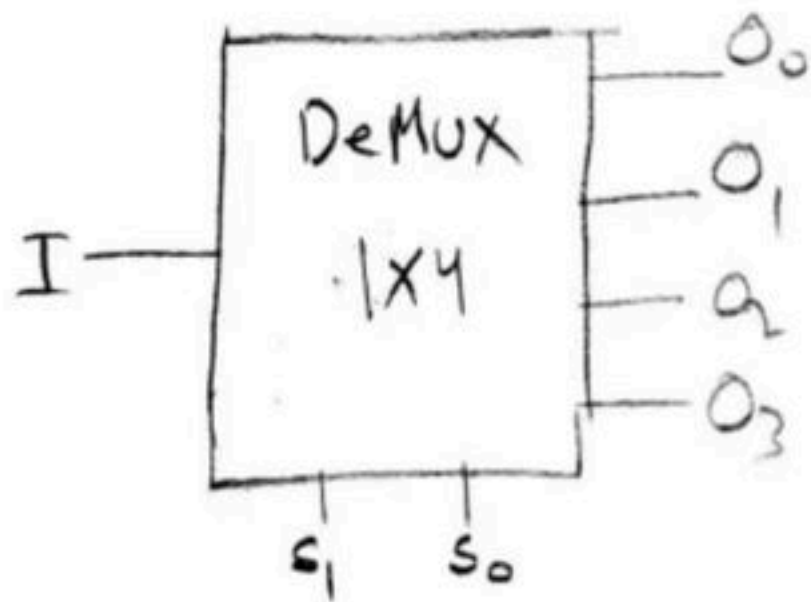
S_0	O_1	O_0
0	0	1
1	1	0



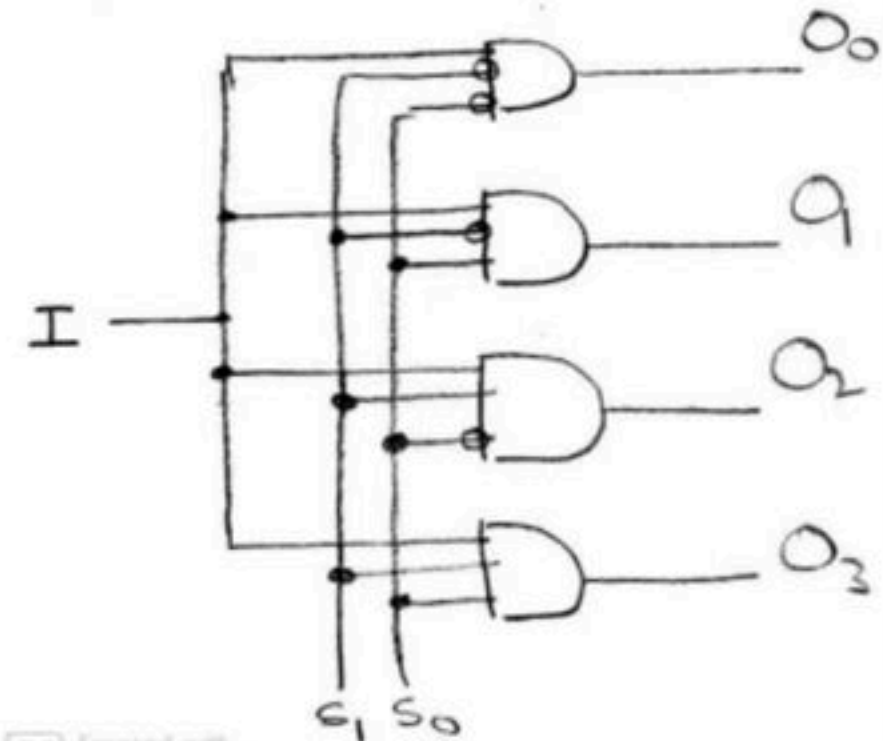
1 to 4 Demultiplexer

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

1 to 4 Demultiplexer



S_1	S_0	O_3	O_2	O_1	O_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



Demultiplexer Expansion

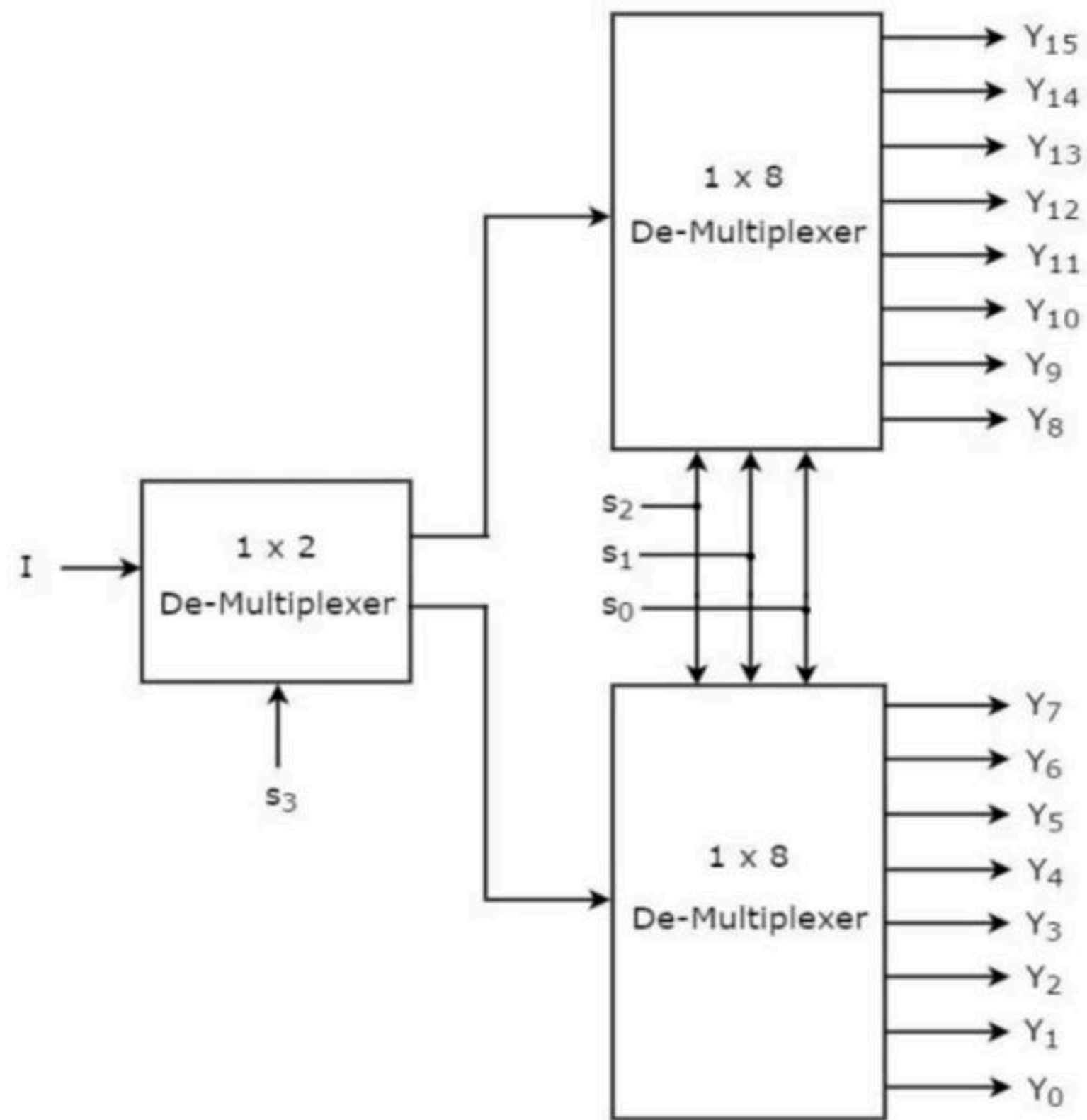
(Implementation of Higher order DeMux using lower order DeMux)

Q 1 : 2 DeMux are required to implement 1 : 4 DeMux ?

Demultiplexer Expansion

(Implementation of Higher order DeMux using lower order DeMux)

Q 1 : 2 DeMux are required to implement 1 : 8 DeMux ?



- A DeMux is a combinational circuit, which is used in data communication
- Serial to parallel conversation
- Different input/output configuration demultiplexers are available in the form of single integrated circuits (ICs).
- Demultiplexers are mainly used in Boolean function generators and decoder circuits.

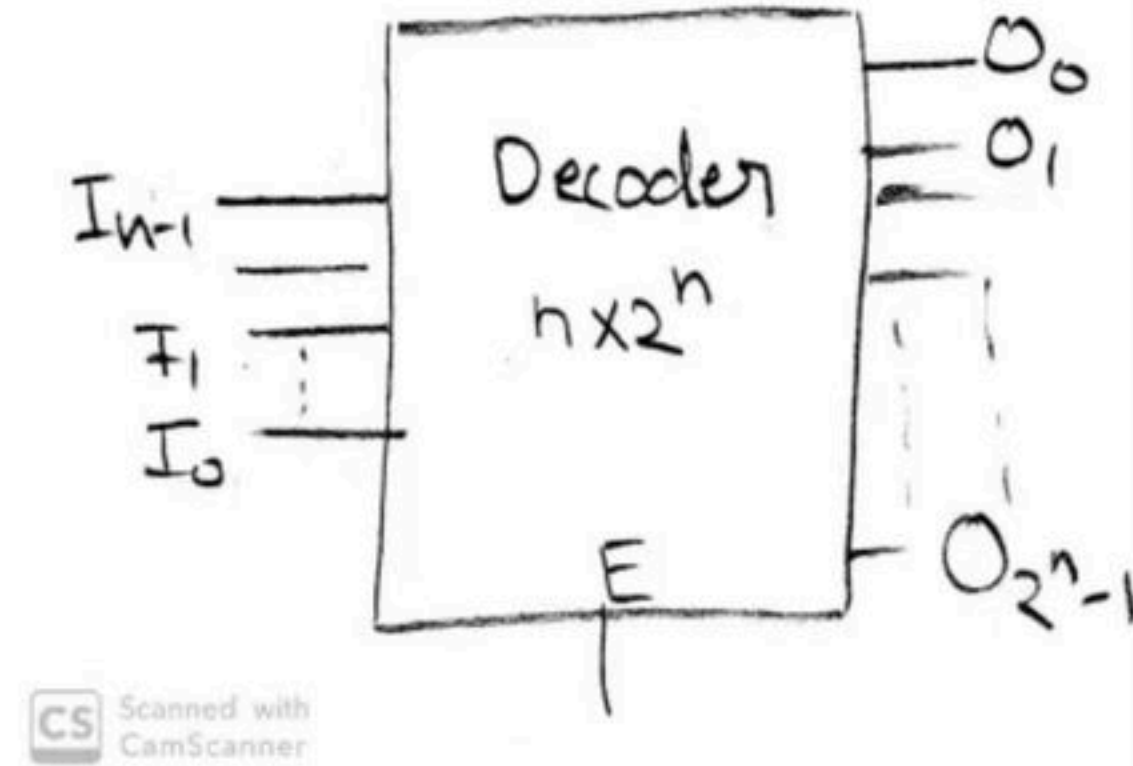
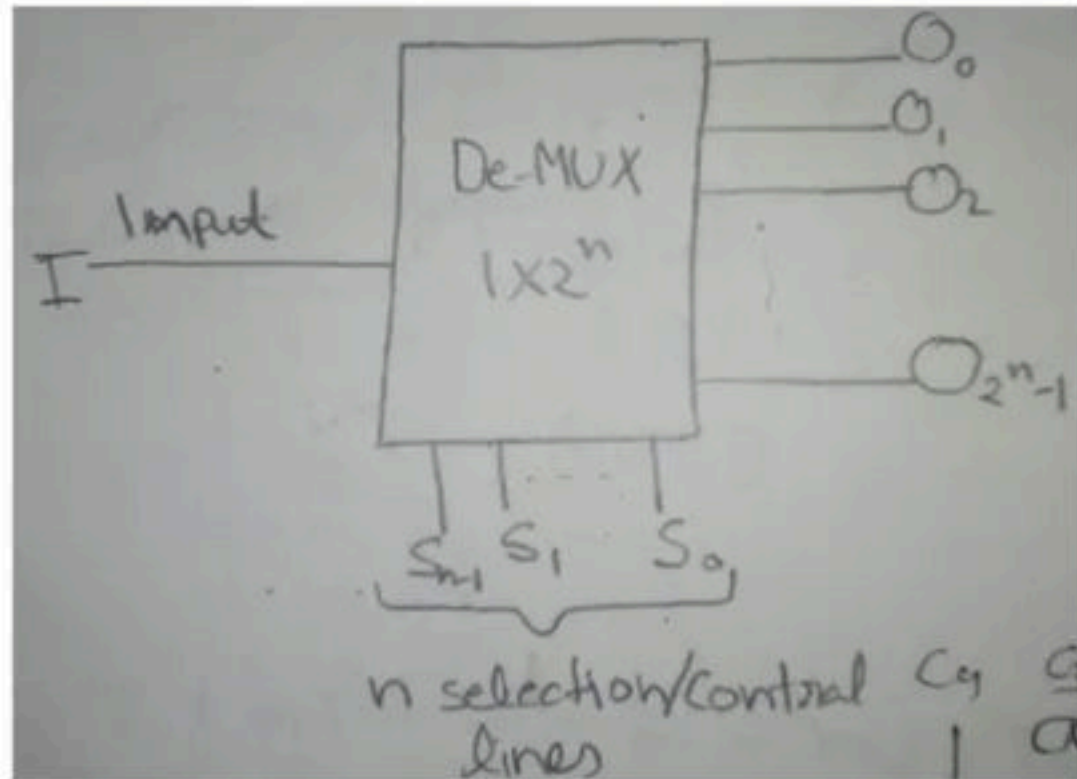
Decoder

- A decoder is a combinational circuit that decodes binary information from n input lines to a maximum of 2^n unique output lines.
- The decoders are called n -to- m -line decoders, where $m \leq 2^n$.
- Their purpose is to generate the 2^n (or fewer) minterms of n input variables. Each combination of inputs will assert a unique output.
- If the n -bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.



Decoder

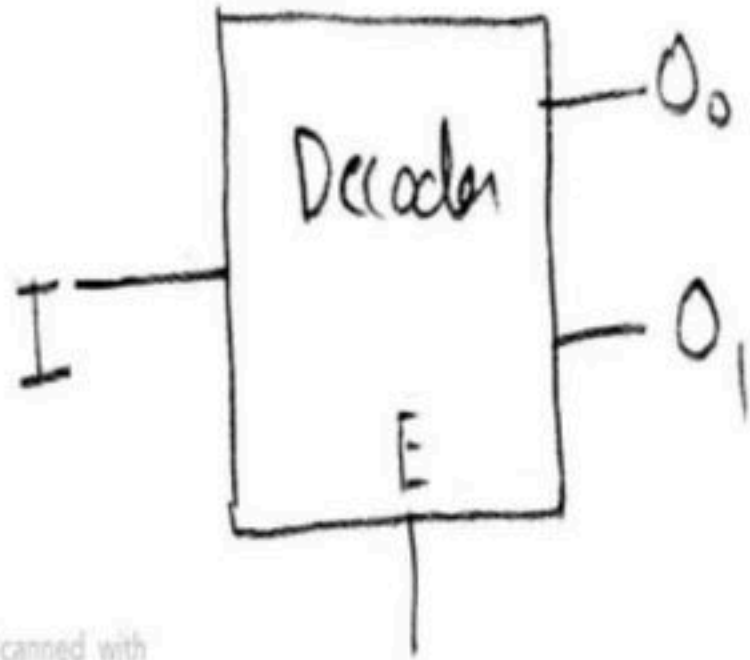
- Decoders are also combinational circuits logically we can say a DeMux can be converted into a decoder by setting input line as enable line and selection line as input lines.



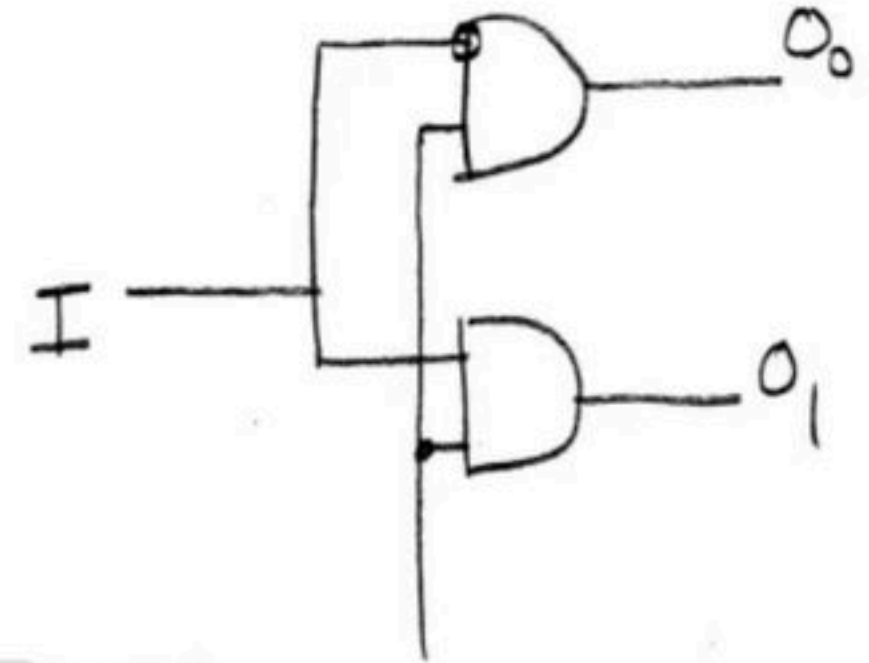
1-to-2 Decoder

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

1-to-2 Decoder



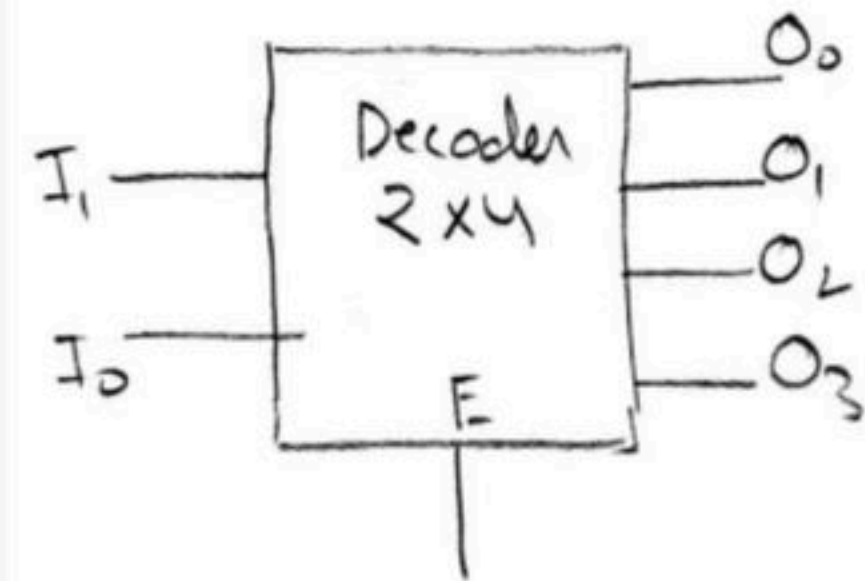
I	O ₁	O ₀
0	0	1
1	1	0



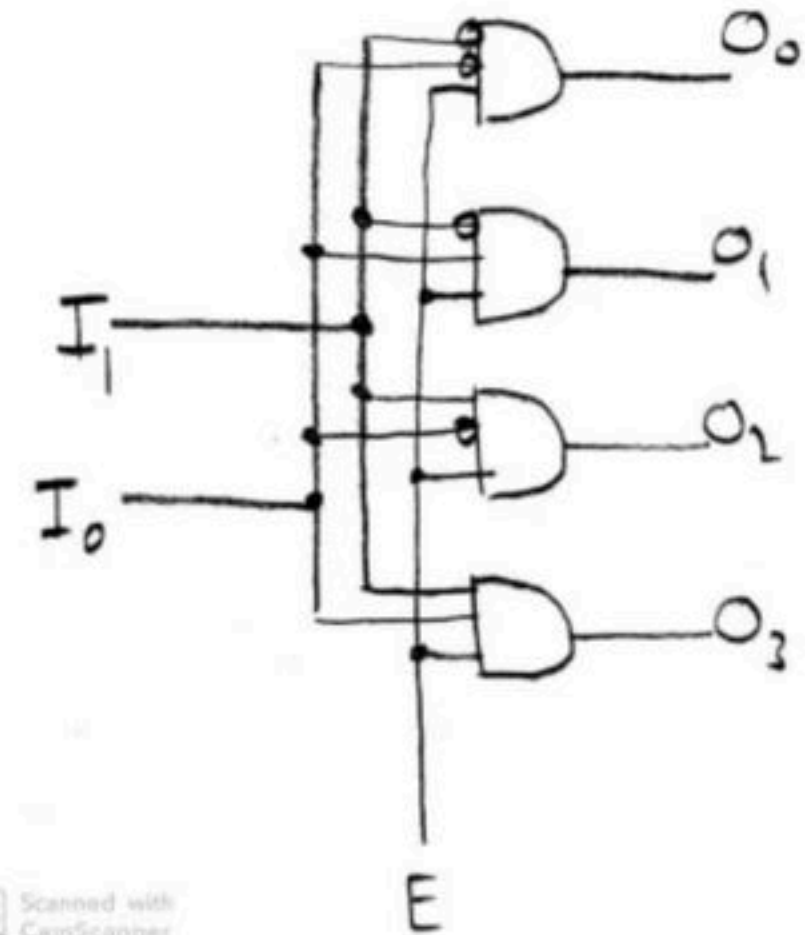
2-to-4 Decoder

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

2-to-4 Decoder



I_1	I_0	O_3	O_2	O_1	O_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

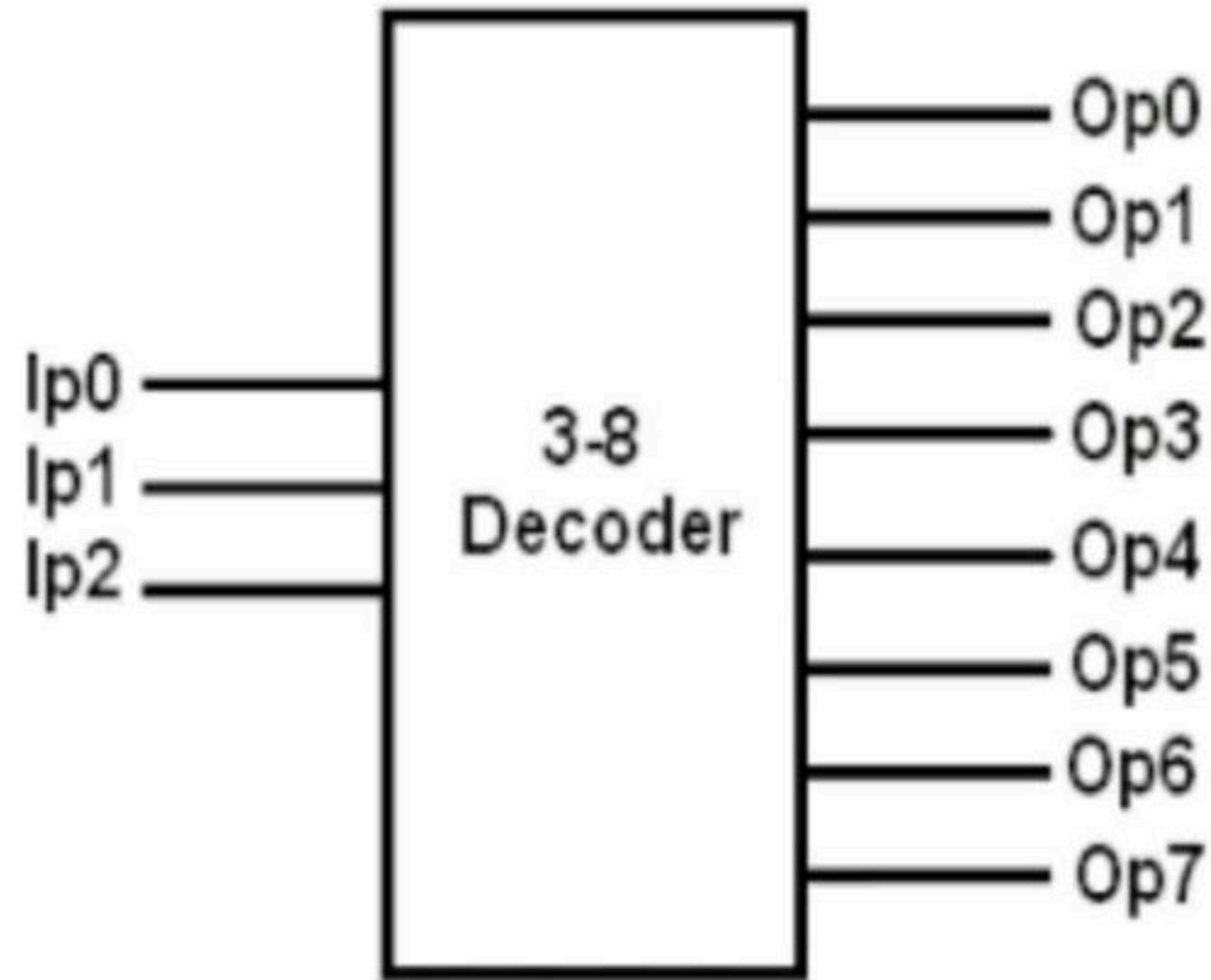


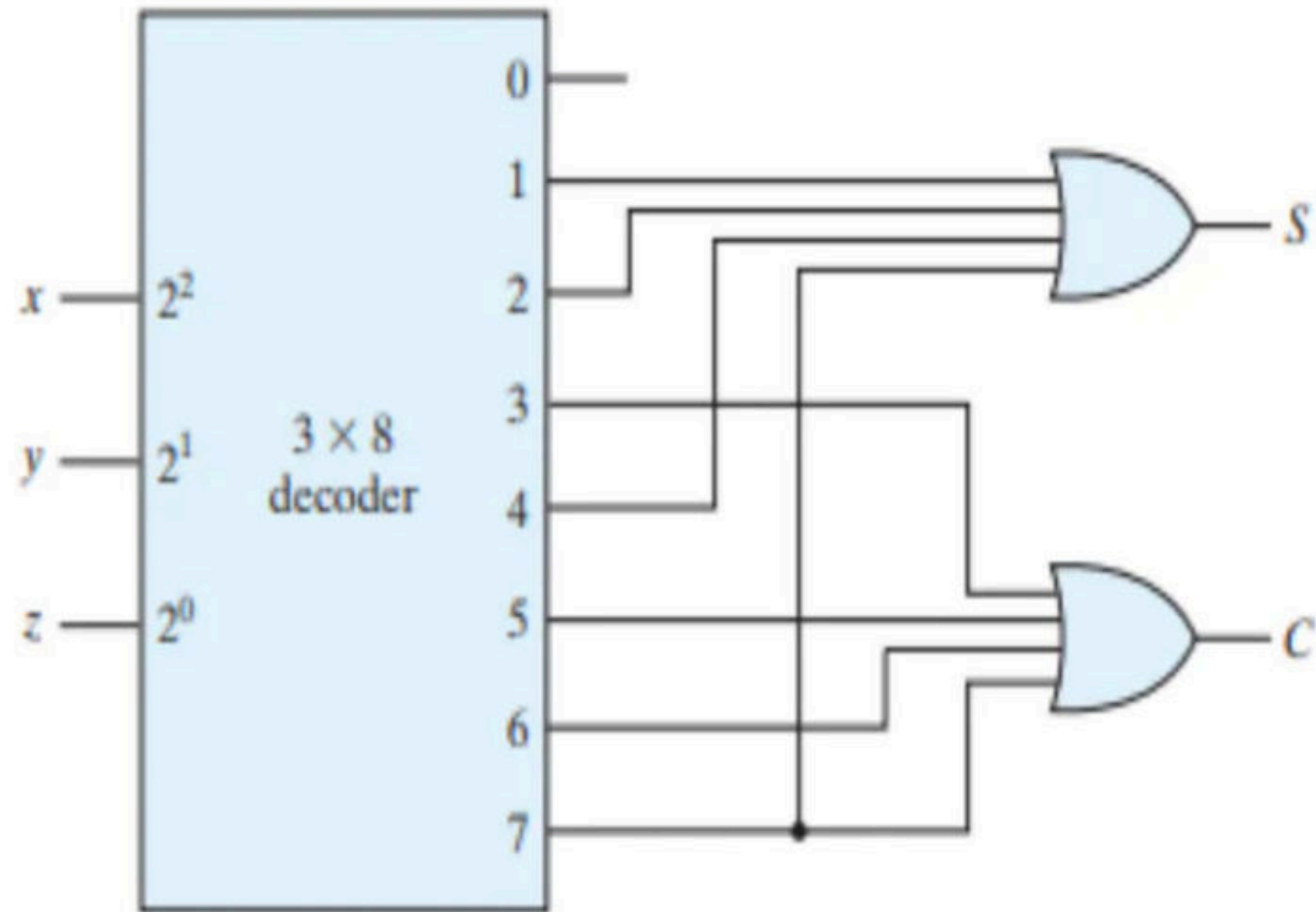
Q Implementation of a full adder with a decoder

From the truth table of the full adder, we obtain the functions for the combinational circuit in sum-of-minterms form:

$$S(x, y, z) = \sum (1, 2, 4, 7)$$

$$C(x, y, z) = \sum (3, 5, 6, 7)$$





Combinational Logic Implementation

- Any combinational circuit with n inputs and m outputs can be implemented with an n -to- 2^n -line decoder and OR gates.
- The procedure for implementing a combinational circuit by means of a decoder and OR gates requires that the Boolean function for the circuit be expressed as a sum of minterms.
- A decoder is then chosen that generates all the minterms of the input variables. The inputs to each OR gate are selected from the decoder outputs according to the list of minterms of each function.

- **Active High Decoder:-** When output is directly from AND gate, we get exact minterms then, it is called active high decoder. In the 2nd level here, we use OR-gate to find the function.
- **Active Low Decoder:-** In active low decoders, output will be from NAND gates, on the 2nd level we again use NAND gate, as we know NAND-NAND implementation is SAME as AND-OR.

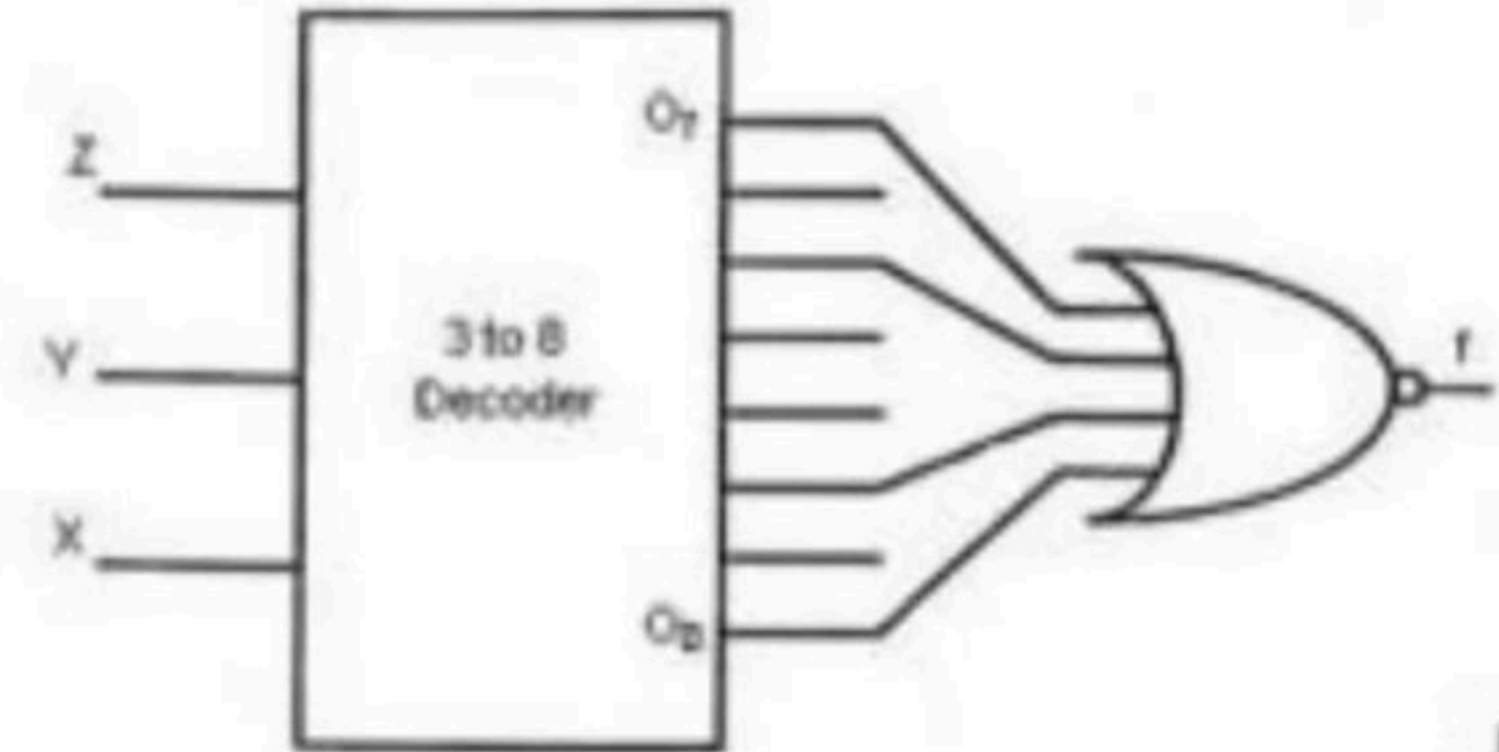
Q What Boolean function does the circuit below realize? (GATE-2006) (2 Marks)

(A) $xz + x'z'$

(B) $xz' + x'z$

(C) $x'y' + yz$

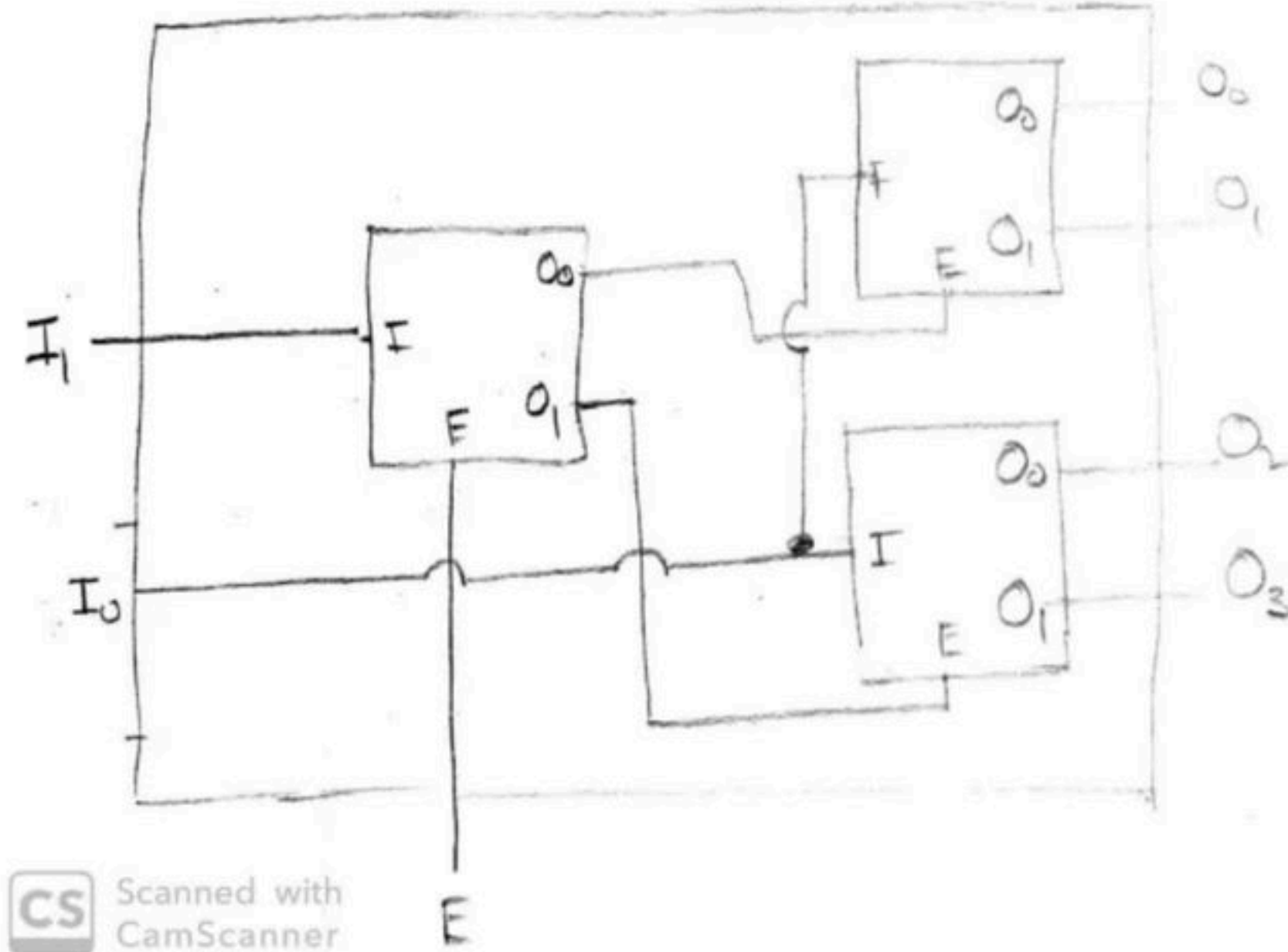
(D) $xy + y'z'$



Decoder Expansion

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

Decoder Expansion



Q 6-to-64 from 2-to-4 ?

Q 7-to-128 from 3-to-8 ?

Target:	$m \times n$
Given:	$p \times q$
No of levels(K)	$\text{floor}(m/p)$
No of Mux at i^{th} level (x_i)	(n/q^{k-i+1})
Total Mux required	$\sum_{i=1}^{i=k} x_i$
Maximum capacity	$(p \times k) \times q^k$

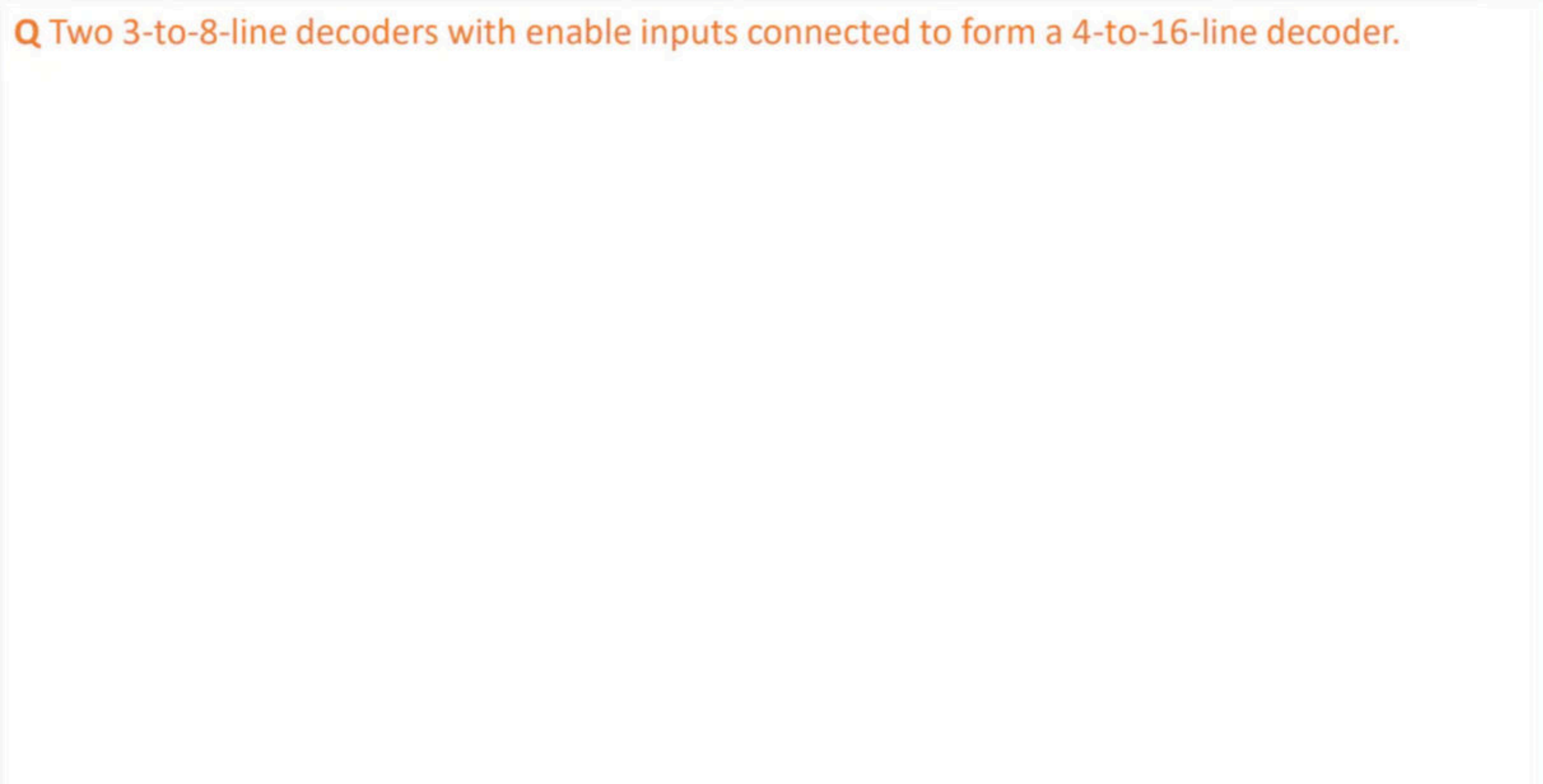
Q How many 3-to-8-line decoders with an enable input are needed to construct a 6-to-64-line decoder without using any other logic gates? **(GATE-2007) (2 Marks)**

(A) 7

(B) 8

(C) 9

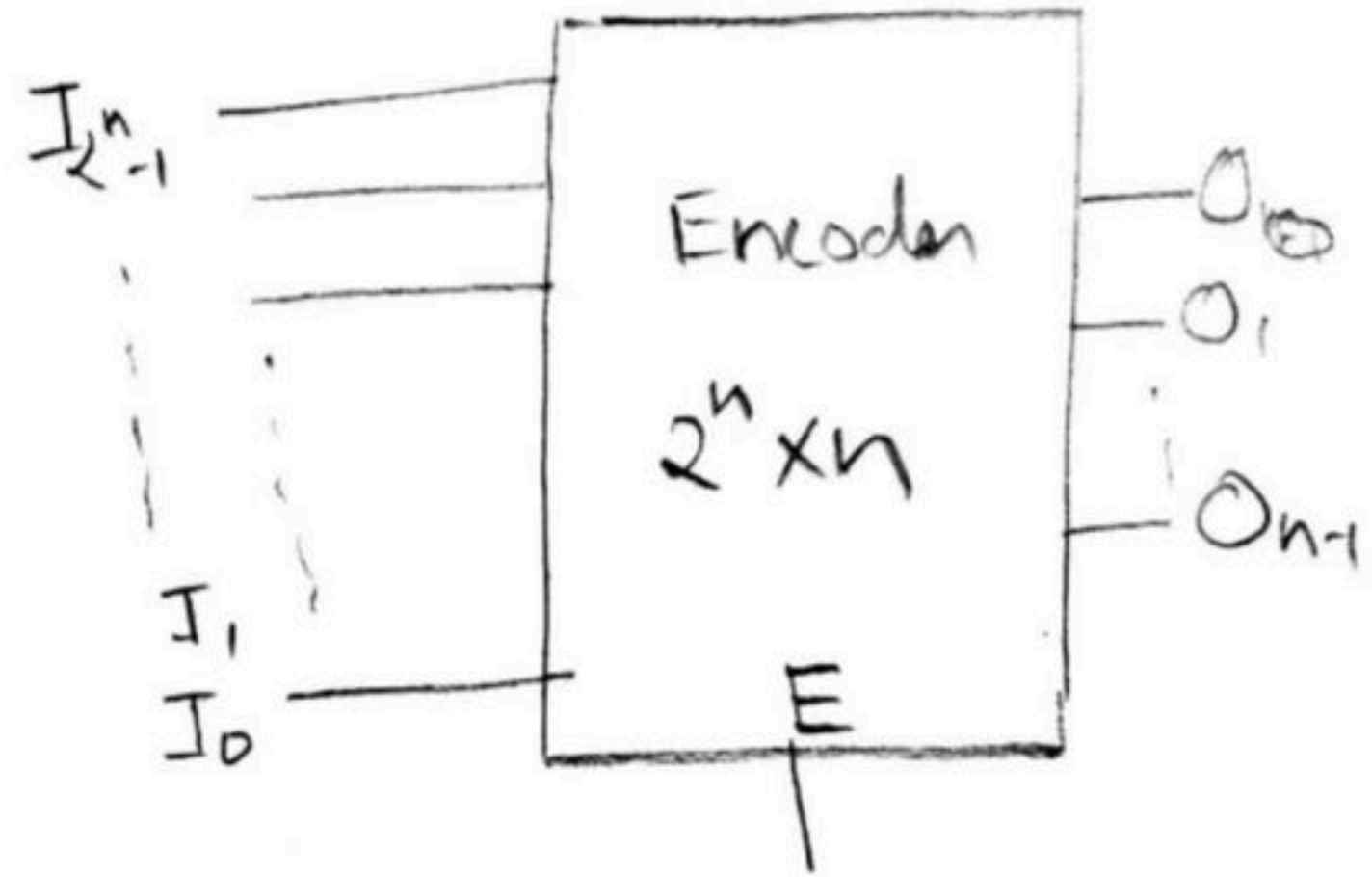
(D) 10



Q Two 3-to-8-line decoders with enable inputs connected to form a 4-to-16-line decoder.

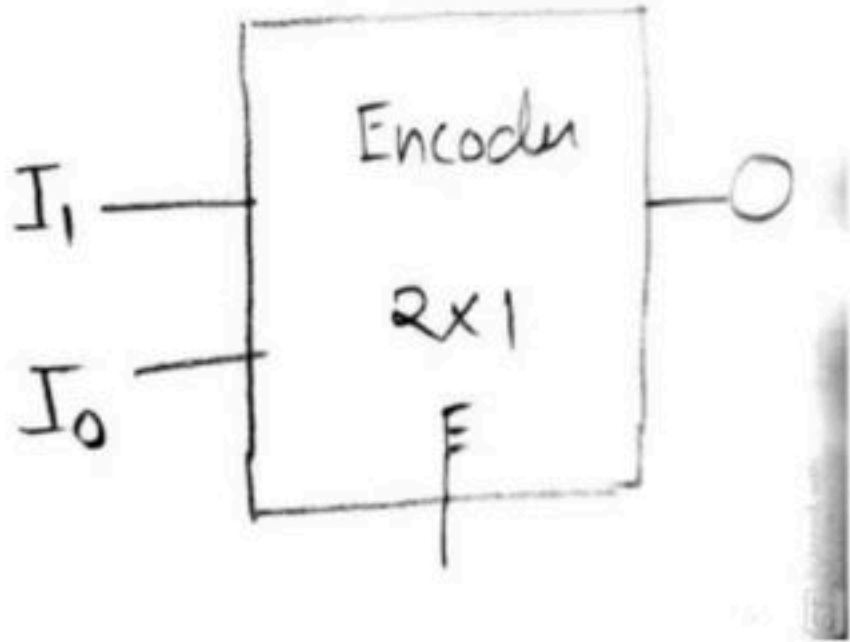
Encoder

- An encoder is a combinational circuit that encode binary information from one of a 2^N input lines and encode it into N output lines, which represent N bit code for the input.
- For simple encoders, it is assumed that only one input line is active at a time.
- Encoder performs the inverse operation of a decoder.



2-to-1 Encoder

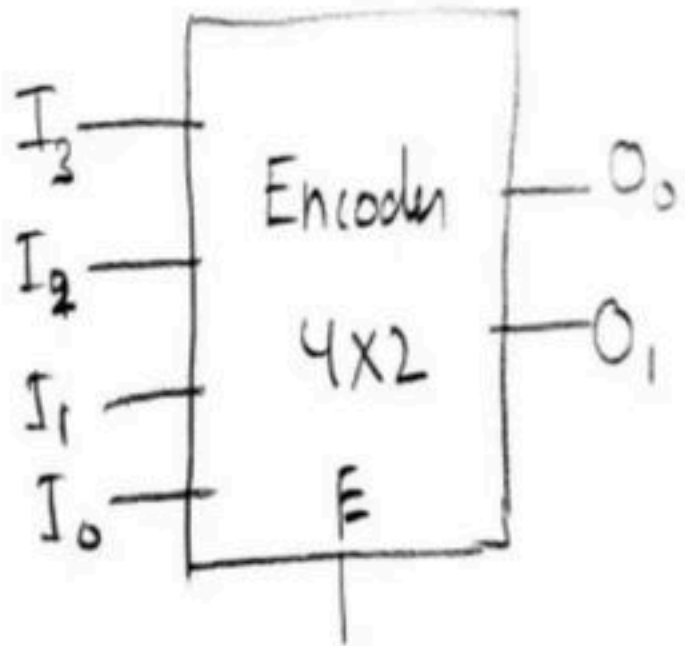
2-to-1 Encoder



I_1	I_0	O_0
0	1	0
1	0	1

4-to-2 Encoder

4-to-2 Encoder



I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

$$\bullet O_0 =$$

$$\bullet O_1 =$$

- $O_0 = I_0' I_1' (I_2 \oplus I_3)$
- $O_1 = I_0' I_2' (I_1 \oplus I_3)$

Priority Encoder

- In some practical cases more than one input can be high at a time, there we can not use simple encoder
- In a priority encoder more than one input can be high at a time. A priority encoder is an encoder circuit that includes the priority function.
- The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.
- They are often used to control interrupt requests by acting on the highest priority interrupt input.
- The *priority encoders* output corresponds to the currently active input which has the highest priority.
- So when an input with a higher priority is present, all other inputs with a lower priority will be ignored.

$$I_3 > I_2 > I_1 > I_0$$

I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1		
0	0	1	d		
0	1	d	d		
1	d	d	d		

$$O_0 =$$

$$O_1 =$$

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

$$I_3 > I_2 > I_1 > I_0$$

I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1	0	0
0	0	1	d	0	1
0	1	d	d	1	0
1	d	d	d	1	1

$$O_0 = I_3 + I_1 I_2'$$

$$O_1 = I_2 + I_3$$

Use Referral Code **KGYT** for Unacademy Plus to Get minimum 10% Discount

Q Match the terms in List - I with the options given in List - II: (NET-JULY-2018)

List - I	List - II
(1) Decoder	(i) 1 line to 2^n lines
(2) Multiplexer	(ii) n lines to 2^n lines
(3) De multiplexer	(iii) 2^n lines to 1 line
	(iv) 2^n lines to 2^n-1 lines

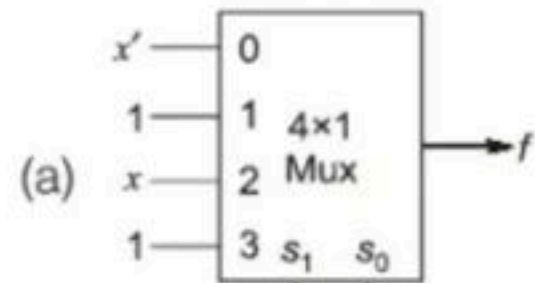
	1	2	3
a)	ii	i	iii
b)	ii	iii	i
c)	ii	i	iv
d)	iv	ii	i

Use Referral Code KGYT for Unacademy Plus to Get minimum 10% Discount

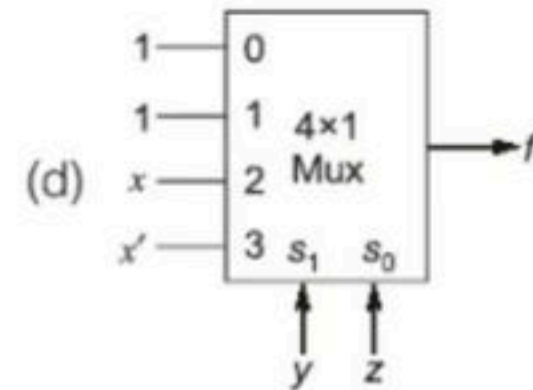
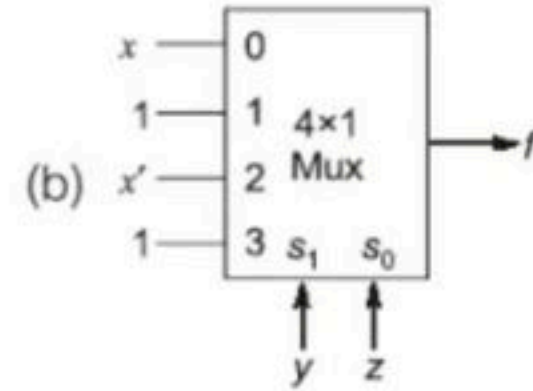
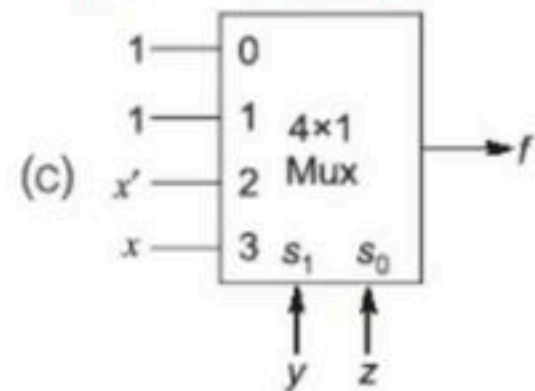
Q In the following truth table, $V = 1$ if and only if the input is valid. **(GATE-2013) (2 Marks)**
What function does the truth table represent?
(A) Priority encoder **(B)** Decoder **(C)** Multiplexer **(D)** Demultiplexer

Inputs				Outputs		
D_0	D_1	D_2	D_3	X_0	X_1	V
0	0	0	0	x	x	0
1	0	0	0	0	0	1
x	1	0	0	0	1	1
x	x	1	0	1	0	1
x	x	x	1	1	1	1

Q.46 Which one of the following circuits implements the Boolean function given below?
 $f(x, y, z) = m_0 + m_1 + m_3 + m_4 + m_5 + m_6$, where m_i is the i^{th} minterm.



(GATE-2021)



Ans. (d)